KLEEM AI / CUSTOMER AGENT OS LITE

Combined HLD and LLD Architecture

Seven high-level design topics and ten low-level design parts

Production-oriented architecture for a multi-tenant customer-service AI platform designed for 10 million registered customer identities, 100,000 concurrent conversations, safe business actions, human takeover, controlled retrieval, and regional recovery.

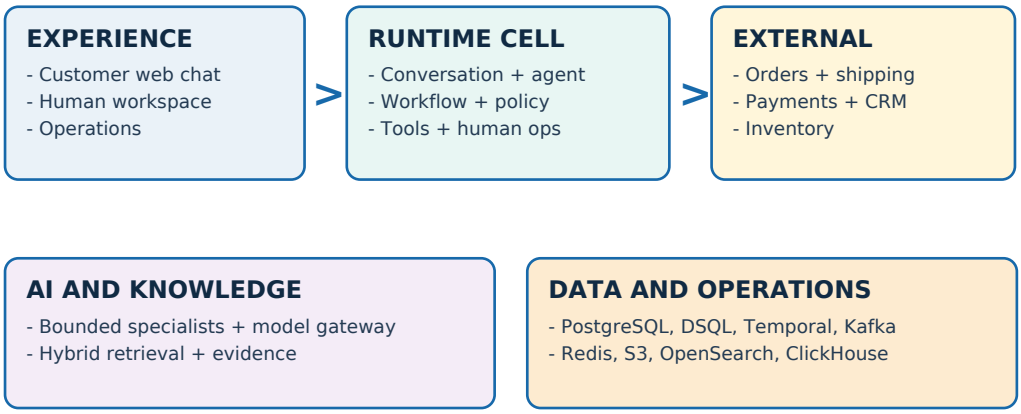
ARCHITECTURE THESIS

The AI interprets, retrieves, reasons, and proposes. Deterministic backend services authenticate, authorize, confirm, approve, execute, reconcile, and record every consequential business action.

Master architecture edition - Version 1.0 - July 24, 2026

How to read this master document

Start with HLD for system boundaries. Use LLD for implementation contracts and failure behavior.



Document layers

HLD	**SEVEN TOPICS**	System boundaries, ownership, scale, reliability, security, AI, and knowledge.
HLD freeze	**VALIDATION**	Amendments, proof obligations, rejected alternatives, and architecture freeze.
LLD 1-3	**DETAILED FROZEN**	Codebase/deployables, TenantContext, and exact data/storage schemas.
LLD 4-10	**CONCEPTUAL BASELINE**	APIs, events, workflows, tools, agent runtime, retrieval, and production proof.

Master contents

Ranges use master PDF page numbers. Each embedded source also retains its original local page numbering.

HLD seven-topic boundary

1. System context and architecture principles
2. Backend services and journey lifecycle
3. Data architecture and communication
4. Cell scaling, multi-region reliability, and disaster recovery
5. Security, identity, privacy, and tenant isolation
6. AI runtime, orchestration, and bounded specialists
7. Knowledge, models, safety, evaluation, and observability

Frozen cross-cutting invariants

TENANT ISOLATION	Every row, key, object, event, workflow, index, cache, trace, and decision is tenant-and-environment scoped.
AI AUTHORITY	The model can interpret, retrieve, reason, and propose. It cannot authenticate, authorize, confirm, approve, or execute.
CONSEQUENTIAL ACTIONS	Policy, exact preview, confirmation, approval when required, one-time capability, provider idempotency, ledger, and verification are mandatory.
TRUTHFUL STATUS	No completed customer state exists before the authoritative business system confirms the outcome.
DUPLICATE SAFETY	At-least-once delivery is expected. Outbox/inbox, stable idempotency, one-time redemption, and reconciliation prevent duplicate effects.
VALIDATED DELIVERY	Raw model tokens are not customer output. Generated content is grounded and validated before delivery.
WORKFLOW OWNERSHIP	Temporal owns durable multi-step journeys; Kafka carries facts; databases and provider systems retain declared truth.
REGIONAL SAFETY	One active writer per tenant; routing epochs fence old writers before failover traffic or protected actions.
BOUNDED AI	One visible agent, one orchestrator, five fixed specialists, typed outputs, explicit budgets, no unbounded swarm, and no stored chain-of-thought.
PRODUCTION PROOF	Security, evaluation, load, chaos, canary, rollback, and recovery evidence are release requirements.

V1 explicit exclusions include direct model write tools, one global runtime, active-active writes for one tenant, exactly-once infrastructure claims, thirty microservices on day one, vector search as universal memory, A2A between internal specialists, raw chain-of-thought storage, and automated damage-image judgment.

CUSTOMER AGENT OS LITE

Combined High-Level Architecture

Backend platform and AI agent architecture for a 10M-user design target

Seven HLD topics | V1 architecture baseline

Version 1.0 - July 2026

Document purpose and authority

This document combines the complete high-level design for Customer Agent OS Lite. It converts the frozen product definition into a production-oriented backend and AI architecture. It is intentionally technology-specific enough to guide implementation, while leaving schemas, endpoint definitions, event payloads, prompt templates, and code structure to the Low-Level Design.

Architecture thesis The AI interprets, retrieves, reasons, and proposes. Deterministic backend services authenticate, authorize, confirm, approve, execute, reconcile, and record every consequential business action.

Precedence	Source	Authority in this HLD
1	Customer Agent OS Lite - V1 Product Contract	Controls all V1 journeys, thresholds, confirmations, human boundaries, outcome states, and acceptance criteria.
2	Customer Agent OS Lite - Fixed Product Features	Controls the complete product roadmap and the ten fixed product sequences.
3	Customer Agent OS Lite - Project Overview	Controls product vision, users, value proposition, and long-term direction.

Interpretation rule

If statements conflict, the higher-precedence source wins. Address change remains a subworkflow, not an eighth customer journey. V1 accepts image uploads as human-review evidence; automated visual damage assessment remains post-V1 unless explicitly promoted through a product-contract revision.

Seven-topic boundary

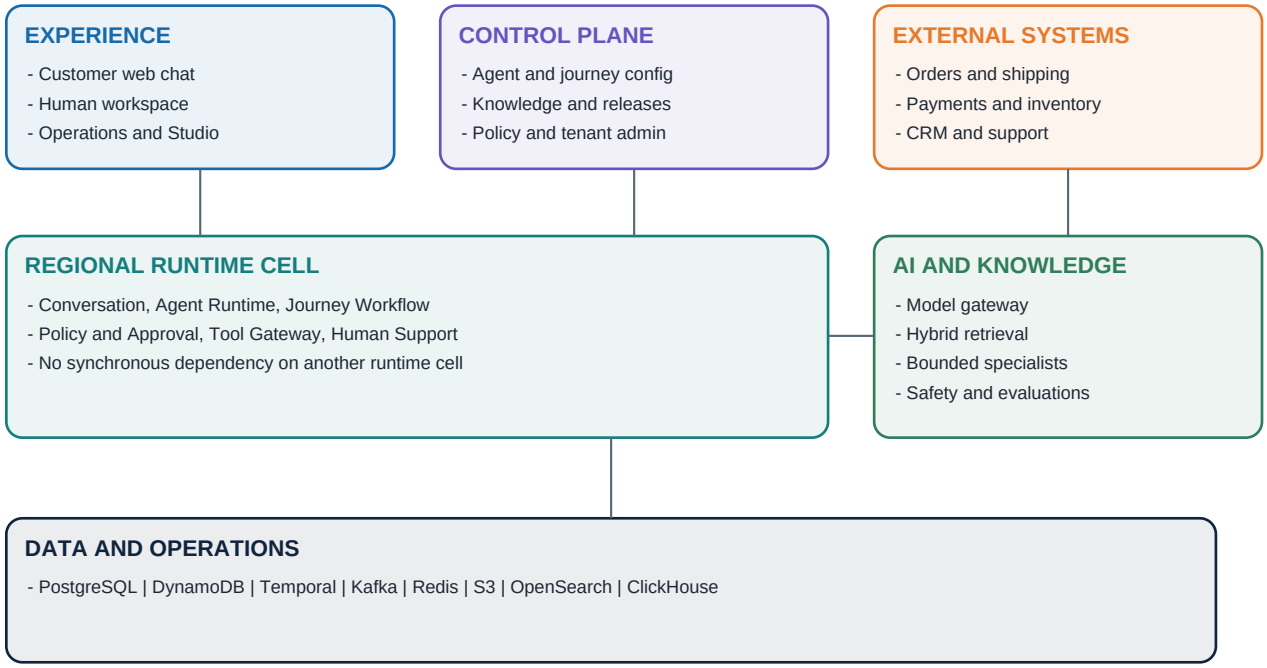
Topic	HLD chapter
1	System context and architecture principles
2	Backend services and journey lifecycle
3	Data architecture and communication
4	Cell scaling, multi-region reliability, and disaster recovery
5	Security, identity, privacy, and tenant isolation
6	AI runtime, orchestration, and bounded specialists
7	Knowledge, models, safety, evaluation, and observability

The combined architecture, journey traceability, decision register, and HLD exit criteria appear after Topic 7. They close the design and do not introduce another HLD topic.

Contents

Executive architecture summary

Customer Agent OS Lite is a multi-tenant platform for building and operating customer-facing AI agents. The e-commerce support agent is the first complete reference application. The platform must resolve customer outcomes, not merely generate fluent messages, and it must remain truthful when a model, tool, workflow, queue, cell, or region fails.



The architecture separates runtime cells from the control plane. A tenant is assigned one home region and one active runtime cell. Conversations are pinned to immutable release identifiers. AI execution is bounded by deterministic policy, explicit confirmation, approval, action authorization, and authoritative business-system verification.

Dimension	Baseline
Scale target	10M customer identities, thousands of tenants, 100k concurrent conversations.
Peak traffic	5,000 messages/second expected peak; 15,000 messages/second short burst design target.
Regional topology	Three active regions; approximately ten active plus two warm-spare cells per region.
Reliability	Runtime availability target 99.95%; control plane 99.9%; graceful degradation over false success.
AI model	One visible support agent with bounded internal specialists; no autonomous unbounded swarm.
Business correctness	No action without required confirmation, no intentional duplicate effect, and no completed state before authoritative confirmation.
Reference cloud	AWS with provider-portable service boundaries and open application contracts.

Honest scale claim The system is designed and load-tested against the 10M-user model. The project must not claim that ten million real users have used it.

01

System context and architecture principles

Define what the platform is, who trusts it, where state lives, and which decisions are always deterministic.

THIS TOPIC FREEZES

- Product and system boundaries
- Control plane versus runtime data plane
- AI authority versus backend authority
- Tenancy, release, and regional placement model
- Reference technology platform and portability boundary

1. System context and architecture principles

1.1 Product boundary

The product contains three customer-visible experiences: web chat, a human support workspace, and minimum operations/administration controls. It also contains the reusable platform capabilities needed to define, release, operate, evaluate, and improve agents.

V1 serves e-commerce support through web chat. It supports order tracking, return eligibility, return creation, damaged replacement, refunds, subscription cancellation, and human escalation. Voice, additional industries, a free-form visual agent builder, autonomous high-risk actions, and automated image judgment are not V1 commitments.

Actor	Primary responsibility	Trust level
Customer	Requests information or outcomes; uploads evidence; confirms actions; requests a human.	Untrusted external principal until verified.
Support representative	Takes over cases, communicates, and performs permitted actions.	Authenticated tenant workforce principal.
Approver	Approves, rejects, or modifies bounded action requests.	Workforce principal with scoped authority.
Operations manager	Configures journeys, policies, knowledge, queues, tests, and releases.	Privileged tenant administrator.

Actor	Primary responsibility	Trust level
Developer	Builds integrations and reusable capabilities through SDK and configuration-as-code.	Privileged machine and workforce principal.
Platform operator	Operates shared infrastructure without implicit access to tenant business data.	Highly privileged, audited platform principal.

1.2 Architectural laws

- Backend-owned truth: the backend owns customer-visible state, workflow progress, approvals, action execution, and completion status.
- AI is advisory: model outputs are untrusted proposals until schema validation, policy evaluation, authorization, and workflow state permit them.
- Durable work is explicit: multi-step or waiting journeys run in a durable workflow engine, not inside a model loop or request process.
- Single-writer tenancy: one home region and active cell write a tenant's operational state at a time, protected by routing epochs and fencing tokens.
- Immutable releases: every conversation is pinned to exact agent, prompt, policy, knowledge, model-route, tool-contract, and evaluation release identifiers.
- At-least-once by design: events and retries may repeat; consumers and actions must be idempotent.
- Truthful degradation: pending, unknown, review-required, and recoverable-failure states are valid results. False completion is not.
- Tenant context is mandatory: tenant and environment identifiers originate from trusted authentication/routing context, never from an unchecked request field.

Non-negotiable boundary

A model can recommend a $75 refund. It cannot authorize, submit, or declare that refund complete. Those are separate deterministic transitions owned by policy, workflow, tool, and business-system components.

1.3 Platform planes

Plane	Responsibilities	Availability behavior
Experience	Customer chat, human workspace, operations interfaces, channel session handling.	Regional, horizontally scalable, stateless where possible.
Runtime data plane	Conversation, agent runtime, workflows, policies, tools, approvals, human cases.	Cell-local, low latency, no synchronous cross-cell dependency.
Control plane	Tenant admin, configuration, knowledge ingestion, release creation, evaluation gates.	Separate SLO; runtime continues on pinned releases if unavailable.
Data and event plane	Operational truth, durable workflow state, event backbone, search, artifacts, analytics.	Per-store recovery and replication policy.
AI plane	Model gateway, retrieval, context construction, specialists, safety, evaluation and trace capture.	Provider-fallback capable; never bypasses backend controls.

1.4 Reference platform mapping

Capability	Reference implementation	Portability boundary
Compute	Amazon EKS, managed node groups, Karpenter	Kubernetes workloads and OCI images.
Relational control data	Aurora PostgreSQL	PostgreSQL SQL and migration contracts.
High-volume runtime data	DynamoDB and Global Tables	Repository abstraction and explicit key design.
Durable workflows	Temporal Cloud	Workflow/activity interfaces; avoid cloud-specific business logic.
Event streaming	Amazon MSK and MSK Replicator	Kafka protocol and schema registry.
Cache and limits	ElastiCache for Valkey/Redis	Disposable cache interface.
Artifacts	Amazon S3	Object-storage abstraction and signed access.
Retrieval	Amazon OpenSearch Service	Search/retrieval contract with rebuildable index.
Analytics	ClickHouse	SQL event and trace warehouse.
Telemetry	OpenTelemetry plus managed metrics/logs/traces	OpenTelemetry semantic conventions.
Models	Multi-provider AI gateway	Provider-neutral request, policy, cost, and fallback contract.

02

Backend services and journey lifecycle

Assign every state transition to one service and trace a customer request through confirmation, approval, execution, reconciliation, and handoff.

THIS TOPIC FREEZES

- Logical service ownership
- Initial physical deployables
- Synchronous and asynchronous boundaries
- Durable journey state model
- Consequential-action protocol

2. Backend services and journey lifecycle

2.1 Logical services and ownership

Service	Owns	Must not own
Customer Chat Gateway	Web session, admission control, message receipt, streaming connection.	Business workflow truth or tool credentials.
Operations Gateway	Human and admin API composition, UI-specific read models.	Primary business state.
Identity and Tenant Context	Principal, tenant, environment, role, region/cell routing context.	Journey decisions.
Conversation Service	Ordered messages, participants, attachments, visible status, AI/human control.	Refund or replacement completion truth.
Agent Runtime	Intent interpretation, context request, specialist invocation, structured proposal, response generation.	Direct business writes, approvals, or completion state.
Journey Workflow	Durable journey state, timers, retries, signals, confirmation, approval waiting, outcomes.	Provider credentials or policy authorship.
Policy and Approval	Deterministic eligibility, threshold and risk decision; approval request lifecycle.	Tool execution.
Tool and MCP Gateway	Tool schemas, credentials, narrow action authorization, idempotency, circuit breakers, action ledger.	Customer conversation or model reasoning.
Human Support	Queues, assignment, takeover, approvals, internal notes, return of control.	AI prompt or model routing.
Release and Configuration	Immutable release bundles and staged rollout state.	Live conversation mutation.
Knowledge	Source metadata, ingestion status, versioning, effective dates, retrieval index publication.	Business action authorization.
Audit and Evaluation	Audit projection, evaluation runs, traces, quality/cost metrics.	Transactional workflow truth.

2.2 Initial deployable boundary

The logical model is deliberately richer than the initial deployment model. V1 should begin with approximately seven independently deployable units, not thirty microservices. Modules retain ownership boundaries so they can split when scaling, security, or team ownership demands it.

Deployable	Contained logical capabilities
Edge/API	Customer and operations gateways, authentication adapters, streaming sessions.
Conversation Runtime	Conversation service, runtime projections, attachment coordination.
Agent Runtime	Context assembly, orchestrator, specialists, model gateway client, response streaming.
Workflow Workers	Journey workflows, policy activities, approval waits, reconciliation jobs.
Integration Gateway	MCP/tool adapters, credentials, idempotency ledger, provider circuit breakers.
Human Operations	Cases, queues, assignments, approvals, takeover and shared context.
Control and Knowledge	Tenant configuration, releases, knowledge ingestion, eval management and admin APIs.

2.3 Request and action lifecycle

Short reads and message acknowledgement may complete synchronously. Long work, approvals, external waiting, retries, reconciliation, analytics, evaluations, and notifications execute asynchronously. The customer must receive a durable workflow state rather than remain attached to a fragile HTTP request.

2.4 Universal journey state model

State	Entry condition	Permitted next states
Collecting information	Required identity, order, item, reason, or evidence is missing.	Checking, unable to complete, needs human review.
Checking	Authoritative knowledge or business data is being retrieved or validated.	Collecting, ready for confirmation, pending, review, failure.
Ready for confirmation	Exact action preview is stable and no action has occurred.	Processing, collecting, cancelled, review.
Processing	Confirmed and authorized action is being submitted.	Completed, pending external event, needs review, recoverable failure.
Completed	Business system confirmed the intended outcome and returned a reference.	Terminal or post-completion follow-up.
Pending external event	Waiting for customer, human, carrier, inventory, payment, or another system.	Checking, processing, completed, review, failure.
Needs human review	Approval, takeover, policy conflict, risk, or explicit customer request.	Processing, pending, completed, unable to complete.
Failed - recoverable	A safe retry or reconciliation path exists.	Checking, processing, pending, review.
Unable to complete	No safe automated continuation exists.	Human escalation or terminal.

2.5 Consequential-action protocol

Confirmation binding

Confirmation is bound to a hash of the exact preview: action type, target, amount or effect, destination, timing, policy version, workflow version, and expiry. Any material change invalidates confirmation and requires a new preview.

- The workflow creates a proposed action and stable idempotency key.
- Policy returns allow, require approval, require takeover, or deny, with reason codes and policy version.
- The action authorization binds tenant, environment, workflow, action, target, preview hash, confirmation, approval, expiry, and routing epoch.
- The Tool Gateway validates authorization and records the action ledger before submission.
- A timeout after submission becomes unknown or pending. Reconciliation queries the provider using the same operation identity; it never blindly repeats a financial or fulfillment action.
- Completed is emitted only after an authoritative response or later reconciliation confirms the outcome and reference.

2.6 Damaged replacement reference flow

Step	Owner	State and event
1	Conversation	Accept and deduplicate request; MessageAccepted.
2	Agent Runtime	Identify damaged-replacement journey and requested address change.
3	Workflow	Verify customer/order; collect item, description, and evidence when required.
4	Knowledge + tools	Retrieve effective policy, claim history, inventory, address constraints.
5	Policy	Evaluate eligibility, risk, evidence, item value, and fulfillment stage.
6	Workflow	Display item, destination, return obligation, timing, and exact preview.
7	Customer	Explicitly confirms the frozen preview.
8	Approval	Automatic when low risk and value <= $250; otherwise human approval.
9	Tool Gateway	Reserve/create with stable idempotency key and authorization token.
10	Business system	Returns confirmed reference, pending state, or unknown result.
11	Workflow	Publish completed only after confirmation; otherwise reconcile or hand off.

03

Data architecture and communication

Choose one source of truth for each kind of state, make event delivery repeatable, and keep analytical and retrieval stores rebuildable.

THIS TOPIC FREEZES

- Authoritative data ownership
- Polyglot persistence responsibilities
- Event envelope and topic strategy
- Consistency, ordering, and idempotency model
- Protocol selection and regional data model

3. Data architecture and communication

3.1 Storage responsibility map

Store	Authoritative data	Not authoritative for
Aurora PostgreSQL	Tenants, users, roles, environments, agents, journeys, policies, tools, releases, queues, cases, approvals, knowledge metadata.	High-volume messages, workflow execution history, analytics.
DynamoDB	Conversation messages and metadata, runtime projections, message/action idempotency records, action ledger.	Policy authorship, search index, aggregate analytics.
Temporal	Durable workflow execution, timers, retries, signals, activity history.	UI query model or business-system truth.
Kafka/MSK	Ordered domain-fact transport and replay log within retention.	Primary command handling or permanent business record.
Redis/Valkey	No primary truth; cache, rate limits, presence, circuit state, release cache.	Messages, approvals, action status.
S3	Evidence, knowledge originals, release bundles, evaluation datasets, long-term audit/trace archives.	Low-latency mutable workflow state.
OpenSearch	Rebuildable lexical, vector, hybrid retrieval and operational search index.	Original documents or permissions.
ClickHouse	Analytical projections for traces, costs, latency, evaluations, support outcomes.	Transactional decisions.

3.2 Consistency contract

A service uses a strongly consistent local transaction for its aggregate and writes an outbox record in that same transaction. Outbox publishing to Kafka is at-least-once. Consumers deduplicate by event ID and reject or reconcile stale aggregate versions.

Cross-service state is eventually consistent. User interfaces read purpose-built projections that contain source version and freshness metadata. A projection may lag; it may not invent completion. For critical action status, the read path can query the action ledger or authoritative business adapter directly.

Problem	Design
Duplicate message	Client message ID plus tenant/conversation uniqueness condition.
Duplicate event	Globally unique event ID and idempotent consumer inbox.
Out-of-order event	Aggregate version check; buffer, retry, or rebuild projection.
Duplicate action	Stable business idempotency key, action ledger, provider idempotency where available, and status lookup.
Concurrent workflow command	Workflow run ID plus expected state/version; Temporal serializes workflow state.
Region split-brain	Tenant routing epoch/fencing token included in every consequential command.
Schema evolution	Backward-compatible additions; versioned event type for breaking semantic changes.

3.3 Canonical domain event envelope

Field	Purpose
event_id	Unique delivery identity for deduplication.
event_type / schema_version	Business fact name and evolution contract.
tenant_id / environment_id	Mandatory isolation and routing dimensions.
aggregate_type / aggregate_id / aggregate_version	Ordering and optimistic projection control.
occurred_at / producer	Business time and accountable source.
correlation_id / causation_id / trace_id	Connect request, workflow, event chain, and distributed trace.
routing_epoch	Reject writes from a fenced region or cell.
data_classification	Redaction, retention, residency, and access handling.
payload	Versioned domain fact; never raw model reasoning or credentials.

Event naming

Events are past-tense business facts such as MessageAccepted, ConfirmationRecorded, ApprovalRequested, ActionSubmitted, RefundConfirmed, ReplacementPending, or HumanTookControl. Commands remain imperative and point to one owner.

3.4 Kafka topology

Topic family	Partition key	Example events
conversation	tenant_id + conversation_id	MessageAccepted, ParticipantChanged, ConversationStatusChanged.
workflow	tenant_id + workflow_id	JourneyStarted, ConfirmationRecorded, WorkflowWaiting, JourneyCompleted.
tool-action	tenant_id + action_id	ActionAuthorized, ActionSubmitted, ActionUnknown, ActionConfirmed.
human-support	tenant_id + case_id	CaseQueued, Assigned, ApprovalDecided, HumanTookControl.
release	tenant_id + release_id	ReleasePublished, CanaryExpanded, ReleaseRolledBack.
knowledge	tenant_id + source_id	SourceSynced, IndexPublished, KnowledgeConflictDetected.
evaluation	tenant_id + eval_run_id	EvalStarted, CaseScored, ReleaseGateDecided.

Do not create one topic per tenant. Tenant identity is carried in the event, key, authorization context, quota, and observability dimensions. Sensitive payloads use references to encrypted objects when practical rather than duplicating PII through every topic.

3.5 Communication matrix

Interaction	Protocol	Rule
Client commands	HTTPS/JSON	Idempotency key for message and action-intent submissions.
Response streaming	SSE initially; WebSocket where bidirectional presence is needed	Stream tokens only after output checks; persist final message.
Internal short call	gRPC or typed HTTP	Strict deadline, retry only for safe/idempotent operations.
Domain facts	Kafka	At-least-once delivery; schema registry; consumer inbox.
Durable orchestration	Temporal workflow/activity/signal	No business waiting in request threads or model loops.
Business integrations	MCP through Tool Gateway plus REST/webhooks	MCP describes tools; gateway owns security and reliability.
Files	Presigned S3 upload/download	Malware scan, content validation, size/type limits, short expiry.
Analytics	Kafka to ClickHouse/S3	Asynchronous; never on transactional critical path.

04

Cell scaling, multi-region reliability, and disaster recovery

Scale by adding isolated runtime cells, protect the platform from hot tenants and provider failures, and fail over without duplicate customer outcomes.

THIS TOPIC FREEZES

- 10M-user capacity model
- Cell topology and tenant placement
- Autoscaling, quotas, priorities, and load shedding
- Multi-region replication and split-brain prevention
- Failure behavior, SLOs, RPO/RTO, and canary rollout

4. Cell scaling, multi-region reliability, and disaster recovery

4.1 Capacity assumptions

Metric	Design assumption	Validation
Customer identities	10,000,000	Synthetic tenant and identity distribution.
Tenants	Thousands, with long-tail and hot-tenant distribution	Placement and noisy-neighbor tests.
Concurrent conversations	100,000	Connection, workflow, and model-concurrency load tests.
Message peak	5,000/sec; 15,000/sec short burst	Sustained and burst scenarios with backpressure.
Complex journey tools	3 to 6 calls	Failure injection on each tool boundary.
Runtime availability	99.95% target	SLI from admitted message to durable state/response.
Control-plane availability	99.9% target	Runtime remains on last approved pinned release.
Regions	Three active regions	Cell and regional evacuation exercises.

4.2 Cell architecture

A cell is a bounded, independently operated unit containing gateways, conversation/runtime services, workflow and tool workers, human-support workers, task queues, policy/release caches, and assigned tenant data partitions. A runtime request never synchronously depends on another cell.

Baseline	Value
Regions	3
Cells per region	10 active + 2 warm spare
Total	30 active + 6 warm = 36
Per-cell identity envelope	250k to 500k identities
Per-cell concurrency	5,000 active conversations
Per-cell throughput	250 messages/sec sustained; 750/sec burst
Normal utilization	At or below 60% to 65%
Emergency ceiling	At or below 85% during evacuation
Normal active capacity	150k concurrent; 7,500 messages/sec sustained
After one-region loss	20 active + 4 warm: 120k concurrent; 6,000 messages/sec sustained

Tenant placement considers home geography, residency, cell headroom, model/provider quota, integration locality, and tenant size. A hot or enterprise tenant can move to a dedicated cell with dedicated worker pools, cache, workflow namespace, tables, or account where justified.

4.3 Autoscaling and backpressure

Subsystem	Primary scaling signals
Gateway	Active streams, accepted RPS, connection churn, buffered bytes, admission latency.
Conversation	Queue depth/age, write latency, conditional-write contention.
Agent Runtime	Request queue age, concurrent model calls, time to first token, token throughput.
Workflow	Temporal task backlog, schedule-to-start latency, open workflow count.
Tool Gateway	In-flight calls, downstream p95/p99, quota remaining, circuit state.
Kafka consumers	Consumer lag and oldest event age, not CPU alone.
Retrieval	Query QPS, p95 latency, rejected queries, shard utilization.

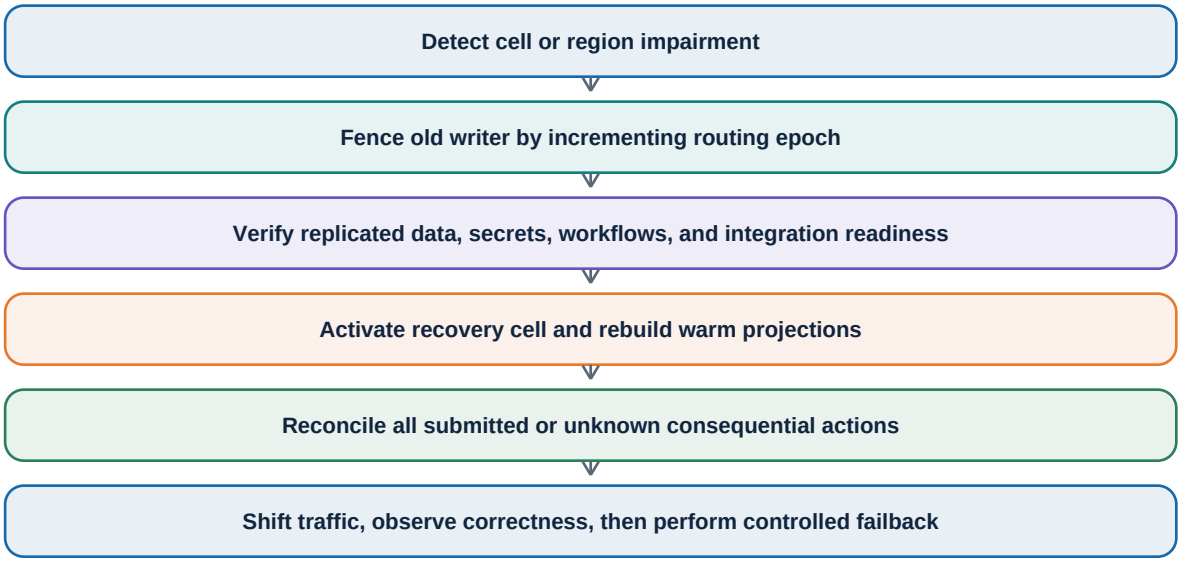
Kubernetes HPA, Karpenter, and cluster autoscaling provide compute elasticity. Per-tenant controls cover RPS, concurrent conversations, model tokens, workflow starts, tool calls, uploads, evaluations, and batch work. Bulkheads isolate tenant, provider, tool, and read/write workloads.

Priority	Work	Overload behavior
P0	Reconcile already-submitted consequential actions	Never intentionally shed.
P1	Confirmations and interactive journey progression	Protect reserved capacity.
P2	Human support and approvals	Protect; degrade enrichment first.
P3	Ingestion and customer-visible asynchronous updates	Delay within freshness SLO.
P4	Evaluations, analytics, and batch work	Pause first.

4.4 Multi-region and fencing

Each tenant has one active home region and cell. The routing record contains tenant_id, home_region, active_cell, failover region/cell, residency policy, routing_epoch, and status. The edge and regional routers cache it. The routing epoch is a fencing token included in every consequential command; a region holding an old epoch cannot write even if it remains network-reachable.

Data system	Regional design
DynamoDB	Global Tables for messages/projections; strongest practical coordination plus reconciliation for action ledger.
Aurora PostgreSQL	Aurora Global Database with one writer for control-plane data.
Kafka/MSK	Asynchronous regional replication; accept duplicates and lag.
Temporal	Managed multi-region HA preferred; workflow failover rehearsed.
S3	Versioning and cross-region replication for evidence, releases, datasets, and audit archives.
OpenSearch	Replicate or rebuild from authoritative content and metadata.
ClickHouse	Asynchronous replicated analytics; not on recovery critical path.



4.5 Recovery objectives

Failure	RPO	RTO target
Pod or node	0	< 1 minute
Availability zone	0	< 5 minutes
Runtime cell	< 1 minute	< 10 minutes
Region	< 1 minute	< 20 minutes
Control plane	< 5 minutes	< 30 minutes
Analytics	15 minutes	< 1 hour

Business recovery invariant

Recovery metadata may lag, but the platform must preserve no duplicate effect and no false completion. Unknown submitted actions are reconciled before a replacement command is allowed.

4.6 Failure behavior

Failure	Customer-safe behavior
Primary model unavailable	Route to allowed fallback; otherwise provide bounded status/help and offer human handoff.
Business tool unavailable	Open circuit, keep workflow pending/recoverable, communicate next step.
Timeout after submission	Mark unknown; reconcile by provider reference/idempotency key; no blind retry.
Redis unavailable	Use primary stores with reduced throughput; rate limiting fails according to risk policy.
Kafka unavailable	Commit local state plus outbox; publish after recovery; cap backlog with admission control.
OpenSearch unavailable	Use safe cached authoritative content where permitted; otherwise escalate and do not guess.
Temporal unavailable	Accept/persist conversation messages where safe; pause new consequential transitions.
Control plane unavailable	Continue on pinned immutable release; block new publishing/config mutations.
Analytics unavailable	Queue or archive telemetry; never block customer transaction.

4.7 SLIs and release rollout

SLI	Healthy target
Message acknowledgement p95	< 300 ms
Runtime dispatch p95	< 500 ms
Visible progress	< 1 second
First token with healthy provider p95	< 2.5 seconds
Order tracking outcome p95	< 5 seconds
UI update after external confirmation p95	< 2 seconds

A release proceeds through development, staging, shadow evaluation, internal use, one canary cell, 5%, 25%, and all eligible cells. Regression, safety, cost, latency, and business-correctness gates can stop or roll back the release. Conversations remain pinned unless a security emergency requires a controlled migration.

05

Security, identity, privacy, and tenant isolation

Make every principal, tenant, model call, document, workflow, and business action prove its authority at every trust boundary.

THIS TOPIC FREEZES

- Authentication and workload identity
- RBAC plus ABAC authorization model
- Defense-in-depth tenant isolation
- Action authorization and confirmation integrity
- LLM-specific threat controls, privacy, audit, and incident response

5. Security, identity, privacy, and tenant isolation

5.1 Trust boundaries and principal types

Principal	Authentication	Authorization scope
End customer	Tenant identity federation, commerce session, email/OTP, or anonymous session for public flows.	Conversation and verified customer/account/order resources only.
Tenant workforce	OIDC/SAML SSO with MFA and tenant membership.	Role, queue, environment, resource, risk, and action attributes.
Platform workforce	Corporate SSO, phishing-resistant MFA, just-in-time elevation.	Least privilege; no ambient tenant-data access; all access audited.
Service workload	Short-lived workload identity, preferably SPIFFE-compatible/mTLS.	Service-to-service policy and narrow resource capability.
Integration	OAuth client, API key, mTLS, or signed webhook kept in secrets service.	Specific tenant, environment, tool, operations, and endpoint.

Tokens are short-lived and audience-bound. Tenant and environment claims are derived from verified membership and routing context. Step-up authentication is required for sensitive data, account changes, address changes, or policies that demand stronger identity.

5.2 Authorization model

Use RBAC for understandable job roles and ABAC for resource, environment, risk, ownership, queue, amount, and action attributes. A centralized policy decision interface returns allow/deny plus obligations; enforcement occurs at every gateway and resource-owning service. Cedar or Open Policy Agent is suitable, but the policy language choice remains an LLD decision.

Role	Representative permissions
Support Agent	Read assigned cases, join conversations, use approved knowledge, perform low-risk human actions.
Approver	Review and decide action classes/amounts within assigned authority.
Queue Manager	Manage queue assignment, priority, operating state, and authorized members.
Knowledge Manager	Manage sources, conflicts, effective dates, ingestion, and retrieval previews.
Release Manager	Run evaluations, compare releases, approve staged publication and rollback.
Tenant Admin	Manage users, roles, integrations, environments, policies, retention, and residency.
Platform Operator	Operate infrastructure through audited break-glass paths; no default content access.

5.3 Tenant isolation matrix

Layer	Required isolation control
Edge	Resolve tenant from trusted host/config and token; enforce origin, WAF, bot, DDoS, rate, and upload limits.
Application	Mandatory TenantContext object; repository methods require tenant/environment; deny missing context.
PostgreSQL	Tenant key on every tenant row, composite constraints/indexes, row-level security where practical, separate admin path.
DynamoDB	Tenant-scoped partition-key prefix and IAM access paths; no unbounded scans.
Kafka	Tenant in envelope and partition key; consumer authorization; encryption and redaction; no topic per tenant.
Redis	Tenant/environment key prefix; per-tenant quotas; no credentials or durable sensitive truth.
S3	Tenant/environment prefixes, access points/policies, short-lived signed URLs, per-class encryption and retention.
OpenSearch	Mandatory tenant/environment filters injected server-side; separate index/domain for high-isolation tier.
Model context	Context builder accepts only authorized resource handles; provider request contains one tenant; no shared prompt cache across tenants.
Logs/traces	Tenant pseudonym, classification-aware field allowlist, redaction before export, restricted correlation lookup.

Isolation invariant Every storage query, retrieval request, event, model context, cache key, object path, workflow, tool call, and trace is tenant- and environment-scoped. Cross-tenant exposure is a severity-zero event and a release-blocking test failure.

5.4 Action authorization capability

Bound claim	Why it is required
tenant_id, environment_id	Prevents cross-tenant or cross-environment execution.
workflow_id, action_id	Binds the command to one durable journey and action ledger record.
tool_id, operation	Allows only the intended integration operation.
resource target	Binds customer, order, transaction, item, subscription, or address.
amount/effect	Prevents a confirmed amount or outcome from being changed later.
preview_hash, confirmation_id	Proves the customer confirmed the exact proposed action.
policy_version, decision	Proves the deterministic rule set and obligations applied.
approval_id	Required when policy returns approval; contains authority scope and decision.
routing_epoch, expires_at, nonce	Rejects fenced regions, stale commands, and replay.

The capability is signed by a trusted authorization service and validated by the Tool Gateway. The model never sees signing keys, integration credentials, or a reusable bearer capability. For a material preview change, the workflow invalidates both confirmation and capability.

5.5 Data protection, secrets, and network controls

- TLS for all external and internal transport; mTLS for workload identity at sensitive service boundaries.
- KMS-backed encryption at rest; envelope encryption and field-level encryption/tokenization for selected PII and integration credentials.
- Secrets Manager or Vault for per-tenant integration credentials, rotation, access policy, and audit. Secrets never enter prompts, events, logs, or analytics.
- Only gateways are public. Runtime and data services use private subnets/endpoints. Egress passes through allowlisted proxies with DNS/IP/URL validation and SSRF protection.
- Uploads use short-lived signed URLs, content-length and MIME checks, malware scanning, decompression limits, metadata stripping, and quarantine before retrieval or human access.
- Backups, replicas, exports, and dead-letter paths inherit data classification, residency, encryption, access, retention, and deletion requirements.

5.6 LLM-specific threats

Threat	Control
Prompt injection in customer text	Treat as untrusted data; system policy outside user content; structured tool allowlist; no credential access.
Malicious knowledge document	Ingestion quarantine, provenance, permissions, instruction stripping/classification; retrieved content cannot redefine authority.
Cross-tenant RAG leakage	Server-injected tenant filters, retrieval authorization, corpus canaries, adversarial isolation tests.
Tool abuse	Typed schema, narrow capability, deterministic policy, confirmation/approval, rate/amount limits, idempotency and human takeover.
Data exfiltration to model provider	Data minimization, field redaction, approved provider/region, retention-disabled enterprise settings where available.
Hallucinated status or citation	Grounding checks, tool/result provenance, response state derived from workflow, no completed language without confirmed state.
Model-output exploit	Parse strict structured output, validate schemas/enums/ranges, escape UI output, never execute generated code or URLs.
Denial-of-wallet	Per-tenant token/concurrency budgets, model tiers, loop limits, cached safe results, anomaly alerts.

5.7 Audit, privacy, and incident response

The audit record is append-only and reconstructs who/what accessed data, which release and policy were used, what was proposed, what the customer confirmed, who approved, what tool was called, the redacted request/response hashes, and the authoritative result. Long-term audit archives use retention locks where required; raw chain-of-thought is neither required nor stored.

- Classify data as public, internal, confidential, restricted PII, secret, or evidence; attach handling rules to fields and events.
- Support tenant-configurable retention within product/legal bounds, regional residency, access export, and deletion workflows with tombstones and verifiable downstream propagation.
- Keep only operationally necessary conversation, evidence, model input/output, and trace detail. Redact before analytics and limit production support access.
- Provide kill switches by tenant, region/cell, release, model/provider, knowledge source, tool, operation, and action class.
- Incident flow: detect, contain/fence, preserve audit evidence, rotate/revoke, notify accountable owners, recover, and convert the failure into a regression and chaos test.

06

AI runtime, orchestration, and bounded specialists

Use AI for interpretation and grounded proposals while deterministic workflows retain control of state, permissions, waiting, and actions.

THIS TOPIC FREEZES

- One visible agent and bounded internal specialist pattern
- Runtime request pipeline and context contract
- ReAct versus plan-and-execute decision
- Durable workflow integration
- MCP, A2A, reasoning privacy, and model-loop limits

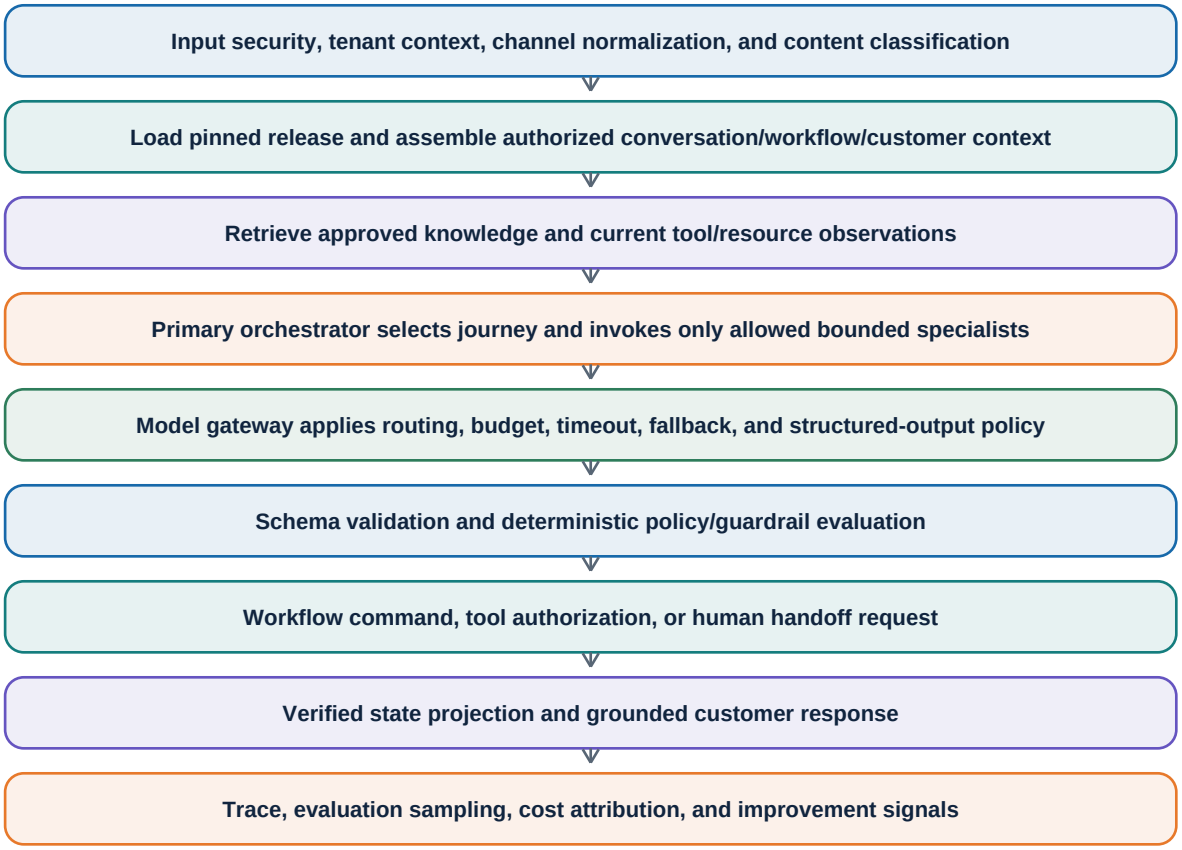
6. AI runtime, orchestration, and bounded specialists

6.1 Agent topology

The customer interacts with one support agent. Internally, a primary orchestrator may invoke a fixed set of specialists. Specialists are capabilities with typed inputs/outputs and bounded authority, not independent customer identities or an open-ended swarm.

Component	Responsibility	Prohibited behavior
Primary Orchestrator	Understand goal, select journey, ask minimal clarification, compose specialist outputs, request workflow transitions, generate response.	Direct tool execution or business truth mutation.
Triage Specialist	Classify journey, urgency, sentiment, risk hints, missing facts.	Final policy or fraud decision.
Knowledge Specialist	Construct retrieval query, rank grounded sources, identify conflict/missing authority.	Invent policy or bypass source permissions.
Customer Operations Specialist	Map customer goal to supported order/return/refund/replacement/subscription operations.	Authorize or submit action.
Risk and Policy Specialist	Explain deterministic policy output and surface signals for policy engine/human review.	Override deterministic decision.
Handoff Specialist	Produce structured summary, pending decisions, attempts, failures, evidence and recommended next step.	Choose unauthorized queue or hide context.

6.2 Runtime pipeline



6.3 Context contract

Context block	Rules
System and release	Immutable purpose, behavior policy, journey definitions, tool schemas, response contract, release IDs.
Tenant	Brand/tone, locale, environment, supported journeys, permitted models and tools; no raw secrets.
Conversation	Recent turns plus structured summary; message IDs and participant/control state.
Workflow	Current durable state, collected fields, missing fields, proposed action, confirmation/approval state, pending timers.
Customer and business	Minimum authorized facts needed for this journey; source and freshness included.
Knowledge	Authorized passages with source ID, version, effective date, locator, score, and conflict status.
Safety and budget	Action permissions, sensitive-topic route, loop/tool/token limits, model tier and latency budget.

Context assembly is server-side and deterministic. The model receives a snapshot and returns a typed proposal containing intent, journey, extracted facts with evidence, missing information, proposed next step, requested specialist/tool observation, confidence category, and customer response draft.

6.4 Reasoning strategy

Pattern	Use in this product	Boundary
Direct structured decision	Simple classification, extraction, summarization, response revision.	Preferred when one model call is sufficient.
ReAct	Short bounded loops for retrieval/read-only observations and clarification.	Maximum steps, allowed observations, token/time budget; no consequential writes.
Plan-and-execute	Multi-step customer journeys coordinated with a durable workflow plan.	Workflow owns state and waiting; model proposes or updates bounded plan.
Chain-of-thought	Internal model reasoning may occur, but the system requests decisions, evidence, reason codes, and concise rationale.	Never expose or store raw hidden reasoning.
Tree-of-thought	Rare offline design/evaluation or difficult case analysis where multiple hypotheses add value.	Not the normal latency-sensitive production loop.

6.5 Workflow and model handshake

- The workflow decides which AI decision type is permitted in the current state and supplies a typed request.
- The agent runtime returns a structured proposal and evidence references, not an imperative side effect.
- The workflow validates expected state/version, persists accepted facts, and chooses the next deterministic transition.
- If clarification is needed, the conversation service sends the generated question and waits for a customer signal.
- If an action is ready, the workflow builds the preview; after confirmation it invokes policy/approval and Tool Gateway activities.
- When a model call times out or produces invalid output, the workflow retries within a bounded policy, chooses a fallback, or hands off without losing state.

6.6 MCP and A2A decisions

MCP: yes

MCP is the standard tool and resource contract between the platform's Tool Gateway and approved business connectors. MCP does not grant trust: the gateway still supplies identity, authorization, validation, idempotency, quotas, audit, retries, and reconciliation.

A2A: not for internal V1

Internal specialists run behind typed runtime interfaces because they share one product authority boundary. An A2A adapter may be added later for independently hosted partner agents, delegated domains, or external organizations with their own identity, discovery, task, and trust lifecycle.

6.7 Runtime guardrails

- Per-turn maximum model calls, specialist invocations, read-only tool observations, total tokens, wall time, and cost.
- Tool allowlist is journey-state-specific. A model cannot name an arbitrary URL, operation, tenant, or resource identifier.
- Every model output is parsed against a versioned JSON schema; unknown fields and invalid enums are rejected.
- Confidence is not authorization. Low confidence triggers clarification or human review; high confidence cannot bypass policy.
- The response renderer derives action status from workflow state and attaches citations or action references from trusted records.
- Circuit breakers isolate failing model routes; fallbacks must pass the same safety and release gates.

07

Knowledge, models, safety, evaluation, and observability

Make every answer traceable to authorized evidence, every model route measurable, and every release provably safer than its predecessor.

THIS TOPIC FREEZES
- Knowledge ingestion and hybrid retrieval
- Model gateway routing, fallbacks, budgets, and economics
- Layered safety and response grounding
- Offline and online evaluation gates
- End-to-end observability and continuous-improvement loop

7. Knowledge, models, safety, evaluation, and observability

7.1 Knowledge lifecycle

S3 plus PostgreSQL metadata remain authoritative. OpenSearch is rebuildable. Publishing is atomic at the index-version level so a conversation sees one coherent knowledge snapshot rather than a partially updated corpus.

7.2 Retrieval pipeline

Stage	Design
Query understanding	Create bounded search queries from customer goal, journey, product, locale, and workflow state.
Authorization filter	Inject tenant, environment, ACL, locale, effective time, source status, and journey filters server-side.
Candidate generation	Parallel BM25 lexical and vector semantic retrieval; optional structured lookup for policy metadata.
Fusion	Reciprocal-rank or calibrated fusion; deduplicate by source/section and preserve scores.
Reranking	Small reranker only within latency/cost budget; prevent unauthorized candidate expansion.
Conflict handling	Prefer authoritative/current source class; surface unresolved conflicts to workflow or human.
Answer grounding	Pass minimal passages with source/version/locator; require claim-to-source mapping.
Post-check	Verify cited source exists in authorized set and answer does not exceed evidence.

Knowledge rule

Retrieved content is evidence, not instruction. A document can state a return policy; it cannot redefine system authority, request credentials, grant tool permission, or alter confirmation and approval boundaries.

7.3 Memory model

Memory	Lifetime	Use
Turn context	One model request	Exact current input, retrieved passages, tool observations, and response contract.
Conversation window	Active conversation	Recent messages selected by token budget and importance.
Structured conversation summary	Conversation lifetime	Goal, verified facts, decisions, unresolved items, sentiment, and handoff continuity.
Workflow state	Journey lifetime	Authoritative collected fields, confirmations, approvals, action and waiting state.
Customer profile	Tenant-defined retention	Authorized stable preferences/facts; never inferred sensitive attributes without purpose.
Learning memory	Release lifecycle	Curated failures, regressions, eval datasets, and approved knowledge changes.

Vector memory is not a substitute for workflow state. Durable facts are typed, source-attributed, consent/retention-aware, and written through deterministic services. The platform does not preserve hidden chain-of-thought as memory.

7.4 Model gateway

Function	HLD decision
Route selection	Choose by task type, release policy, risk, language, context size, latency target, regional availability, and cost.
Provider abstraction	Normalize messages, structured outputs, streaming, usage, errors, safety signals, and trace metadata.
Fallback	Fallback chain is task-specific and pre-evaluated; never silently move restricted data to an unapproved provider/region.
Budgets	Tenant/release/task token, call-count, concurrency, latency, and dollar budgets.
Caching	Only for safe deterministic/semantic cases; tenant/release/knowledge/model/safety scoped; never cache secrets or mutable action status.
Resilience	Deadlines, hedging only where safe/economic, provider bulkheads, rate limit adaptation, circuit breakers.
Privacy	Data minimization, redaction, approved retention settings, provider allowlist and regional route.
Economics	Attribute input/output tokens, retrieval, tool, and workflow costs to tenant, journey, release, model route, and outcome.

7.5 Layered safety architecture

Layer	Examples
Input	Abuse/content classification, PII handling, prompt-injection indicators, file quarantine, size/rate limits.
Retrieval	ACL and temporal filters, source trust tiers, conflict detection, malicious-instruction isolation.
Reasoning	System policy, bounded specialists, tool allowlist, loop budget, structured proposal.
Decision	Deterministic eligibility, risk/threshold rules, identity obligations, confirmation, approval/takeover.
Execution	Narrow signed capability, schema validation, idempotency/action ledger, egress allowlist, circuit breaker.
Output	Grounding/citation validation, status derived from workflow, sensitive-language rules, HTML/UI escaping.
Operations	Sampling, anomaly detection, audit, human override, kill switches, incident regressions.

7.6 Evaluation and release gates

Evaluation class	Core measures
Journey correctness	Intent/journey, required inputs, state transition, final outcome and alternative/failure paths.
Policy correctness	Eligibility, evidence, confirmation, approval threshold, takeover, exception routing.
Tool correctness	Correct tool/arguments, no call before authority, idempotency, unknown-result reconciliation.
Knowledge quality	Recall@k, nDCG/MRR where useful, citation precision, groundedness, freshness, conflict behavior.
Safety/security	Prompt injection, data leakage, tenant isolation, unauthorized action, sensitive topic and abuse cases.
Conversation quality	Task progress, clarification economy, tone, accessibility, neutral risk language, handoff summary.
Reliability/performance	Latency distributions, timeout/fallback, queue age, provider/tool chaos, recovery correctness.
Economics	Cost per resolved journey, tokens/calls per step, escalation and recontact cost.
Human outcomes	Override/approval rate, handoff completeness, time to resolution, repeat contact, satisfaction.

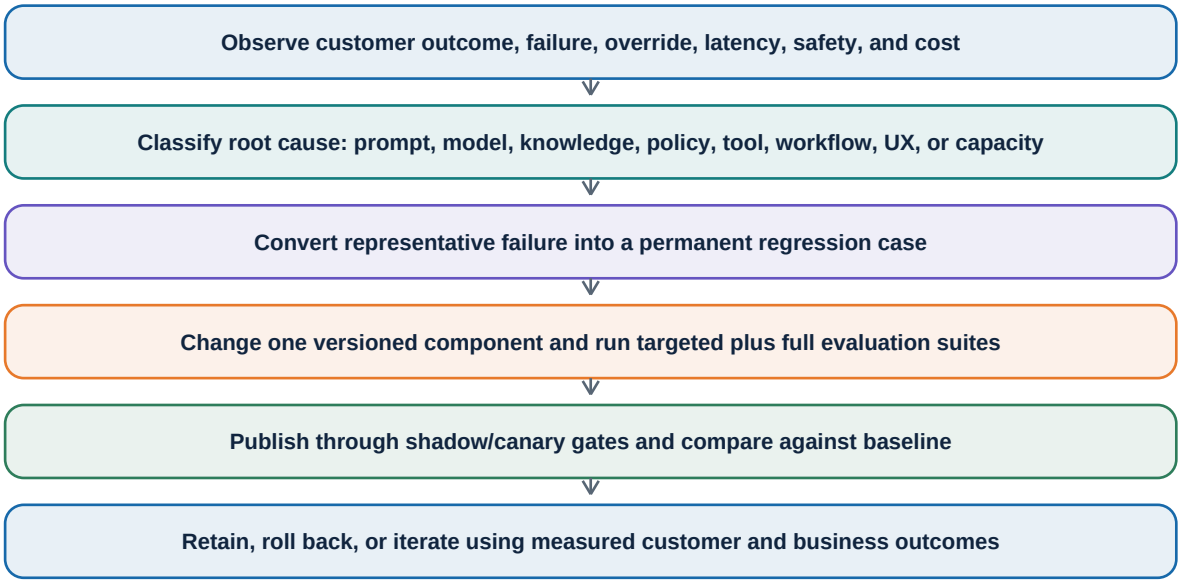
Datasets include golden normal cases, boundaries, alternative and failure paths, adversarial prompts, policy-version cases, historical failures, multilingual cases, provider/tool chaos, and synthetic load. High-risk gates require deterministic assertions; LLM-as-judge may supplement subjective quality but never be the sole judge of action safety or tenant isolation.

7.7 Release gate policy

- Zero critical failures for cross-tenant leakage, missing confirmation, unauthorized action, duplicate consequential action, or false completion.
- No statistically or practically significant regression on journey success, policy correctness, handoff completeness, latency, or cost beyond approved budgets.
- New model routes, prompts, tools, policies, and knowledge indexes are all versioned and evaluated as one release bundle.
- Shadow and canary traffic compare candidate versus current release with identical redaction and privacy policy.
- Rollback is one operation at the release-routing layer; in-flight conversations remain pinned unless the release is security-blocked.

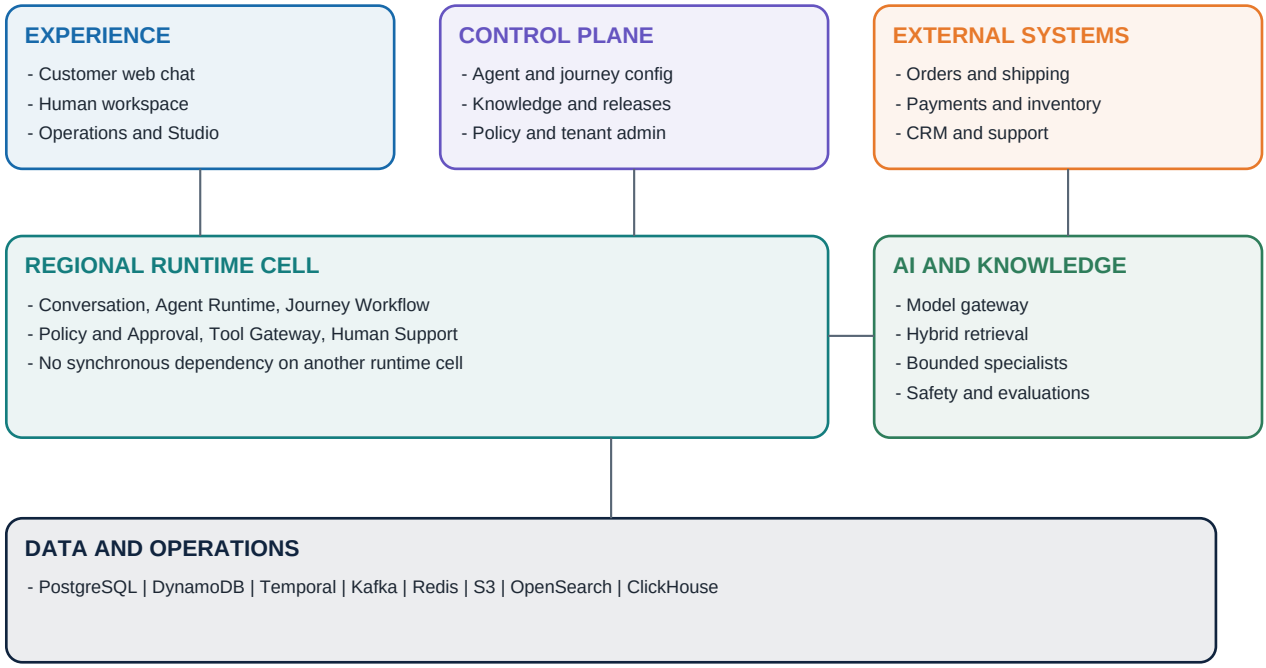
7.8 Observability and improvement loop

Telemetry	Required dimensions
Distributed trace	tenant pseudonym, region/cell, release, conversation, workflow, journey, model route, retrieval, tool, approval and outcome spans.
Metrics	SLIs, queue age, concurrency, token/cost, retrieval quality, tool results, approval and escalation, business outcomes.
Logs	Structured allowlisted fields, redacted before export, event/trace correlation, no credentials or raw hidden reasoning.
Audit	Immutable actor/action/resource/decision/version/result record with redacted hashes and retention controls.
Product analytics	Resolution, containment, transfer, recontact, customer satisfaction, policy exceptions, broken-process signals.


Observe customer outcome, failure, override, latency, safety, and cost
Classify root cause: prompt, model, knowledge, policy, tool, workflow, UX, or capacity
Convert representative failure into a permanent regression case
Change one versioned component and run targeted plus full evaluation suites
Publish through shadow/canary gates and compare against baseline
Retain, roll back, or iterate using measured customer and business outcomes

Combined architecture and HLD freeze

The seven topics combine into one execution path: the cell-local backend preserves tenant and workflow truth; the AI runtime proposes bounded next steps; knowledge and model systems provide authorized evidence and language; deterministic policy and tool gateways control action; events and telemetry make the result reconstructable and improvable.



Journey traceability

V1 journey	Core read/write path	Confirmation and human boundary
1. Order tracking	Order + shipment read; explain current milestone and delay.	No action confirmation. Escalate for identity, missing data, conflicting status, or customer request.
2. Return eligibility	Verified order/item + effective policy; explain decision.	No action confirmation for explanation; exceptions/conflicts go to human.
3. Create return	Eligibility -> preview -> return creation -> label/reference.	Explicit confirmation; human review for exceptions/risk; no success before return system confirms.
4. Damaged replacement	Evidence + policy + risk + inventory + address -> create replacement.	Evidence required above $75. Automatic <= $250 low risk; > $250 approval. Address-change restrictions may require approval.
5. Refund	Eligibility + amount/method -> submit -> payment confirmation.	Always confirm. <= $100 low risk automatic; $100.01-$500 approval; > $500 takeover.
6. Subscription cancellation	Plan/options -> transparent preview -> cancel -> confirmed timing.	Confirm. Standard permitted cancellation automatic; exceptions/disputes to human; at most one retention offer.
7. Human escalation	Create/update case with transcript, summary, policy, tools, evidence, state and recommendation.	Explicit human request honored. Queue routing does not require separate approval; action authority still applies.

Cross-cutting correctness invariants

- Zero cross-tenant data exposure.

- Zero consequential actions without the required customer confirmation.
- Zero intentional duplicate return, refund, replacement, cancellation, or case creation.
- Zero completed status before the authoritative business system confirms the outcome.
- Every approval and action is attributable to principal, policy version, release, workflow, and exact preview.
- A customer can request a human at any time and the representative receives enough context to continue without repetition.
- Model or retrieval confidence never substitutes for authentication, authorization, policy, confirmation, or business truth.
- Runtime can continue on a pinned release while control-plane publishing is unavailable.
- Region failover fences the old writer before accepting consequential writes in the recovery region.
- The full customer outcome is reconstructable from transactional state, events, action ledger, and audit records without raw chain-of-thought.

Frozen technology decisions

Area	HLD decision
Cloud/topology	AWS reference deployment; EKS regional cells; three active regions; one active writer per tenant.
Core stores	Aurora PostgreSQL, DynamoDB, Temporal, Kafka/MSK, Redis/Valkey, S3, OpenSearch, ClickHouse.
Interaction	HTTPS and SSE/WebSocket at edge; typed gRPC/HTTP internally; Kafka events; Temporal workflows; MCP tool contracts.
Agent pattern	One customer-visible agent, one orchestrator, fixed bounded specialists; no unbounded swarm.
Reasoning	Structured decisions; bounded ReAct for short observations; workflow-backed plan-and-execute; no stored/exposed chain-of-thought.
External agent protocol	A2A deferred until independently operated agents create a real trust and task boundary.
Safety	Deterministic policy, confirmation binding, approval/takeover, action capability, idempotency ledger, verified result.
Release	Immutable release bundle, conversation pinning, offline gates, shadow/canary rollout, rapid routing rollback.

Explicitly rejected at HLD

Rejected design	Reason
Model directly calls commerce/payment APIs	Breaks authorization, confirmation, idempotency, audit, and truthful status guarantees.
One giant global runtime	Expands blast radius, complicates noisy-neighbor control, and makes regional recovery unsafe.
Fully active-active writes for one tenant	Creates split-brain and duplicate-action risk disproportionate to V1 value.
Exactly-once delivery claim	Infrastructure may duplicate; correctness comes from idempotency, ledger, versions, and reconciliation.
Thirty microservices on day one	Operational cost without proven independent scaling or ownership need; use modular deployables.
Vector database as universal memory	Cannot replace authoritative workflow, customer, policy, or action state.
A2A between internal specialists	Adds discovery/trust/task protocol overhead inside one authority boundary.
Raw chain-of-thought storage	Unnecessary for audit; increases privacy, security, and governance exposure.
Automated damage-image judgment in V1	Product contract supports upload/human evidence; vision automation is not frozen V1 scope.

Risks and validation obligations

Risk	Validation before production
Provider/tool timeout after submission	Integration-specific reconciliation contract and chaos tests for every consequential operation.
Cross-tenant leakage	Automated isolation suite across APIs, stores, retrieval, cache, events, models, logs, exports, and support tools.
Workflow/version drift	Conversation release pinning tests; replay/migration policy; rollback and long-running workflow compatibility.
Cost/latency blowout	Per-journey budgets, concurrency limits, representative load, provider-failure, and long-context tests.
Knowledge conflicts/staleness	Effective-date tests, authoritative-source hierarchy, conflict queue, citation review, rollback.
Human handoff overload	Queue capacity model, priority/fallback routing, wait-state UX, and approval SLA alerts.
Regional evacuation	Fencing, recovery-cell readiness, workflow resumption, unknown-action reconciliation, and controlled failback exercise.

HLD exit criteria

- Every V1 journey maps to owned services, authoritative data, workflow states, policy/approval rules, tool contracts, events, safety controls, and measurable outcomes.
- The AI/backend authority boundary is explicit and testable.
- Capacity, cells, regional routing, failover fencing, backpressure, RPO/RTO, and graceful degradation are defined.
- Identity, authorization, tenant isolation, data classification, secrets, network, LLM threats, audit, privacy, and kill switches are defined.
- Knowledge, model routing, evaluation gates, observability, and release rollback are defined.
- Remaining decisions fit inside LLD without changing product scope or these system boundaries.

HLD status With this document reviewed and accepted, High-Level Design is complete. New detail goes into LLD unless it changes a frozen product or architecture boundary.

Low-Level Design handoff

LLD is one implementation design divided into ten finite parts. Each part produces schemas, contracts, state machines, test cases, or executable scaffolding.

Part	LLD output
1	Codebase, modules, deployables, dependency rules, configuration and local environment.
2	Identity, tenant context, roles, policy interfaces, workload identity and isolation enforcement.
3	PostgreSQL/DynamoDB/Redis/S3 schemas, keys, indexes, migrations, retention and encryption.
4	Conversation, admin and real-time REST/gRPC/SSE contracts, pagination, errors and idempotency.
5	Kafka topics, event schemas, outbox/inbox, schema evolution, retry and dead-letter handling.
6	Temporal workflows for all seven journeys, state machines, timers, signals, approvals and recovery.
7	Tool/MCP contracts, connector SDK, credentials, action capability, idempotency ledger and reconciliation.
8	Agent runtime interfaces, context schema, specialist contracts, prompts, structured outputs, budgets and fallbacks.
9	Knowledge ingestion, chunking, indexes, retrieval/reranking, citations, memory and model-gateway contracts.
10	Telemetry schemas, evaluations, security tests, load/chaos tests, deployment, canary and DR runbooks.

Implementation begins after Part 10, using vertical slices through the reference journeys rather than building every infrastructure component in isolation. Order tracking proves the read path; return creation proves confirmation and idempotent writes; damaged replacement proves evidence, policy, approval, inventory, address subworkflow, and recovery; refund proves financial thresholds and unknown-result reconciliation; cancellation proves transparent consequence handling; human escalation proves shared context and control transfer.

Appendix A - Decision vocabulary

Term	Meaning in this architecture
Domain event	A past-tense business fact emitted after an owner commits state, such as ConfirmationRecorded.
Command	An imperative request to one owner, such as SubmitRefund, which may be accepted or rejected.
Runtime cell	A bounded, independently operated tenant-serving unit with no synchronous dependency on another cell.
Home region	The single active writer region for a tenant's operational data and workflows.
Routing epoch	A monotonically increasing fencing token that invalidates writes from an old cell or region.
Release bundle	Immutable set of agent, prompt, journey, policy, tool, knowledge, model-route, and evaluation versions.
Bounded specialist	Internal typed AI capability invoked by the orchestrator with fixed purpose, inputs, outputs, budgets, and no independent authority.
MCP	Tool/resource contract used behind the Tool Gateway; not an authentication or authorization system.
A2A	Protocol boundary for independently hosted/operated agents; deferred for internal V1 specialists.
ReAct	Bounded reason-and-observe loop used for short read/clarification tasks.
Plan-and-execute	Durable workflow controls the plan, state, waiting, and actions; AI proposes bounded steps.
Action ledger	Durable record of action identity, authorization, submission, provider reference, status, and reconciliation.
Authoritative result	Business-system response or later verified status that proves the outcome actually occurred.

Appendix B - Source documents

This HLD was derived from and checked against the following complete product documents:

Source	Pages	Role
Customer Agent OS Lite - Project Overview	17	Vision, users, product goals, build sequence, scope and portfolio story.
Customer Agent OS Lite - Fixed Product Features	14	Ten-sequence product roadmap and fixed capability inventory.
Customer Agent OS Lite - V1 Product Contract	18	Binding V1 journeys, workflows, screens, states, thresholds and acceptance criteria.

Document note

This is a design baseline, not a claim of a completed production deployment. Capacity values are provisional engineering assumptions to be confirmed through LLD calculations, benchmarks, failure injection, provider quotas, and representative load tests. Security and compliance obligations must be refined for each tenant, jurisdiction, data class, and integration before production use.

End state Seven HLD topics are complete and combined. The next design activity is LLD Part 1: codebase and deployable structure.

KLEEM AI / ARCHITECTURE AUTHORITY

Customer Agent OS Lite

Integrated HLD Validation and Architecture Freeze

Final end-to-end validation of the V1 product contract, backend platform HLD, and regenerated AI HLD Parts 1 and 2.

STATUS	FROZEN FOR LOW-LEVEL DESIGN
Decision	PASS after ten binding amendments incorporated by this document
Scope	Seven V1 journeys, state ownership, API boundaries, failure behavior, security, tenant isolation, regional recovery, and backend-AI consistency
Version	1.1 - July 21, 2026

Architecture thesis: the AI interprets, retrieves, reasons, and proposes. Deterministic backend services authenticate, authorize, confirm, approve, execute, reconcile, and record every consequential business action.

1. Executive freeze decision

FINAL DECISION The complete HLD is frozen for LLD. All seven V1 journeys can be implemented without giving the model business authority. Ten conflicts or ambiguities found during integration are resolved in Section 8 and are binding amendments to the older combined HLD.

This is not a claim that production readiness is proven. It means the high-level owners, boundaries, execution modes, safety invariants, failure states, regional strategy, and technology roles are coherent enough to begin detailed design. Capacity, provider quotas, recovery behavior, model quality, policy correctness, and tenant isolation still require implementation evidence in LLD and pre-production testing.

1.1 Validation scorecard

#	Validation task	Result	Conclusion
1	Trace seven V1 journeys end to end	PASS	Every journey has a normal, alternate, failure, and human path with a named truth owner.
2	Verify state ownership and API boundaries	PASS	One owner per durable state; model output crosses only typed proposal and composition boundaries.
3	Simulate external failures	PASS	Unknown, pending, recoverable, denied, and handoff states remain truthful; no blind consequential retry.
4	Validate security and tenant isolation	PASS	Authorization occurs before retrieval and tool execution; model authority remains zero for writes.
5	Confirm regional deployment and recovery	PASS	Single active writer, fencing, action reconciliation, and cell independence are preserved with the action-store amendment.
6	Resolve backend and AI contradictions	PASS	Ten binding amendments remove the remaining conflicts.
7	Freeze complete HLD	PASS	No unresolved HLD design decision remains; proof obligations move to LLD.

1.2 Binding precedence after freeze

When two statements conflict, use this order:

Rank	Source	Authority
1	Customer Agent OS Lite - V1 Product Contract	Controls V1 journeys, thresholds, confirmations, human boundaries, outcome states, and acceptance criteria.
2	This Integrated HLD Validation and Architecture Freeze	Controls the ten amendments and the final interpretation of backend-AI boundaries.
3	AI HLD Parts 1 and 2 - regenerated July 21, 2026	Controls AI runtime, knowledge, models, guardrails, evaluations, and AI operations.
4	Customer Agent OS Lite - Combined HLD v1.0	Remains the backend, data, security, cell, and deployment baseline except where amended here.
5	Fixed Product Features and Project Overview	Control broader roadmap and product vision outside the V1 contract.

1.3 Non-negotiable system invariants

- Tenant and environment identity come from trusted authentication and routing context, never from an unchecked model or request field.
- The model has no consequential write tool, integration credential, approval authority, confirmation authority, or completion authority.
- Every material factual claim is grounded in approved evidence, an authorized business read, deterministic policy output, or a stable customer-provided fact.
- Every consequential action has a backend-generated preview, explicit confirmation, current policy evaluation, required approval, a single-use action capability, idempotent submission, and verified outcome.
- RESULT_UNKNOWN never becomes success or failure by inference; it enters reconciliation.
- Cross-tenant exposure, unconfirmed execution, duplicate consequential effect, and false completed status are release-blocking violations.
- A customer can request a human at any time, and accepting human ownership stops automated customer-facing replies until explicit release-back.

2. Validation method and architecture under test

The validation traced the product contract through the combined backend HLD and the regenerated AI HLD. Each journey was tested against the same chain of questions: Who owns the input? Who owns durable state? Which facts are authoritative? What may the model do? Who creates the preview? Who records confirmation? Who authorizes and executes? What happens after an uncertain result? How does the state survive process, cell, or region failure?

2.1 Frozen execution chain

1	Channel authenticates the session, admits the message, and persists it through the Conversation Service with a monotonic sequence and client idempotency key.
2	The runtime resolves tenant, environment, home cell, active agent release, current workflow state, and the permitted capability envelope.
3	The AI Runtime performs one bounded mode: deterministic response, grounded generation, bounded read loop, validated information plan, or participation in a durable journey.
4	All model output is schema-validated. A workflow proposal carries expected workflow state and version; it cannot directly mutate business state.
5	Temporal owns the durable journey. Policy Service owns deterministic eligibility and risk. The backend renders the canonical preview and records exact confirmation.
6	After policy, approval, and confirmation succeed, the workflow obtains an opaque, short-lived, single-use Action Capability that the model never receives.
7	The Tool Gateway atomically redeems the capability, records the action ledger, submits idempotently, and reconciles any unknown result.
8	Conversation state exposes completed only after the authoritative business system confirms the result; telemetry and evaluations run asynchronously.

2.2 Supported orchestration modes

Mode	Pattern	Used for	Boundary
A	Deterministic response	Acknowledgements, validated status templates, fixed progress states	No generation required
B	Grounded generation	Policy and product explanation	Authorized evidence package plus claim validation
C	Bounded read loop	Order, shipment, account, inventory reads	Read or pure-computation calls only; fixed call and time budget
D	Validated information plan	Several reads plus a typed workflow proposal	Every step validated; plan cannot perform a business write
E	Durable journey participation	Returns, replacements, refunds, cancellations, handoff	Temporal owns state; AI performs bounded Activities

2.3 Architecture facts checked against platform semantics

- Temporal Workflow Executions are durable and replayed from event history; external calls belong in Activities, and Temporal recommends idempotent Activity behavior [T1][T2].
- Temporal Cloud multi-region replication publishes an RPO under one minute and RTO under twenty minutes; those service objectives do not by themselves guarantee zero-loss action safety [T3].
- DynamoDB MREC Global Tables use asynchronous replication and last-writer-wins conflict resolution; optimistic locking cannot be assumed to work across Regions [A1][A2].
- OpenSearch supports filtering before scoring, which is the required placement for mandatory authorization filters in hybrid retrieval [O1].
- MCP standardizes tool names and schemas and defines transport authorization, but it does not grant tenant, object, workflow, or business-action authority [M1][M2].

3. Seven V1 journey traces

The following traces are binding HLD sequences. Exact endpoint names, message schemas, table layouts, and workflow code are deferred to LLD.

3.1 Order tracking

1	Conversation Service deduplicates and sequences the message; Identity and Tenant Context supplies trusted tenant, environment, customer, and verification state.
2	AI Runtime classifies the request and extracts only bounded identifiers. If no order is identified, it requests an authorized recent-order list and asks the customer to select.
3	Tool Gateway enforces tenant, customer, object ownership, field allowlist, purpose, query size, rate, and audit rules before order and shipment reads.
4	The composer explains each shipment separately using authoritative tool results; it cannot invent delivery dates or merge conflicting shipment states.
5	Output validation compares all status claims with the tool result, then Conversation Service commits the validated response at the expected conversation version.
6	Identity failure, order mismatch, delivered-but-missing risk, carrier conflict, or unavailable tools produce clarification, truthful unavailability, or durable handoff.

Execution	Truth owner	Confirmation	Human boundary	Failure rule
Bounded read loop	Order and carrier systems	None for reads	Dispute, identity, unsupported promise, repeated tool failure	No guessed status; clarify or hand off

3.2 Return eligibility

1	The journey identifies and verifies customer, order, item, seller, region, purchase date, and fulfillment facts through authorized business reads.
2	Policy Service selects and executes the rule version applicable to the purchase. RAG may retrieve customer-visible policy text for explanation but cannot decide eligibility.
3	The AI may extract a return reason or condition and submit it as a typed proposal; the workflow validates the fact and current state before accepting it.
4	The response shows eligible or ineligible, the decisive reason, deadline or condition, and the permitted next step, with evidence or policy version recorded.
5	An eligibility-only request never creates a return. An eligible result may transition into a separate create-return workflow without reusing stale policy or object facts.
6	Missing or conflicting policy, marketplace ambiguity, unsupported category, identity mismatch, suspected abuse, or customer dispute enters human review rather than model choice.

Execution	Truth owner	Confirmation	Human boundary	Failure rule
Business reads plus deterministic policy	Policy Service	None for decision	Conflict, exception, missing authority, dispute	INSUFFICIENT/CONFLICTING cannot be filled by model knowledge

3.3 Create a return

1	Temporal starts or resumes a return workflow and revalidates customer, order, item, quantity, active-return conflict, eligibility, permitted resolution, method, fee, and deadline.
2	The backend renders the canonical preview from validated data. The AI may explain it but cannot construct its authoritative fields.
3	Confirmation Service records the customer's explicit agreement to the exact preview hash and workflow state version. Any material change invalidates confirmation.
4	Policy Service returns allow, approval, takeover, or deny. Standard eligible low-risk returns proceed; configured exceptions or risk wait for approval.
5	Workflow obtains an Action Capability for the exact return operation. Tool Gateway redeems it once, records the ledger, and calls the return system with a stable idempotency key.
6	A confirmed return ID establishes the business outcome. Label failure after return creation remains pending/recoverable and must not cause a second return authorization.

Execution	Truth owner	Confirmation	Human boundary	Failure rule
Durable journey	Temporal plus return system	Required	Exception or unusual risk	Existing/unknown action reconciled; label handled as substate

3.4 Damaged-item replacement

1	Temporal verifies customer, order, fulfilled item, product-category support, damage description, and requested resolution.
2	Image evidence is requested when value is above $75 or policy requires it. V1 scans, stores, and presents the upload for human-review evidence; automated visual damage judgment remains out of scope.
3	Policy, prior-claim, risk, inventory, replacement-alternative, return-obligation, address, and fulfillment-stage facts are evaluated by their authoritative services.
4	The backend preview binds item, quantity, destination, return obligation, timing, policy, and state. Changed address requires re-verification; restricted-stage change requires approval.
5	Customer confirms. Low-risk value <= $250 may proceed automatically; value > $250, unclear evidence, exception, or risk requires approval.
6	Tool Gateway reserves/creates idempotently. Partial reservation, timeout after submission, or lost network result becomes pending or RESULT_UNKNOWN and enters reconciliation before another replacement is permitted.

Execution	Truth owner	Confirmation	Human boundary	Failure rule
Durable journey plus secure evidence	Temporal, inventory, fulfillment	Required	> $250, unclear evidence, risk, restricted address	Neutral language; reconcile or hand off; no duplicate

3.5 Refund

1	The workflow verifies customer, refundable transaction or returned item, prior refunds, payment method, permitted reason, amount, fees, deductions, and destination.
2	Policy Service calculates the permitted amount. The backend renders the exact amount, currency, destination, timing, and consequences in the canonical preview.
3	Customer confirmation is always required. For $100.01-$500, confirmation precedes the approval request. If approval changes any material field, a new preview and confirmation are mandatory.
4	Low-risk refunds <= $100 may proceed automatically. $100.01-$500 require approval. Above $500 requires full human takeover; the representative still must obtain confirmation of the exact final preview before execution.
5	After all gates, the workflow obtains an Action Capability. Tool Gateway atomically redeems it and submits using a stable idempotency key.
6	Only a payment-system reference and confirmed state permit completed. Timeout after possible submission is RESULT_UNKNOWN, is never blindly retried, and is sampled for review at 100 percent.

Execution	Truth owner	Confirmation	Human boundary	Failure rule
Durable consequential journey	Temporal, Policy, payment system	Always	Approval $100.01-$500; takeover > $500; risk/exception	Unknown result blocks repeat and triggers reconciliation

3.6 Subscription cancellation

1	Authorized reads load the active subscription, plan, renewal, price, contractual options, balances, and existing cancellation state.
2	The response explains immediate versus end-of-term effects. A reason is optional unless policy requires it, and no more than one permitted retention offer may be shown.
3	The backend renders effective date, access end, billing/credit effects, and irreversible consequences. Customer confirmation binds to this preview.
4	Policy revalidates contractual permission and current state. Standard cancellation proceeds; contract exception, disputed balance, identity uncertainty, special credit, or policy conflict enters review.
5	Tool Gateway redeems a single-use Action Capability and submits idempotently. An existing scheduled cancellation is displayed rather than duplicated.
6	The subscription-system confirmation controls completed. Failure preserves the customer's intent and a visible pending/recovery state, but any changed effect requires new confirmation.

Execution	Truth owner	Confirmation	Human boundary	Failure rule
Durable consequential journey	Temporal and subscription system	Required	Contract/billing/identity exception	Preserve intent; no false completion or duplicate

3.7 Human escalation

1	An explicit request for a person or an internal risk/unsupported/failure trigger signals a durable handoff transition. No separate customer confirmation is required to honor the request.
2	Temporal pauses unsafe automation and preserves the active journey, pending actions, timers, confirmations, and unresolved external results.
3	The backend assembles identity/verification, authoritative records, workflow state, policy decision, tool outcomes, evidence links, attempts, and unresolved operations. AI provides only a labeled draft summary and inferred sentiment/urgency.
4	Human Support selects the authorized queue and priority and creates/updates a case idempotently. The customer sees joining-now or queued status and the next expected step.
5	When a representative accepts ownership, automated customer replies stop. AI suggestions become internal-only, and representative actions use separate workforce authorization.
6	Automation resumes only through explicit release-back. Case-creation failure remains pending/recoverable; customer disconnect does not cancel accepted durable work.

Execution	Truth owner	Confirmation	Human boundary	Failure rule
Durable handoff	Temporal and Human Support	None for handoff	Representative authority remains action-specific	Retry case creation safely; preserve queue and journey continuity

4. State ownership and API boundary validation

OWNERSHIP RULE Every durable fact has one authoritative owner. Other components may cache, project, explain, or propose, but they cannot silently become a second writer.

Owner	Authoritative for	Must not own
Customer Chat Gateway	Session admission, connection, transport progress	Conversation truth, workflow state, action status
Identity and Tenant Context	Principal, tenant, environment, customer verification, home cell, routing epoch	Journey or policy decisions
Conversation Service	Ordered messages, sequence, participants, attachments, visible response, AI/human control	Refund/replacement/cancellation completion
Agent Runtime	Turn-local context, bounded specialist calls, typed proposal, response composition	Durable private state, approval, confirmation, consequential write
Temporal Journey Workflow	Journey state, legal transitions, timers, retries, confirmation wait, approval wait, cancellation, compensation, handoff	Policy authorship, provider credentials, model secrets
Policy Service	Deterministic eligibility, risk, thresholds, obligations, decision and version	Workflow ordering or tool execution
Confirmation Service/module	Canonical preview hash and exact customer confirmation record	Generated wording or policy decision
Approval/Human Support	Approval lifecycle, queues, assignment, takeover, release-back, internal notes	AI prompt/model routing

Owner	Authoritative for	Must not own
Action Capability Service	Opaque single-use capability, bound claims, expiry, revocation, redemption count	Business integration credentials
Tool Gateway	Tool contracts, credentials, read authorization, idempotency ledger, submission, result reconciliation	Customer conversation or AI reasoning
Knowledge Platform	Sources, ACLs, authority, effective dates, releases, revocation epoch	Business policy execution or customer ownership
Model Gateway	Approved models, routing, provider policy, structured output, token/cost accounting	Business truth or authorization
External business system	Actual order, shipment, inventory, payment, return, replacement, subscription result	Kleem AI workflow or conversation state
Audit and Evaluation	Append-only/projection evidence, quality and operations measurements	Transactional workflow truth

4.1 Required typed boundaries

Boundary	Minimum contract	Validator/owner	Outcome
Message command	client_message_id, expected conversation version, channel identity	Conversation Service	Existing result or version conflict
Authorized read	tenant, environment, customer, object, fields, purpose, limits	Tool/Retrieval Gateway	DENIED, NOT_FOUND, TEMPORARILY_UNAVAILABLE
Workflow proposal	proposal_id, workflow_id, expected state/version, typed fields, evidence refs	Temporal	ACCEPTED or explicit rejection code
Policy evaluation	versioned facts, action, amount/effect, jurisdiction, risk	Policy Service	ALLOW, APPROVAL, TAKEOVER, DENY plus obligations
Confirmation	preview_hash, workflow/state version, customer/session, timestamp	Confirmation module	Recorded, expired, stale, or mismatched
Action capability	tenant, workflow, confirmation, operation, argument hash, policy, approval, epoch, expiry	Capability Service	Opaque single-use reference
Tool redemption	capability reference plus canonical arguments and idempotency key	Tool Gateway	SUCCEEDED, SUBMITTED, RESULT_UNKNOWN, RECONCILING, FAILED_FINAL
Response commit	expected conversation version, validated text, evidence/result refs	Conversation Service	Committed or stale/discarded

4.2 Conversation concurrency conclusion

Messages receive monotonic sequence numbers. Read-only work for an older message may be cancelled; stale model output is discarded by expected-version checks. Only one state-changing response commits at a time per conversation. A later message can invalidate an unconfirmed proposal, but it cannot erase an external action that was already submitted. The system must report and reconcile actual state before considering any compensating or cancellation journey.

5. Failure simulation and truthful degradation

Injected failure	Required system behavior	Invariant preserved
Model timeout	Retry only inside turn deadline; use approved fallback or deterministic response	No unsupported answer or action proposal
Invalid model schema	One constrained repair; then deterministic fallback or handoff	Invalid structure never reaches workflow/tool
Retrieval insufficient/conflicting	Clarify, state uncertainty, or hand off	Model cannot use general training knowledge to fill tenant policy gaps
Retrieval unavailable	Use revocation-checked authoritative cache only when permitted; otherwise hand off	No stale or revoked policy answer
Read tool unavailable	Report unavailable state and next step	No guessed order, carrier, inventory, or account fact
Workflow rejects proposal	Reload workflow state and discard stale proposal	Model never decides that proposal succeeded
Runtime worker crash	Reprocess persisted message idempotently	Duplicate response/action suppressed
Temporal worker crash	Replay workflow history; Activities remain idempotent	No duplicate external effect
Temporal service/region outage	Pause protected transitions; fail over namespace/cell; reconcile before writes	No false continuity claim
Provider timeout before submission	Safe bounded retry when deadline and idempotency permit	No duplicate submission
Provider timeout after possible submission	Record RESULT_UNKNOWN and reconcile by idempotency key/reference	Never claim success/failure or blindly retry
Kafka unavailable	Commit local aggregate plus outbox; publish after recovery; apply admission control	Transactional state remains authoritative
Redis unavailable	Use primary stores with lower capacity; fail closed where risk limits cannot be enforced	Cache loss cannot change truth
Control plane unavailable	Use local non-revoked release snapshots; block publishing and protected actions if revocation status is unknown	Runtime does not bypass kill switches
Customer disconnect	Continue accepted durable work; stop turn-local optional generation	Submitted actions and handoffs survive
Human case creation failure	Retry idempotently; preserve pending state and alternative contact path	No false queued/assigned status
Cost budget exhausted	Stop optional calls; answer partially when safe; clarify or hand off	Authorization, confirmation, grounding, audit, and reconciliation remain intact
Regional failover during action	Fence old epoch, activate only after action-safety readiness, reconcile submitted/unknown actions	No second write in recovery region

5.1 Unknown-result state machine

1	A write may have reached the provider but no authoritative response was received.
2	Tool Gateway records RESULT_UNKNOWN with the stable action identity and blocks a second submission.
3	A reconciliation Activity queries the provider by idempotency key or external reference.
4	Confirmed success becomes completed; confirmed non-submission may permit a policy-controlled retry; unresolved state remains RECONCILING and escalates.

5.2 Release and dependency revocation

Pinned execution never overrides an emergency revocation. Agent releases, knowledge sources, model routes, specialists, prompts, tool operations, tenant automation, action classes, and runtime cells each require a kill switch. A revoked dependency causes migration to an approved compatible release, deterministic fallback, or handoff. It never silently resumes on a forbidden component.

6. Security and tenant-isolation validation

Layer	Validated control
Edge and identity	Resolve tenant from trusted host/config and verified token; short-lived audience-bound identity; step-up for sensitive operations
Application	Mandatory TenantContext; repository and service interfaces deny missing tenant/environment; no tenant override from model
PostgreSQL/DynamoDB	Tenant keys in every row/item and conditional access path; no unbounded scans; high-isolation tiers may use dedicated stores/accounts
Kafka	Tenant in envelope and partition key; consumer authorization; schema/redaction; no topic per tenant
S3 evidence	Tenant/environment prefix or access point; short-lived upload; quarantine, malware scan, MIME/size/decompression checks; controlled human access
OpenSearch retrieval	Mandatory tenant, environment, audience, ACL, product, region, locale, effective-date, and revocation filters during candidate generation
Model context	Only authorized resource handles; one tenant per provider request; no secrets; trust-labeled source sections; no cross-tenant prompt cache
Tools	Per-turn projection of registered contracts; read authorization at object/field/purpose level; no consequential write exposed to model
Action execution	Opaque single-use capability; canonical argument hash; exact preview/confirmation; current policy/approval; routing epoch; atomic redemption
Logs and traces	Tenant pseudonym, field allowlist, pre-export redaction, purpose-limited access, no full prompt/response by default
Human workspace	Queue/role/resource authorization; authoritative links generated by backend; separate representative action authority
Evaluation	Cross-tenant, prompt-injection, confirmation-bypass, leakage, and duplicate-action invariants are pass/fail release gates

6.1 Prompt-injection conclusion

DEFENSE MODEL	Detection and filtering reduce exposure, but they are not the security boundary. The decisive defense is limited authority: retrieved text cannot change ACLs or system instructions; the model receives no credentials or consequential tool; all proposals, confirmations, approvals, capabilities, arguments, and effects are independently validated.

6.2 Tenant-isolation release suite

- Attempt cross-tenant reads through every API, repository, cache, index, event consumer, export, human view, evaluation dataset, and log correlation path.
- Insert canary documents and object identifiers and verify that unauthorized candidates never influence ranking, snippets, counts, model context, or timing behavior.
- Replay valid proposals and action-capability references under a different tenant, environment, customer, workflow, state version, routing epoch, or canonical argument hash.
- Test admin and platform-operator paths for ambient content access; require audited just-in-time elevation and scoped break-glass behavior.
- Treat any observed cross-tenant exposure as severity zero and an automatic release block, not a metric to average.

7. Regional deployment and recovery validation

The cell model remains sound: each tenant has one home region and active cell, runtime calls do not synchronously depend on another cell, and a routing epoch fences stale writers. The original HLD's generic use of DynamoDB Global Tables for both conversation projections and the action ledger was not sufficiently precise for consequential-action safety. Amendment A5 splits these responsibilities.

7.1 Frozen regional rules

Concern	Binding rule
Tenant writer	Exactly one active region/cell per tenant for operational workflow and consequential writes
Fencing	Routing epoch included in every protected command and capability; stale epoch rejected at resource owner and Tool Gateway
Conversation data	May use asynchronously replicated/global projection stores with expected-version checks and application conflict handling
Action Safety Store	Confirmation, approval, capability redemption, submission ledger, and reconciliation identity require single-writer conditional semantics and zero-loss safety behavior
Temporal	Use a supported HA/failover design; Workflow code remains deterministic and external calls remain idempotent Activities
Control plane	Runtime uses replicated local release snapshots; no synchronous dependency for normal turns; publishing stops during outage
Knowledge	Replicated/rebuildable index plus source manifests; every lookup checks current revocation epoch
Failover activation	Fence old writer, verify routing/secrets/workflow/action-store readiness, reconcile submitted/unknown actions, then shift traffic
Failback	Same fenced procedure; completed effects are never rolled back by infrastructure failback

7.2 Recovery objectives by data class

Data class	Recovery objective	Safety behavior
Submitted action identity and ledger	0 effect-loss tolerance	Block new related writes until provider and ledger reconciliation completes
Confirmation and approval	May be re-collected if safely lost, but never inferred	No action without a present valid record
Workflow history	Provider/service objective may be < 1 minute RPO	Failover must expose pending and reconcile external effects before progress
Conversation message/projection	< 1 minute design objective	Rebuild from durable events/outbox where possible; never invent completion
Control configuration	< 5 minutes design objective	Use last approved non-revoked snapshot
Analytics/evaluations	15 minutes acceptable	Never on transactional recovery path

7.3 Why MREC alone is insufficient for the action ledger

AWS documents that MREC Global Tables replicate asynchronously and reconcile concurrent writes using last-writer-wins; it also warns that optimistic locking does not behave as expected across Regions [A1][A2]. Therefore, a routing epoch checked only by application code is not enough if a stale region can still write or if the recovery region has not received the last action record. LLD must implement the Action Safety Store with an appropriate strongly coordinated/single-writer design, or must hard-block recovery writes until replication and external reconciliation prove safety.

8. Ten binding amendments that close the HLD

ID	Area	Binding amendment
A1	Release/version vector	Replace the statement that every conversation pins every dependency. Pin agent/workflow compatibility; record the actual prompt/model route, knowledge release, policy, tool contract, and evaluation version used for each turn/action.
A2	Knowledge and policy freshness	Knowledge may advance between turns to a compatible active release; emergency revocation always wins. Policy is selected for the applicable purchase/action and re-evaluated before preview and execution. A changed material result invalidates preview and confirmation.
A3	Validated response delivery	V1 does not expose an unvalidated generated draft as customer truth. It may stream deterministic progress. Final factual content is fully grounded and validated before commit; transport may chunk an already validated response.
A4	Action Capability	Use an opaque, server-side, short-lived, single-use capability reference. Bind tenant, customer, workflow/state, preview/confirmation, tool/operation, canonical arguments, amount/effect, policy, approval, routing epoch, idempotency, expiry, and redemption count. The model never sees it.
A5	Action Safety Store	Do not rely on MREC Global Tables and cross-region optimistic locking for capability redemption or action ledger correctness. Use single-writer conditional semantics and a failover gate that preserves no duplicate effect and no false completion.
A6	Revocation-aware cache	Every knowledge cache and fallback lookup includes tenant, release, ACL, locale, query, and current revocation epoch. If revocation freshness cannot be checked, protected policy answers/actions fail closed or hand off.
A7	MCP and tool exposure	Tool discovery/import occurs during integration registration. Production AI receives only a per-turn projection of pre-approved versioned contracts. MCP schema or transport authorization never grants business authority.
A8	Temporal, Policy, and AI ownership	Temporal owns durable state and action ordering; Policy Service owns deterministic rules; Agent Runtime owns bounded interpretation/composition. Model and external service calls run as Activities, not deterministic Workflow code.
A9	Human handoff authority	AI handoff summaries and sentiment are labeled drafts. Backend-generated record links and authoritative state travel with the case. Accepted human ownership stops automated customer replies; release-back is explicit.
A10	Regional recovery gate	Before recovery writes, fence the old epoch, verify workflow and Action Safety Store continuity, reconcile all submitted/unknown operations, validate non-revoked releases and secrets, then shift traffic. Service RPO/RTO figures do not replace this safety gate.

8.1 Resolved interpretation of release pinning

Component	Frozen behavior	Permitted change
Agent/workflow code	Pinned for active conversation/journey	Compatible migration, security revocation, or controlled handoff
Prompt bundle/model policy	Contained in agent release; actual route recorded per call	Approved fallback inside same capability/residency policy
Knowledge release	Stable within a turn; selected and recorded per turn	May advance between turns; revocation immediate
Policy version	Selected by Policy Service for applicable facts; bound to preview/action	Re-evaluate before effect; changed result requires new preview/confirmation
Tool contract	Exact version bound to capability and invocation	Backward-compatible upgrade or explicit migration/handoff
Evaluation version	Recorded with score/outcome	Does not control transactional truth

8.2 Scope contradictions explicitly closed

- Address change remains a subworkflow, not an eighth V1 journey.
- V1 accepts image uploads as evidence but does not make autonomous visual damage judgments.
- The core architecture is channel-neutral, but V1 product scope remains web chat; email, SMS, push, and voice are post-V1.
- Bounded specialists are typed model operations with no write authority, private durable state, recursive delegation, or dynamic production discovery.
- A2A remains deferred until independently operated agents create a real trust and task boundary.
- Temporal is not the conversation database, vector store, tenant knowledge store, analytics warehouse, or large-payload repository.

9. Final architecture freeze

FREEZE STATUS FROZEN FOR LLD. There is no remaining high-level ambiguity about product scope, state ownership, AI authority, consequential-action control, tenant isolation, failure truth, regional writer authority, or the seven V1 journeys.

9.1 Frozen technology and topology decisions

Area	Frozen decision
Cloud/topology	AWS reference deployment; EKS regional runtime cells; three active regions; one active writer per tenant
Core data	Aurora PostgreSQL, DynamoDB, Temporal, Kafka/MSK, Redis/Valkey, S3, OpenSearch, ClickHouse, plus the explicitly defined Action Safety Store semantics
Interaction	HTTPS and SSE at edge; typed HTTP/gRPC internally; Kafka facts; Temporal workflows/signals/Activities; registered MCP contracts
AI pattern	One customer-visible agent, one turn orchestrator, fixed bounded specialists, no recursive swarm
Knowledge	Controlled ingestion, immutable manifests, hybrid retrieval, pre-scoring authorization, evidence packages, revocation epoch
Actions	Deterministic policy, canonical preview, exact confirmation, approval/takeover, opaque Action Capability, idempotent Tool Gateway, reconciliation
Release	Immutable agent release, per-turn dependency recording, offline/adversarial gates, shadow/canary rollout, kill switches and compatible migration
Observability	Stable internal trace/evaluation schema mapped to OpenTelemetry; full prompts/responses not logged by default

9.2 Explicitly rejected designs

Rejected	Reason
Model directly calls commerce/payment APIs	Breaks authorization, confirmation, idempotency, audit, and truthful status
Consequential tool in AI capability envelope	Lets prompt manipulation cross the deterministic action boundary
One giant global runtime	Expands blast radius and creates noisy-neighbor/recovery coupling
Fully active-active writes per tenant	Creates split-brain and duplicate-effect risk
MREC last-writer-wins action ledger	Cannot guarantee cross-region conditional redemption/optimistic locking
Exactly-once infrastructure claim	Correctness comes from idempotency, ledger, fencing, versions, and reconciliation
Vector database as universal memory	Cannot replace conversation, workflow, policy, customer, or business truth
Autonomous specialist swarm/A2A internally	Unnecessary agency, discovery, and trust overhead inside one authority boundary
Raw chain-of-thought storage	Not required for audit and creates privacy/security exposure
Unvalidated token stream as final truth	Can expose unsupported factual or action claims before validation
Automated visual damage decision in V1	Product contract supports upload/human evidence only

9.3 HLD exit criteria

- All seven V1 journeys have typed normal, alternate, failure, and escalation paths.
- Every durable state and consequential transition has exactly one authoritative owner.
- AI runtime, specialist, retrieval, model, workflow, policy, confirmation, approval, capability, tool, and human boundaries are explicit.
- Tenant isolation applies before retrieval and at every data, tool, model, event, cache, human, analytics, and export boundary.

- Unknown external results, duplicate delivery, stale model output, customer disconnect, worker crash, provider outage, cell failure, and region failure have safe behavior.
- Regional action safety no longer relies on asynchronous last-writer-wins replication.
- Release, knowledge, policy, tool, cache, and revocation behavior is consistent across backend and AI architecture.
- All unresolved decisions below are implementation proofs or detailed contracts appropriate for LLD, not missing HLD choices.

10. LLD handoff and proof obligations

The next design activity is LLD Part 1: codebase and deployable structure. The LLD must preserve the frozen boundaries while turning them into schemas, interfaces, workflows, policies, storage designs, tests, runbooks, and measurable capacity.

#	LLD package	Must prove
1	Codebase and deployables	Repository/module structure for the seven initial deployables, ownership rules, dependency direction, build/release layout
2	Identity and TenantContext	Claims, verification levels, workload identity, routing epoch propagation, repository enforcement, workforce roles
3	Conversation protocol	Message schema, monotonic sequence, idempotency, expected-version commit, outbox, SSE progress/final response
4	Workflow state machines	Seven journey definitions, signals, timers, retries, cancellation, compensation, search attributes, versioning, replay tests
5	Proposal and policy contracts	JSON schemas, rejection codes, fact versions, applicable-policy selection, obligations, approval and takeover transitions
6	Preview and confirmation	Canonical preview schemas, material-field hashes, expiry, invalidation, accessibility and audit records
7	Action Capability service	Opaque reference lifecycle, bound claims, atomic single redemption, revocation, routing epoch, abuse/race tests
8	Action Safety Store	Technology and consistency proof, transactions, idempotency keys, submission ledger, failover readiness and zero-duplicate tests
9	Tool and MCP Gateway	Registration pipeline, risk classes, credentials, read authorization, retries, circuit breakers, reconciliation adapters
10	Knowledge and retrieval	Source/ACL schema, parsing/chunking, release manifest, revocation epoch, hybrid query, pre-filtering, evidence contract, conflict logic
11	Model and prompt gateway	Capability registry, provider routing, structured output, budgets, fallbacks, prompt layers, redaction, cost accounting
12	Guardrails and evaluation	Synchronous gates, offline datasets, hard invariants, cohort metrics, judge calibration, shadow/canary release workflow
13	Regional recovery	Routing/fencing store, cell placement, Action Safety readiness, Temporal failover, provider reconciliation, failback runbook
14	Observability/privacy	Stable internal trace schema, OpenTelemetry mapping, sampling, retention, deletion, audit reconstruction, access controls
15	Capacity and economics	Load model, queue/backpressure math, provider quotas, p95/p99 budgets, cost limits, degradation modes, chaos tests

10.1 LLD release-blocking tests

- No cross-tenant candidate, record, event, cache entry, model context, human view, trace, or export is accessible.
- No return, replacement, refund, cancellation, address change, or case is duplicated under retry, replay, concurrent message, failover, or lost response.
- No consequential action executes without exact valid confirmation and all required approval/takeover gates.
- No response states completed before the external system or later reconciliation confirms the effect.
- No stale routing epoch or previously redeemed capability can execute an action.
- No revoked knowledge, prompt, model route, specialist, tool, release, or tenant automation remains usable after its enforcement deadline.
- No regional recovery admits related writes before action ledger and provider reconciliation are safe.

11. Sources and validation references

11.1 Internal architecture sources

[I1] Customer Agent OS Lite - V1 Product Contract, July 2026, especially pp. 6-18.

[I2] Customer Agent OS Lite - Combined High-Level Design v1.0, July 20, 2026, especially pp. 6-24 and 33-38.

[I3] Customer Agent OS Lite - AI HLD Part 1: Runtime, Orchestration, Tools, Workflows and Memory, regenerated July 21, 2026.

[I4] Customer Agent OS Lite - AI HLD Part 2: Knowledge, RAG, Models, Guardrails, Evaluations and Operations, regenerated July 21, 2026.

[I5] Customer Agent OS Lite - Fixed Product Features; Project Overview, July 2026.

11.2 Primary platform references

[T1] Temporal Workflow Execution
https://docs.temporal.io/workflow-execution

[T2] Temporal Activities
https://docs.temporal.io/activities

[T3] Temporal Cloud RPO and RTO
https://docs.temporal.io/cloud/rpo-rto

[A1] AWS DynamoDB Global Tables
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GlobalTables.html

[A2] AWS DynamoDB optimistic locking and Global Tables
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/DynamoDBMapper.OptimisticLocking.html

[O1] OpenSearch hybrid search and pre-filtering
https://docs.opensearch.org/latest/vector-search/ai-search/hybrid-search/index/

[M1] MCP tools specification 2025-11-25
https://modelcontextprotocol.io/specification/2025-11-25/server/tools

[M2] MCP authorization specification 2025-11-25
https://modelcontextprotocol.io/specification/2025-11-25/basic/authorization

[OT1] OpenTelemetry semantic conventions
https://opentelemetry.io/docs/specs/semconv/

KLEEM AI / LOW-LEVEL DESIGN

Customer Agent OS Lite

LLD Part 1: Codebase and Deployable Structure

Buildable repository, module, contract, configuration, local-development, test, and release boundaries for the seven frozen V1 deployables.

STATUS	PART 1 FROZEN - READY FOR LLD PART 2
Architecture	One monorepo; seven release units; TypeScript-first online runtime; contract-first service boundaries
Preserves	Seven V1 journeys, ten HLD amendments, deterministic action authority, regional cells, tenant isolation
Version	1.0 - July 21, 2026

Design rule: repository convenience must never weaken the frozen authority boundary. AI code can interpret and propose. Only deterministic workflow, policy, confirmation, capability, and integration modules can authorize and execute consequential effects.

1. Part 1 decision

FINAL PART 1 DECISION	Adopt one governed monorepo containing seven independently versioned release units. All interactive and transactional production services use Node.js 24 LTS with strict TypeScript. Python 3.13 is permitted only for offline knowledge-processing and evaluation tooling in V1. Cross-unit communication uses generated contracts; no deployable imports another deployable's source or database adapter.

This choice optimizes for correctness and delivery speed. The online request path uses one language, one dependency graph, one observability wrapper, one workflow SDK, and one contract-generation pipeline. A future Python runtime service remains possible behind an existing gRPC boundary, but it is not created until measured quality or library needs justify another independently operated service.

1.1 Outcomes frozen by this LLD

- Seven release units exactly mirror the HLD physical boundary: Edge/API, Conversation Runtime, Agent Runtime, Workflow Workers, Integration Gateway, Human Operations, and Control and Knowledge.
- A release unit is an independently versioned product boundary; a workload role is a separately scaled Kubernetes Deployment, Job, or static surface shipped by that release unit.
- Business modules stay inside their owning release unit. Shared packages contain only contracts, platform primitives, generated clients, and test kits.
- REST/OpenAPI is used at public edges, gRPC/Protobuf for synchronous internal calls, Protobuf for Kafka events, and JSON Schema for model and tool contracts.
- Temporal Workflow code and Activity code live in separate projects with compile-time import restrictions.
- Consequential connector code and Action Capability redemption code are physically absent from the Agent Runtime dependency graph.
- Configuration is split into bootstrap config, immutable release artifacts, tenant config, and authoritative dynamic safety state. Secrets are references resolved through workload identity.
- Local development supplies deterministic model and commerce simulators and uses real protocol-compatible infrastructure for integration tests.
- Every image, chart, contract bundle, workflow build, migration set, and UI artifact is traceable to a Git commit and content digest.

1.2 Scope and exclusions

Class	Boundary
Included	Repository tree, internal module pattern, release/workload boundaries, dependency rules, contract layout, config/secrets structure, local environment, build/release packaging, CI gates, ownership, implementation sequence.
Deferred	Exact TenantContext schema (Part 2), database keys and tables (Part 3), message APIs (Part 4), event topics (Part 5), workflow state machines (Part 6), connector/capability details (Part 7/8), AI and retrieval internals (Parts 8/9), production runbooks and capacity proof (Part 10).
Not reopened	Product journeys, thresholds, regional single-writer strategy, Action Safety requirement, Temporal ownership, AI authority, release/revocation semantics, or the ten integrated-freeze amendments.
Not V1	Voice, email/SMS channels, additional domains, free-form agent builder, autonomous damage-image judgment, A2A between internal specialists, unbounded agent swarms.

1.3 Document map

Section	Purpose
2	Technology and toolchain baseline
3	Canonical monorepo layout
4	Seven release units and workload roles
5	Internal module architecture
6	Dependency and authority enforcement
7	Contract system and compatibility
8	Configuration, secrets, and release resolution
9	Local developer environment
10	Build, CI, testing, and release packaging
11	Deployment source layout and regional cells
12	Implementation sequence and ownership
13	HLD amendment preservation and acceptance

2. Technology and toolchain baseline

Area	Decision	Reason
Online runtime	Node.js 24 LTS; strict TypeScript	Node 24 is an LTS production line as of the freeze date [N1]. One hot-path language reduces deployment and incident complexity.
Service framework	NestJS with Fastify adapter	Consistent modules, dependency injection, lifecycle, validation, REST/SSE and gRPC support; Fastify supplies the HTTP engine.
Workflow	Temporal TypeScript SDK	Workflow code is sandboxed and deterministic; external calls remain Activities; worker versioning and replay tests are mandatory [T1][T2].
Public APIs	OpenAPI 3.1 over HTTPS; SSE for progress/final delivery	Browser-friendly, generated clients, explicit errors and idempotency. No unvalidated model-token stream exists.
Internal APIs	Protobuf plus gRPC	Typed service boundaries, deadline/cancellation propagation, generated clients, compatibility gates.
Events	Protobuf envelopes over Kafka/MSK	Schema-governed asynchronous facts; registry publication and breaking checks.
AI/tool schemas	JSON Schema 2020-12 plus generated TypeScript validators	Provider-neutral structured outputs, MCP contracts, proposal validation, canonical hashing.
Web surfaces	React, TypeScript and Vite	Customer widget, human console, and admin console are static assets coupled to their owning release units.
Workspace/build	pnpm workspaces plus Nx	Single lockfile, project graph, affected builds, caching, ownership tags, module-boundary checks.
Offline AI tools	Python 3.13 plus uv	Used for batch parsing, corpus analysis, and evaluation notebooks/scripts only; no V1 interactive service depends on Python.
Infrastructure source	Terraform modules, Helm charts, Argo CD application definitions	Cloud resources, Kubernetes packaging, and GitOps state remain versioned separately from application code.
Telemetry	OpenTelemetry through a Kleem-owned stable wrapper	Internal attribute contract remains stable even when upstream GenAI conventions evolve.

2.1 Version policy

- Runtime majors are pinned in **mise.toml**; package-manager and dependency resolutions are pinned in lockfiles; container bases are pinned by digest.
- Production uses Active or Maintenance LTS runtimes only. Major upgrades require an ADR, compatibility suite, replay suite, and canary; patches flow through automated dependency pull requests.
- No floating tags such as latest are allowed in CI, Helm, Docker Compose, or generated-code plugins.
- A release manifest records actual runtime, SDK, schema compiler, workflow build, prompt bundle, and generated-client versions. Version labels never replace content digests.

2.2 Why TypeScript-first online

Disposition	Option	Rationale
Selected	One TypeScript runtime for gateways, services, workflows, model routing, retrieval orchestration and connectors	Fast vertical slices; shared generated types without shared business logic; first-class Temporal determinism and replay tooling.
Rejected for V1	Python Agent Runtime plus TypeScript backend	Creates two on-call stacks and more serialization boundaries before an actual Python-only capability is required.
Reserved	Python service behind agent-runtime gRPC port	May be added only after a benchmark shows a material evaluated quality/cost/latency advantage and an ADR assigns ownership/SLO.

3. Canonical monorepo layout

The repository is a monorepo for atomic contract changes, shared CI policy, reproducible releases, and rapid development by a small initial team. It is not a distributed monolith: seven release units have independent package boundaries, images, charts, owners, data adapters, and version manifests.

```
kleem-ai/
  deployables/
    edge-api/                conversation-runtime/
    agent-runtime/           workflow-workers/
    integration-gateway/     human-operations/
    control-knowledge/
  surfaces/
    customer-widget/         operations-console/      admin-console/
  contracts/
    public-api/              internal-api/             events/
    workflows/               ai-io/                    tools/
    generated/typescript/    generated/python/
  packages/
    platform/config/         platform/context/         platform/telemetry/
    platform/errors/         platform/transport/       platform/crypto/
    testkit/contracts/       testkit/tenancy/           testkit/temporal/
  tools/
    python/ingestion/        python/evaluation/         simulators/
    codegen/                 ci/                        dev/
  config/
    schemas/                 defaults/                 local/
  deploy/
    helm/                    argocd/                   policies/
  infra/
    terraform/modules/      terraform/environments/
  tests/
    contract/               component/               journeys/
    isolation/              replay/                  failure/
  docs/
    adr/                    architecture/            runbooks/
  nx.json  pnpm-workspace.yaml  pnpm-lock.yaml  mise.toml  justfile
```

3.1 Directory rules

Path	Rule
deployables/	Only code shipped in one of the seven online release units. A deployable may have multiple process entry points but one owner and compatibility manifest.
surfaces/	Browser assets. Each surface is attached to an owning release: customer-widget to Edge/API, operations-console to Human Operations, admin-console to Control and Knowledge.
contracts/	Human-authored boundary definitions and generated clients. Generated code is never edited manually and cannot contain business policy.
packages/platform/	Narrow technical primitives only: context propagation, errors, telemetry, transport, config, cryptographic wrappers. No order/refund/policy/journey domain types.
packages/testkit/	Deterministic fixtures, simulators and invariant assertions. Production code cannot import testkit projects.
tools/python/	Offline or CI jobs only. Importing these projects from an online deployable fails architecture checks.
config/	Schemas and non-secret defaults. Tenant config and release snapshots are not checked in as production truth.
deploy/ and infra/	Helm/GitOps workload definitions and Terraform infrastructure. Application code cannot invoke Terraform or mutate cluster configuration.
tests/	Cross-unit tests and proof suites. Unit/module tests remain beside their owner.

3.2 Standard deployable layout

```
deployables/<release-unit>/
  project.json
  package.json
  Dockerfile
  src/
    bootstrap/        # process composition, startup checks, health
    modules/          # owner-scoped business capabilities
    transport/        # inbound REST/gRPC/event/Temporal adapters
    platform/         # release-local technical adapters
    entrypoints/      # api, worker, consumer, job commands
  test/
    unit/             # pure domain/application tests
    component/        # real datastore/broker/provider simulator
    contract/         # provider and consumer compatibility
  migrations/         # owner-only, expand/contract; exact design in LLD Part 3
  release/
    compatibility.yaml
    workload-roles.yaml
```

Every business module follows ports-and-adapters internally: domain rules at the center, application commands/queries around them, typed ports at the boundary, and transport/storage/provider adapters outside. Framework decorators and SDK types do not enter domain code.

4. Seven release units and workload roles

IMPORTANT DISTINCTION

The HLD's seven deployables are seven release and ownership boundaries, not seven Kubernetes pods. A release unit may ship multiple workload roles from one source version so APIs and workers can scale independently without creating another independently versioned service.

Release unit	Image family	Workload roles	Owns	Must not own
1. Edge/API	edge-api	edge-api Deployment; customer-widget static artifact	Customer/operations gateway, auth adapters, admission, REST, SSE, request routing	Business repositories; model/provider SDKs; workflow truth
2. Conversation Runtime	conversation-runtime	conversation-api; conversation-event-consumer	Ordered messages, participants, attachments metadata, response commit, runtime projections, AI/human response control	Action completion, policy decision, connector credentials
3. Agent Runtime	agent-runtime	agent-interactive; agent-background	Context assembly, one orchestrator, five bounded specialists, retrieval/model gateways, grounding and response validation	Consequential connectors, Action Capability, durable workflow state
4. Workflow Workers	workflow-workers	workflow-interactive; workflow-background; policy-api	Seven journeys, address subworkflow, deterministic policy module, previews, confirmation/approval waits, timers, reconciliation coordination	Provider credentials; model prompts; direct connector calls
5. Integration Gateway	integration-gateway	integration-api; reconciliation-worker	Approved tool execution, credential broker, connector adapters, action-safety boundary, idempotent submission, reconciliation	Conversation text; model orchestration; customer-visible completion wording
6. Human Operations	human-operations	human-api; queue-worker; operations-console artifact	Cases, queues, assignment, approvals, takeover/release-back, notes, human projections	Workflow mutation outside signals; AI prompt release; connector credentials
7. Control and Knowledge	control-knowledge	control-api; knowledge-worker; evaluation-worker; admin-console artifact; release jobs	Tenant config, integration registration, agent/knowledge releases, ingestion, evaluation management, admin APIs and publication	Synchronous normal-turn dependency; customer conversation state; action execution

4.1 Runtime interaction path

1	Customer widget calls Edge/API. Edge authenticates, resolves trusted routing context, admits the request, and forwards a typed command.
2	Conversation Runtime persists and sequences the message, writes its outbox record, and returns the durable message identity.
3	Agent Runtime receives a turn task, resolves approved release context, performs bounded reads/retrieval, and emits a validated response or typed workflow proposal.
4	Workflow Workers accept/reject proposals and own all durable journey transitions. Policy, preview, confirmation, approval and handoff waits stay outside the model.
5	Integration Gateway performs authorized reads or redeems an opaque Action Capability for a consequential write; unknown results enter reconciliation.
6	Conversation Runtime commits only validated factual content and authoritative progress/outcome. Edge may chunk the already validated response over SSE.

4.2 Control-plane publication path

- Control and Knowledge builds immutable agent, prompt, integration-contract and knowledge artifacts after approval/evaluation gates.
- Published manifests are content-addressed, signed, and distributed asynchronously to regional cells. Normal turns never synchronously call the control plane.
- Runtime release resolvers maintain a last-known-approved local snapshot but check authoritative revocation and kill-switch state within the required freshness window.

- A release job changes future selection. It does not mutate a running conversation, Temporal history, submitted action, or audit record.

5. Internal module architecture

Logical services remain visible as modules even when physically co-located. This creates split-ready seams without paying the operational cost of a network hop for every logical boundary. Each module owns its domain, application operations, ports, adapters, tests, migrations, telemetry names and error catalog.

Owner	Required modules	State boundary
Edge/API	customer-gateway; operations-gateway; authentication; admission; realtime-delivery; routing	Only transport/session state; Redis limiter data is disposable
Conversation Runtime	conversation; message-ledger; participant-control; attachments; response-commit; runtime-projection; outbox	Conversation records, monotonic sequence, visible response and control mode
Agent Runtime	release-resolution; context-assembly; orchestrator; specialists/triage; specialists/knowledge; specialists/customer-operations; specialists/risk-policy; specialists/handoff; retrieval-gateway; model-gateway; response-validation	Turn-local state only; telemetry/evidence references; no private durable agent memory
Workflow Workers	journeys/tracking; eligibility; return; replacement; refund; cancellation; handoff; subworkflows/address-change; policy; preview; confirmation; approval; reconciliation-coordinator	Temporal history and owner-specific policy/confirmation records; no large payloads
Integration Gateway	approved-tool-registry; business-read; credential-broker; action-safety; action-execution; connector-runtime; reconciliation; circuit-breaker	Tool contract runtime, submission/reconciliation identity; Action Safety Store through one owner
Human Operations	case; queue; assignment; approval-workbench; takeover; release-back; notes; human-projection	Human case, assignment and internal records; workflow changes only via typed signals
Control and Knowledge	tenant-config; agent-release; integration-registration; source-registry; ingestion; knowledge-release; evaluation; rollout; admin-api; publication	Control-plane records, immutable manifests, source/eval metadata; not live journey truth

5.1 Workflow code versus Activities

Project	May import	Forbidden
workflow-core project	Temporal workflow APIs, pure workflow state, type-only Activity contracts, deterministic helpers	HTTP/gRPC clients, database drivers, filesystem, provider SDKs, secrets, ordinary clocks/randomness, model calls
activities project	Generated service clients, model/tool providers when authorized, database/repository adapters, tracing	Direct mutation of Temporal history; pretending an Activity result is an external fact after timeout
worker bootstrap	Task-queue registration, interceptors, build ID, metrics, graceful shutdown	Journey policy or business decisions

Temporal's TypeScript SDK enforces important determinism properties, but repository rules remain explicit because determinism failures can also enter through local utility imports. Workflow replay against captured histories is a release gate, not an optional test [T1][T2].

5.2 Bounded specialist placement

Specialist	Responsibility	Boundary
Triage	Intent/entities/route proposal	Typed input/output; fixed budget; no tool write
Knowledge	Evidence query and synthesis	Retrieval Gateway only; evidence IDs required
Customer Operations	Select authorized business reads and interpret results	Per-turn read projection; cannot access consequential connectors
Risk and Policy	Extract risk signals and explain deterministic policy result	Cannot produce authoritative allow/deny/threshold
Handoff	Draft representative summary and customer transition wording	Draft label required; backend supplies links/state

Specialists are modules inside Agent Runtime, not separately deployed agents. They have no recursive delegation, private durable memory, production discovery, credentials, or independent release authority.

6. Dependency and authority enforcement

NON-NEGOTIABLE

No release unit imports another release unit's source code, migrations, repositories, framework modules, or configuration. Integration happens only through generated clients, events, Temporal contracts, or published immutable artifacts.

Rule	Allowed direction	Enforced prohibition
D1	surfaces -> generated public clients	A browser surface cannot import server modules or construct internal TenantContext.
D2	edge-api -> generated service clients	Edge cannot query service databases, call model providers, or import workflow implementation.
D3	deployable -> contracts/platform primitives	A deployable may use approved narrow shared packages; never another deployable source.
D4	domain -> nothing framework-specific	Domain code cannot import NestJS, Temporal, AWS SDK, Kafka, OpenSearch, database clients, provider SDKs or generated transport types.
D5	application -> domain and local ports	Application code coordinates local domain behavior through interfaces; adapters implement those ports.
D6	workflow-core -> workflow-safe packages only	No networking, I/O, secrets, non-deterministic SDK or Activity implementation import.
D7	agent-runtime !-> consequential write contracts	Write connector packages, Action Capability reference type and credential modules are not visible in the Agent Runtime project graph.
D8	integration-gateway !-> AI internals	Connector execution cannot import prompts, specialists, conversation transcript assembly, or model router.
D9	control-plane !-> runtime truth	Control jobs publish immutable artifacts; they do not edit live conversation/workflow/action records.
D10	production !-> testkit	Simulators, fakes and fixtures cannot appear in a production bundle.

6.1 Enforcement mechanisms

Mechanism	Checks	When
Nx project tags	scope:edge, scope:conversation, scope:agent, scope:workflow, scope:integration, scope:human, scope:control, type:domain/application/port/adapter/entrypoint	Static dependency matrix in CI
ESLint restricted imports	Bans specific SDKs and path families by project type	Local editor and CI
dependency-cruiser	Detects cycles, cross-deployable source edges and forbidden transitive paths	Architecture test job
package exports	Only explicit public entry points; internal folders unavailable	TypeScript compiler/package resolution
separate package manifests	Agent Runtime cannot resolve connector-write or Action Safety packages	Install/build graph
container content assertions	Scan final image for forbidden provider credentials, test fakes and source paths	Image verification
CODEOWNERS	Contract, workflow, action-safety, integration, policy, identity and deploy definitions require named review	Pull request protection

6.2 Release-blocking architecture tests

- A synthetic import from Agent Runtime to any consequential connector or capability-redemption implementation must fail.
- A Temporal workflow importing an HTTP client, database adapter, model SDK or filesystem module must fail.
- An Edge module importing a Conversation repository or migration must fail.
- Any deployable importing another deployable's source path, including transitively, must fail.
- Any production entry point importing testkit or simulator code must fail.
- Every inbound service handler must declare the trusted context interceptor; every repository port must require TenantContext once Part 2 lands.
- No public SSE/OpenAPI schema can contain an event representing raw/unvalidated generated tokens.

7. Contract system and compatibility

Boundary	Canonical source	Owner	Gate
Public REST/SSE	contracts/public-api/openapi/*.yaml	Edge/API	Generated browser clients; OpenAPI lint and breaking diff; explicit idempotency/error/progress/final schemas
Internal synchronous	contracts/internal-api/proto/kleem//v1/*.proto	Providing release unit	Buf lint/generate/breaking with WIRE_JSON rules; deadlines and typed errors
Kafka facts	contracts/events/proto/kleem/events/v1/*.proto	Fact owner	Registry compatibility, stable field numbers, idempotent consumer expectations
Temporal	contracts/workflows//v1	Workflow Workers	Start/signal/query/result types; replay compatibility; build-ID compatibility
AI structured I/O	contracts/ai-io//v1/*.json	Agent Runtime	JSON Schema validation; evidence/claim mapping; bounded decision codes
Workflow proposals	contracts/workflows/proposals/v1/*.json	Workflow Workers	Expected state/version, proposal identity and rejection codes
Tool/MCP	contracts/tools///*.json	Integration registration owner	Approved versioned contracts; runtime projection; risk/side-effect metadata
Release manifests	contracts/releases/v1/*.json	Control and Knowledge	Content hashes, compatibility vector, revocation metadata, image/chart/workflow identities

7.1 Contract authoring rules

- The provider owns the canonical contract and fixtures; consumers use generated clients. Hand-written duplicate DTOs are prohibited at network boundaries.
- Contract versions are explicit namespaces. Additive changes are preferred; removed Protobuf fields and enum numbers are reserved; unknown enum values are handled safely.
- A contract change includes positive, negative, authorization, timeout, duplicate, stale-version and redaction examples where applicable.
- Internal service errors use stable machine codes plus retry classification. Human text is composed at the edge or owning UI, not embedded as a distributed protocol guarantee.
- Large payloads never cross Kafka, gRPC or Temporal history. Contracts carry immutable S3 resource handles with tenant, content hash and authorization metadata.
- Canonical serialization used for preview/action hashing is versioned and tested across generated implementations. Generic JSON.stringify is not a security primitive.

7.2 Compatibility workflow

1	Author edits the canonical OpenAPI, Protobuf or JSON Schema source and examples.
2	Local generation produces TypeScript and approved Python clients into contracts/generated; CI verifies no generated drift.
3	Lint, schema validation and breaking-change checks compare against the production baseline. Buf supplies deterministic Protobuf breaking checks [B1].
4	Provider and affected consumer contract tests run. Workflow changes additionally run replay against retained histories.
5	Release manifest declares minimum/maximum compatible contract and workflow versions. Deployment blocks if a required consumer is incompatible.
6	Contracts publish before providers; providers deploy before optional new consumers. Removals follow measured consumer retirement and an explicit deprecation window.

7.3 Trusted context envelope reservation

Part 2 defines exact fields, claims and signatures. Part 1 reserves one mandatory generated envelope for tenant ID, environment, principal, verification level, home cell, routing epoch, request/trace identity and purpose. Transport adapters create or verify it; downstream business payloads cannot override it. Events and tool requests carry the trusted envelope separately from model/customer data.

8. Configuration, secrets, and release resolution

Layer	Examples	Source	Change model
Build identity	Git SHA, image digest, package lock hash, contract bundle hash, SBOM, workflow build ID	Embedded release manifest	Immutable
Bootstrap environment	region, cell, service name, endpoints, queue names, secret references, telemetry destination	Helm values/ConfigMap and workload identity	Restart or rollout
Tenant configuration	enabled journeys, approved integrations, locale, quotas, data policy, agent selection	Control-plane store -> signed immutable snapshot	Published release
Agent/knowledge release	prompt/model policy, specialist set, tool projection, knowledge manifest, compatibility	Content-addressed artifact in S3 plus local cell cache	Per-turn resolution; revocable
Dynamic safety state	kill switches, revocation epoch/denylist, routing epoch, writer fence, action blocks	Authoritative safety/routing stores and cell-local watcher	Immediate within enforcement target
Request context	tenant, environment, principal, customer verification, workflow/message identities	Verified channel and internal propagation	Per request/turn

8.1 Configuration rules

- Each workload has one versioned configuration schema. Startup validates required values, unknown keys, ranges, URL schemes, region/cell consistency and production-safe defaults before readiness becomes true.
- No tenant-specific behavior is encoded in environment variables, Helm conditionals or application branches. It is published as controlled tenant/release data.
- Dynamic config is read through owner clients with explicit freshness and failure behavior. A cached value may not silently outlive its safety TTL.

- Every turn records the resolved dependency vector actually used; active conversation pinning applies only as defined by Amendment A1.
- Configuration logs include a non-secret fingerprint and source version, never raw secrets or private tenant content.

8.2 Secret handling

Concern	Decision	Guardrail
Production source	AWS Secrets Manager/KMS-backed references resolved with IRSA/workload identity	No long-lived access keys in environment, image, Git, prompt, Kafka or Temporal payload
Provider credentials	Only Model Gateway and Integration Gateway workload identities can resolve their approved secret namespaces	Agent orchestrator receives no raw provider/integration credential
Rotation	Version-aware cache with short TTL and dual-key overlap where provider supports it	Rotation events and failures audited; no restart-only assumption
Local	.env.local or local secret files excluded from Git; fake keys for simulators	Real production credentials prohibited in default developer profiles

8.3 Release manifest

```
release_unit: agent-runtime
source_commit: <git-sha>
image_digest: sha256:<digest>
contract_bundle: sha256:<digest>
runtime: node-24
agent_release_compatibility: [agent-v1]
workflow_contract_compatibility: [journey-v1]
prompt_bundle_digest: sha256:<digest>
generated_clients_digest: sha256:<digest>
sbom_digest: sha256:<digest>
required_config_schema: agent-runtime-config-v1
```

This deployment manifest is distinct from the product's **agent_release_id**. The former identifies code and artifacts; the latter selects approved AI behavior. Actual model route, knowledge release, policy version, tool contract and evaluation version remain recorded per turn/action rather than falsely pinned forever.

9. Local developer environment

Local development must be reproducible on macOS arm64 and Linux, cheap enough for routine use, and strict enough to catch protocol and ownership mistakes. The default path uses deterministic simulators; real model or commerce credentials are opt-in and never required for the test suite.

Compose profile	Services	Use
core	Aurora-compatible PostgreSQL container; DynamoDB Local; Valkey; Redpanda; Temporal dev server; MinIO; identity stub	Daily service and workflow development
search	OpenSearch plus prepared seed corpus	Knowledge/retrieval component tests
analytics	ClickHouse	Trace/evaluation projection tests
observe	OpenTelemetry Collector plus local trace/metric/log viewer	End-to-end correlation and redaction checks
full	core + search + analytics + observe	Journey acceptance, failure injection and release rehearsal

9.1 Simulators

Simulator	Behavior	Purpose
model-simulator	Deterministic structured responses keyed by scenario; invalid schema, timeout, rate limit, safety block and cost-budget cases	No live provider dependency in CI
commerce-simulator	Orders, shipments, returns, inventory, replacement, refund and cancellation reference APIs	Stable read/write/idempotency fixtures
mcp-registration-fixture	Approved and malicious tool schemas, OAuth metadata and schema changes	Registration/risk/projection tests
provider-fault-proxy	Timeout before submit, timeout after submit, connection reset, duplicate reply, stale read	RESULT_UNKNOWN and reconciliation proof
identity-stub	Signed development identities for tenants, roles and verification levels	No unchecked tenant header shortcut

9.2 Developer commands

```
just bootstrap                   # verify pinned tools and install locked deps
just infra core                 # start the default local profile
just dev edge conversation agent # run selected affected services with reload
just generate                   # regenerate all contract clients
just check                      # format, lint, types, boundaries, generated drift
just test unit                  # affected unit/module tests
just test contracts             # API/event/schema compatibility
just test replay                # Temporal replay suite
just test journey tracking      # deterministic end-to-end journey
just test isolation             # cross-tenant negative suite
just down                       # stop local workloads without deleting volumes
```

Destructive cleanup is a separate explicit command with an exact project-scoped target. Ordinary shutdown preserves local data. Seed data uses synthetic tenants and contains no copied production conversation or credential.

9.3 Local safety defaults

- Consequential writes target the commerce simulator only. A production integration endpoint is rejected by local configuration validation.
- Action execution defaults to deny until the Action Safety adapter selected in the later LLD is present. Unit fakes cannot satisfy component or journey acceptance tests.
- Model responses are buffered and validated exactly as production responses. The local UI never receives a privileged raw-token stream.
- Search caches carry tenant/release/ACL/locale/query/revocation dimensions even in local mode; no simplified cross-tenant cache implementation is allowed.

10. Build, CI, testing, and release packaging

Stage	Gate	Required result
1	Change graph	Nx identifies affected projects; contract/platform changes expand to all transitive consumers.
2	Fast static gates	Format, TypeScript/Python lint, strict type check, secret scan, license policy, generated-code drift.
3	Architecture	Nx boundaries, restricted imports, dependency cycles, forbidden package and production/test graph assertions.
4	Contracts	OpenAPI/JSON Schema validation, Buf lint/generate/breaking, examples, provider/consumer compatibility.
5	Unit/module	Pure rules, application behavior, schema validation, negative authorization and budget behavior.
6	Component	Owner service against real protocol-compatible datastores/brokers and deterministic provider simulators.
7	Workflow replay	New worker code replays retained representative histories and patch/build-ID cases.
8	Invariant suites	Tenant isolation, stale version, duplicate request, concurrent confirmation, capability race, unknown result, revocation.
9	Journey	Seven end-to-end deterministic paths plus alternate, failure and human transitions.
10	Supply chain	Hermetic image build, SBOM, vulnerability/malware scan, provenance, signature and policy verification.
11	Publish	Immutable image digests, Helm chart, UI artifact, contract bundle and release manifest.
12	Environment	Ephemeral/staging deployment and smoke/contract/replay checks; rollout details remain Part 10.

10.1 Test ownership

Suite	Owner	Focus
Unit	Owning module	Domain rules, transformations, validators, budgets, retry classifiers
Component	Owning release unit	Real datastore/broker/protocol adapter with simulator
Contract	Provider plus consumer	Compatibility, examples, error and timeout semantics
Replay	Workflow Workers	Determinism and long-running workflow compatibility
Journey	Cross-unit architecture	Normal, alternate, failure, handoff, duplicate and concurrency behavior
Isolation	Security/platform	Every API, repository, cache, event, search, model context, trace, export and human view
Failure/chaos	Owner of failed dependency plus journey owner	Truthful degradation, retry safety, reconciliation and backpressure

10.2 Database and workflow release rules

- Database migrations are owned by one release unit, run as explicit pre-deploy jobs, use expand/migrate/contract, and never execute automatically at service startup.
- A release cannot read another unit's tables merely because schemas share an Aurora cluster. Cross-owner data moves through APIs, events or purpose-built projections.
- Destructive schema contraction occurs only after old code, consumers, workflows and rollback windows no longer require the field.
- Temporal Worker Versioning/build IDs and patch markers protect long-running histories. CI replays before a worker is eligible for canary [T2].

- Event consumers use inbox/deduplication and explicit schema versions; infrastructure delivery is treated as at-least-once.

10.3 Release-unit artifact set

Artifact	Contents	Rule
OCI image(s)	One source commit; separate entrypoints or slim role images when contents/permissions differ	Digest signed; no floating tag
Helm chart	Workload roles, service accounts, config schema, probes, resources, network policy	Chart and app compatibility declared
Contract bundle	Provided OpenAPI/Protobuf/events/JSON schemas and examples	Content digest and compatibility result
Release manifest	Source, dependency locks, generated clients, migrations, workflows, UI, SBOM	Machine-verifiable provenance
UI artifact	Only for owning Edge/Human/Control release	Subresource/content digest and rollback pointer

11. Deployment source layout and regional cells

```
deploy/
  helm/
    edge-api/                 conversation-runtime/      agent-runtime/
    workflow-workers/         integration-gateway/       human-operations/
    control-knowledge/
  argocd/
    base/                     environments/dev/          environments/staging/
    environments/prod/regions/<region>/cells/<cell>/
  policies/
    image-policy/             network-policy/            admission-policy/

infra/terraform/
  modules/
    eks-cell/                 networking/                aurora/
    dynamodb/                 msk/                       temporal/
    valkey/                   s3/                        opensearch/
    clickhouse/               routing-fencing/           action-safety/
  environments/<environment>/<region>/
```

11.1 Packaging rules

Concern	Rule
Namespace	Regional runtime cells and control plane have distinct namespaces and policies; tenant isolation is not delegated to namespaces.
Identity	Every workload role has its own Kubernetes service account and least-privilege cloud identity; roles within one release unit do not automatically share permissions.
Network	Default deny; only declared service, broker, datastore, provider and control channels are reachable; egress uses allowlisted gateways/endpoints.
Scaling	API, interactive worker, background worker, consumer and reconciliation roles scale on their own queue/latency/concurrency signals.
Health	Liveness means process health; readiness verifies required local dependencies and release/config validity without making dangerous cross-region calls.
Shutdown	Stop admission/polling, drain in-flight bounded work, preserve durable workflow/activity semantics, then terminate within declared grace period.
Artifact promotion	The same signed image digest and chart version move from staging to production. Environment-specific values never rebuild application code.

11.2 Cell dependency rule

A runtime cell has no synchronous dependency on another runtime cell for normal turns. Shared providers and control data are accessed through regional endpoints or replicated approved snapshots. Tenant routing selects one active cell and writer epoch. Recovery deployment code can route traffic only after the separate regional-safety procedure proves workflow and Action Safety continuity, fences the old epoch, and reconciles submitted/unknown actions.

11.3 Control plane versus runtime

Plane	Contains	Responsibility
Runtime cell	Edge, Conversation, Agent, Workflow, Integration, Human workload roles; local caches and assigned partitions	Interactive turns and durable journeys
Control plane	Control/Knowledge APIs, ingestion/evaluation jobs, release publication and admin surface	Configuration and immutable artifact lifecycle
Publication bridge	Signed manifests/artifacts, Kafka facts, revocation/safety watchers	Asynchronous distribution; no synchronous normal-turn control call

12. Implementation sequence and ownership

SEQUENCING RULE

This Part 1 document defines executable scaffolding, but journey implementation starts only after all LLD parts are complete. When implementation begins, build vertical slices through customer outcomes rather than finishing every database, gateway or AI subsystem in isolation.

Milestone	Vertical slice	Proof
0	Repository spine	Create seven empty release units, contract generation, context/telemetry/config primitives, local core profile, CI boundary tests, signed hello-world artifacts. Add a minimal safe human-contact fallback.
1	Order tracking	Proves auth/context, conversation ordering, outbox, Agent read loop, Integration authorized read, validated response and SSE.

Milestone	Vertical slice	Proof
2	Eligibility	Adds authoritative business facts, deterministic policy, knowledge explanation, conflict/insufficient evidence and policy version recording.
3	Create return	Adds Temporal journey, canonical preview, exact confirmation, Action Capability boundary, idempotent write and authoritative completion.
4	Damaged replacement	Adds evidence upload, human review, inventory, threshold approval, address subworkflow, partial/unknown result and recovery.
5	Refund	Proves financial thresholds, approval/takeover, material-change reconfirmation and 100-percent unknown-result reconciliation review.
6	Cancellation	Proves effect explanation, retention constraint, contract/balance exception and scheduled-state idempotency.
7	Full handoff	Completes queue/assignment/takeover/release-back, authoritative context package, draft AI summary labeling and response suppression.

12.1 Ownership model

Owner	Accountability
Release-unit owner	Source, SLO, on-call, dependencies, datastore adapters, migrations, dashboards, alerts, runbook and compatibility
Contract owner	Schema meaning, examples, compatibility policy, consumer inventory, deprecation and registry publication
Journey owner	End-to-end state correctness, normal/alternate/failure/handoff acceptance and business outcome metrics
Platform owner	Build graph, config/context/telemetry primitives, base images, CI policy, clusters and developer experience
Security owner	Identity, tenant isolation, action boundary, secret policy, architecture tests and hard release invariants

A small initial team may assign several roles to the same people, but the responsibilities and required approvals remain explicit. Source colocation never implies shared authority.

12.2 Mandatory ADR set

ADR	Decision to record
ADR-001	TypeScript-first online runtime and Python offline boundary
ADR-002	Seven release units versus workload roles
ADR-003	Contract families and generated-client policy
ADR-004	Ports-and-adapters module pattern and shared-package restrictions
ADR-005	Configuration layers, secret references and release manifests
ADR-006	Temporal workflow/activity source separation and versioning
ADR-007	No unvalidated response streaming
ADR-008	Action Safety owner boundary pending its storage proof

13. HLD amendment preservation and acceptance

Amendment	Part 1 enforcement
A1 Release vector	Release manifest separates code identity from agent behavior; runtime records actual per-turn dependency vector; only compatible agent/workflow code is pinned.
A2 Freshness	Knowledge is resolved per turn with revocation; policy is an owner module re-evaluated before effect; material change invalidates confirmation.
A3 Validated delivery	Public contract has deterministic progress and validated final events only; Agent validator precedes Conversation commit and SSE.
A4 Action Capability	Capability types and redemption implementation are absent from Agent Runtime; Workflow/Integration boundary is opaque and single-use.
A5 Action Safety	Integration Gateway owns one action-safety port/service boundary; ordinary DynamoDB repositories cannot implement it; regional recovery waits for later consistency proof.
A6 Revocation cache	Retrieval/cache API requires tenant, release, ACL, locale, query and revocation epoch; protected fallback fails closed when freshness is unknown.
A7 MCP exposure	Control owns registration/import; Integration owns approved runtime contracts; Agent receives only a generated per-turn read/tool projection.
A8 Ownership	Workflow core, Activities, Policy and Agent are separate projects with enforced dependency rules.
A9 Human authority	Human Operations owns takeover; Conversation control mode suppresses automated replies; AI handoff module emits draft summary only.
A10 Recovery gate	TenantContext reserves routing epoch; deployment sources reserve fencing/action-safety modules; no chart can label a recovery cell writable without readiness proof.

13.1 Seven-journey coverage

Journey	Required release units/modules	Preserved invariant
Order tracking	Conversation, Agent, Integration read, validated SSE	No guessed status; authorized fields only
Eligibility	Workflow Policy module, Agent Knowledge explanation	Policy output is authoritative; RAG only explains
Create return	Workflow, preview/confirmation, Integration action boundary	No write before exact confirmation/capability
Damaged replacement	Attachment/evidence, Human approval, inventory connector, address subworkflow	No autonomous vision judgment; unknown result reconciled
Refund	Policy thresholds, Human approval/takeover, action-safety/reconciliation	No false financial completion or duplicate effect
Cancellation	Workflow consequence preview, subscription connector	Changed effect requires new confirmation
Human escalation	Workflow signal, Human case/queue, Conversation control mode	Accepted ownership stops automation; release-back explicit

13.2 Part 1 acceptance checklist

Result	Criterion
PASS	Each HLD deployable maps to one source owner, release manifest, image family and Helm chart.
PASS	API, worker, consumer and static-surface roles can scale separately without inventing new independent services.
PASS	Cross-deployable source/database imports are forbidden and machine-checked.
PASS	Agent Runtime cannot obtain consequential connector, credential, Action Capability or redemption code.
PASS	Temporal workflow and Activity code are physically and statically separated.
PASS	Contracts have canonical source, version, owner, generated clients and compatibility gates.
PASS	Configuration and secret layers match release pinning, revocation and runtime-cell independence.
PASS	Local and CI environments exercise real protocols plus deterministic failure simulators.
PASS	Build/release artifacts are content-addressed, signed and traceable.
PASS	All ten amendments and seven journeys map to enforceable codebase boundaries.

13.3 Part 1 freeze decision

PART 1 FROZEN The codebase and deployable structure is complete enough to begin LLD Part 2: Identity and TenantContext. No open Part 1 decision requires changing the HLD. Later LLDs may fill schemas, algorithms and technologies behind the ports defined here, but may not weaken ownership, dependency, release, streaming, tenant, workflow or action-safety boundaries.

13.4 Next LLD

LLD Part 2 will define authentication claims, principal types, verification levels, trusted TenantContext, workload identity, home-cell and routing-epoch propagation, repository enforcement, workforce roles, service-to-service authorization, and the negative isolation tests that every later LLD must inherit.

References

[I1] Customer Agent OS Lite - V1 Product Contract, July 2026.

[I2] Customer Agent OS Lite - Combined High-Level Design v1.0, July 2026.

[I3] Customer Agent OS Lite - AI HLD Parts 1 and 2, regenerated July 21, 2026.

[I4] Customer Agent OS Lite - Integrated HLD Validation and Architecture Freeze v1.1, July 21, 2026.

[N1] Node.js, Previous Releases. https://nodejs.org/en/about/previous-releases

[P1] Python Developer's Guide, Status of Python versions. https://devguide.python.org/versions/

[T1] Temporal, TypeScript Workflow basics. https://docs.temporal.io/develop/typescript/workflows/basics

[T2] Temporal, TypeScript Workflow Versioning. https://docs.temporal.io/develop/typescript/workflows/versioning

[B1] Buf, Detecting breaking changes. https://buf.build/docs/breaking/

AUTHORITY If this Part 1 LLD conflicts with the V1 Product Contract or Integrated HLD Freeze, the higher-precedence frozen document controls. Such a conflict requires correction, not silent reinterpretation.

KLEEM AI / LOW-LEVEL DESIGN

Customer Agent OS Lite

LLD Part 2: Identity and TenantContext

Exact human, customer, workload, tenant, environment, authorization, propagation, isolation, revocation, and audit contracts inherited by every later LLD.

STATUS	PART 2 FROZEN - READY FOR LLD PART 3
Core decision	External credentials stop at Edge; short-lived signed TenantContext plus SPIFFE workload identity; OPA policy enforcement; business Policy Service remains separate
Security invariant	No request, query, event, workflow, retrieval, model context, tool call, human view, or trace may operate without trusted tenant and environment scope
Version	1.0 - July 23, 2026

Design rule: identity proves who is acting; TenantContext proves where and for whom the request is scoped; authorization proves whether this actor may perform this operation on this resource now. None of these proves business eligibility, customer confirmation, or action completion.

1. Part 2 decision

FINAL PART 2 DECISION	Adopt a zero-ambient-authority model. Edge verifies external identity and tenant routing, resolves current membership, and asks a cell-local Context Authority to issue a two-minute signed Internal Context Assertion. Every receiving service verifies that assertion, authenticates the calling workload with SPIFFE mTLS, calls a local OPA Policy Decision Point, enforces returned obligations, and independently scopes its owned resources. External tokens, internal context assertions, and role claims never enter prompts, provider calls, Kafka payload bodies, Temporal histories, or durable database records.

1.1 Frozen outcomes

- Five principal families are explicit: end customer, tenant workforce, platform workforce, workload, and system continuation. Actor, customer subject, and executing workload are never collapsed into one identity.
- Tenant identity comes from trusted host/channel configuration plus verified tenant membership. No body, query, arbitrary header, model output, tool argument, or resource ID can select or override tenant scope.
- OIDC Authorization Code with PKCE is the hosted interactive baseline. Tenant commerce sessions and enterprise federation enter through registered identity adapters. OAuth security follows current best practice [S1][S2][S3].
- The Internal Context Assertion is a distinct JWS profile with an explicit type, fixed algorithm allowlist, separate signing keys, short expiry, audience, routing epoch, and context ID. It is not an OAuth access token and never grants an action by itself.
- SPIFFE X.509 SVIDs provide short-lived service identity for mTLS. Each Kubernetes workload role has a distinct service account and IRSA cloud role [S6][S7].
- OPA 1.x runs cell-locally as the authorization PDP using signed, versioned policy bundles. Resource services remain policy enforcement points. Business eligibility and thresholds remain in the deterministic Policy Service [S8].
- Authorization combines RBAC with ABAC. A role makes permissions understandable; tenant, environment, ownership, queue, resource, purpose, verification, risk, amount class, and routing epoch narrow the decision.
- Short-lived request assertions are never used as durable workflow authority. Temporal and event consumers reconstruct execution context using trusted adapters, current workload identity, durable actor snapshots, and fresh authorization.
- Defense in depth requires tenant scope in repositories, keys, events, caches, objects, retrieval filters, model requests, workflow references, human views, and telemetry. PostgreSQL RLS is a backstop, not the only isolation layer [S9].
- All protected failures are fail-closed. No stale membership, missing PDP bundle, invalid context, unknown routing epoch, or unverifiable workload may silently degrade into broader access.

1.2 Scope and deferments

Class	Boundary
Included	External authentication profiles, principal types, customer verification levels, role catalog, TenantContext schema, issuance and exchange, workload identity, authorization API, OPA decision contract, service call graph, tenant enforcement interfaces, revocation, failure behavior, audit fields, isolation tests.
Deferred to Part 3	Exact identity, membership, decision-audit, session, key, and tenant-routing tables; indexes, constraints, RLS SQL, DynamoDB item shapes, migrations, and retention implementation.
Deferred later	Conversation APIs (Part 4), Kafka topics/envelopes (Part 5), Temporal workflow states (Part 6), Action Capability and connector execution (Part 7), agent internals (Part 8), retrieval/model details (Part 9), capacity/runbooks/DR exercises (Part 10).
Not reopened	Seven release units, durable Temporal ownership, Action Safety single-writer semantics, no consequential write tool in Agent Runtime, validated-only delivery, regional fencing, seven journeys, or HLD amendments A1-A10.

1.3 Language continuity

RUNTIME-NEUTRAL SECURITY

Part 1 currently freezes TypeScript for online services. Part 2 therefore defines the TypeScript verifier and middleware as the reference implementation, while Protobuf and JSON Schema generate equivalent Python bindings. If Agent Runtime is later moved to Python/FastAPI through an approved Part 1 amendment, it must use the same Context Authority, SPIFFE identity, OPA decision contract, isolation tests, and authority limits. Go is not introduced in V1.

1.4 Document map

Section	Purpose
2	Identity architecture and trust boundaries
3	Principal, actor, subject, and role model
4	External authentication and tenant resolution
5	Customer verification and step-up
6	Trusted TenantContext contract
7	Issuance, exchange, propagation, and durable continuation
8	Workload identity and service authorization
9	Authorization engine and policy contract
10	Workforce delegation, approval, and break-glass
11	Tenant isolation enforcement
12	Workflow, event, AI, retrieval, and tool binding
13	Revocation, failures, and regional recovery
14	Audit, privacy, and observability
15	Implementation modules and delivery sequence
16	Tests, acceptance, and freeze decision

2. Identity architecture and trust boundaries

Kleem separates four facts that are often incorrectly merged: external authentication, internal request provenance, service identity, and authorization. The system never treats network location, a customer-supplied tenant ID, or an LLM-generated resource reference as proof. This follows zero-trust guidance that access is evaluated per resource using subject, service, and contextual attributes rather than assumed from network position [S5].

Component	Trust granted	Trust explicitly not granted
External IdP or tenant channel	Authenticates a customer or workforce user and issues an external assertion	Does not assign Kleem permissions or select an arbitrary tenant
Edge authentication adapter	Validates issuer, audience, signature, time, nonce/PKCE, and registered channel	Does not trust role strings, resource IDs, or tenant headers from the browser
Tenant Resolver	Matches trusted host/channel, external organization, and current membership to one tenant/environment	Fails on ambiguity or mismatch; never chooses the most convenient tenant
Context Authority	Issues and exchanges short-lived audience-bound Internal Context Assertions	Cannot grant permissions; cannot broaden parent scope; is not a business Policy Service
SPIRE/workload PKI	Attests workload role and issues short-lived X.509 SVID for mTLS	Does not grant tenant access merely because a service is inside the cluster
Authorization PDP	Evaluates RBAC plus ABAC using signed OPA policy bundle and current identity data	Does not determine refund eligibility, policy thresholds, confirmation, or completion
Resource owner/PEP	Verifies context and caller, asks PDP, applies obligations, scopes data, and audits	Cannot delegate enforcement to a downstream model or client

2.1 Runtime flow

1	A request reaches Edge on a registered tenant host/channel. The browser may carry an OIDC access token, a tenant-signed commerce session exchange token, or an anonymous public-session cookie.
2	The authentication adapter validates the credential profile. Edge resolves tenant and environment from trusted routing configuration and proves that token organization/membership matches that route.
3	The cell-local Identity Read Model returns the internal principal, current membership epoch, role bindings, queues, status, verification state, and access-revocation epoch.
4	Context Authority signs an Internal Context Assertion for the immediate target audience. The assertion includes actor, customer subject, tenant scope, purpose, routing epoch, and provenance but no broad permission list or secrets.
5	The target verifies signature, token type, audience, expiry, route, and caller SPIFFE identity. It builds an authorization input with the intended action and an owner-generated resource descriptor.
6	Local OPA returns DENY, ALLOW, or STEP_UP_REQUIRED plus policy version, reason codes, and enforceable obligations. The resource owner applies obligations and scopes every datastore operation.
7	A downstream call uses Context Exchange to mint a narrower assertion for the next audience. The service cannot edit tenant, actor, subject, purpose, or verification claims.
8	The response carries only business data and stable errors. Tokens, raw claims, role bindings, PDP inputs, and internal resource topology remain server-side.

2.2 Trust boundary invariants

- External JWT validation is issuer-specific. Algorithm, key set, audience, token type, required claims, and maximum clock skew are configured per registered identity adapter; algorithm selection is never taken from token input [S2].

- ID tokens authenticate a login session; API access tokens authorize calling the Edge resource server. An ID token is never accepted as a general API bearer token.
- External identities use an issuer-plus-subject key. Email, display name, organization slug, or mutable username is never the stable identity key.
- Context Authority owns signing keys. Ordinary services only verify assertions and request constrained exchanges; no application workload can mint arbitrary TenantContext values.
- mTLS authenticates the current workload; TenantContext preserves the human/customer actor and tenant scope. Both must be valid.
- Authorization is checked at Edge for admission and again at the resource owner. Edge permission cannot replace owner-side enforcement.
- All AI and retrieved content is untrusted data. It can name a requested object but cannot create identity, tenant membership, verification, role, or authorization facts.

3. Principal, actor, subject, and role model

Every operation records three separate identities: **actor** requested or originally caused the work, **subject** is the customer/account affected, and **executor** is the workload currently running code. For customer self-service, actor and subject match. For a representative, the actor is workforce and the subject is the customer. For a resumed workflow, the executor changes while the original actor snapshot remains immutable.

Principal kind	Authentication	Maximum authorization scope
END_CUSTOMER	Tenant commerce session, hosted OIDC, approved OTP or anonymous public session	Own conversation and verified customer/account/order resources only
TENANT_WORKFORCE	OIDC/SAML enterprise SSO through identity broker; MFA; active tenant membership	Role, queue, environment, assigned resource, purpose, and authority attributes
PLATFORM_WORKFORCE	Corporate SSO; phishing-resistant MFA; managed device and JIT elevation for protected access	Infrastructure operations; no ambient tenant content; break-glass only when explicitly granted
WORKLOAD	SPIFFE X.509 SVID plus mTLS; Kubernetes service account; IRSA for AWS APIs	Declared service operations only; tenant access additionally requires trusted TenantContext
SYSTEM_CONTINUATION	Trusted Temporal/event adapter reconstructs execution from durable scope and current workload identity	Only the scheduled workflow/event operation; no general user session or new sensitive decision

3.1 Actor/subject model

```
type PrincipalKind =
  | "END_CUSTOMER" | "TENANT_WORKFORCE"
  | "PLATFORM_WORKFORCE" | "WORKLOAD" | "SYSTEM_CONTINUATION";

type ActorRef = Readonly<{
  kind: PrincipalKind;
  principalId: PrincipalId;       // opaque internal ID
  membershipId?: MembershipId;   // tenant workforce only
  sessionId?: SessionId;         // interactive only; never logged raw
}>;

type CustomerSubjectRef = Readonly<{
  customerId: CustomerId;
  accountId?: AccountId;
}>;

type DelegationRef = Readonly<{
  mode: "SELF" | "CASE_BOUND" | "SYSTEM_CONTINUATION";
  caseId?: CaseId;
  delegationId?: DelegationId;
  reasonCode?: string;
}>;
```

The stable internal principal ID is created after issuer/subject validation. Raw external subject, email, phone, claims, OTP data, and provider access token are excluded from TenantContext. A one-way provider-subject lookup key belongs to the identity store defined in Part 3.

3.2 Frozen V1 workforce roles

Role	Representative permissions	Explicit boundary
Support Agent	Read assigned cases; join assigned conversations; use approved knowledge; perform explicitly authorized low-risk human operations	No self-assignment outside queue; no approval above authority; no tenant administration
Approver	Review and approve/reject configured action classes and amount bands for assigned queues	Cannot submit the provider action solely because approval was granted
Queue Manager	Manage queue membership, priority, assignment, operating state, and authorized participants	Cannot read unrelated customer content solely through queue administration
Knowledge Manager	Manage sources, conflicts, effective dates, ingestion, retrieval previews, and publication proposals	Cannot publish agent/runtime releases unless also granted Release Manager
Release Manager	Run evaluations; approve staged agent/knowledge publication; canary, rollback, and revoke allowed releases	Cannot change identity policy or obtain raw customer content by default
Tenant Admin	Manage users, roles, environments, identity connections, integrations, retention, quotas, and approval authorities	Cannot disable audit/isolation, assign forbidden platform roles, or silently elevate self
Platform Operator	Operate infrastructure and recover cells through audited JIT scope	No default tenant-data access; no customer business action; break-glass constraints apply

3.3 Role binding rules

- Roles are tenant- and environment-scoped. A production binding never implies staging access and a staging binding never implies production access.
- Role names do not travel from an external IdP as authoritative grants. External groups may map to proposed role bindings through approved tenant configuration, but the Kleem Identity Directory remains the source of effective membership.
- Queue membership, approval authority, resource ownership, verification, purpose, and risk are attributes evaluated in addition to role.

- A user may hold several roles. Effective permission is the union of allowed grants subject to explicit deny rules and obligations; a DENY cannot be canceled by another role.
- Self-elevation is prohibited. A change that would increase the actor's own authority requires another eligible administrator or configured approval.
- Disabled, suspended, expired, or deleted memberships are denied even if an external token remains cryptographically valid.

4. External authentication and tenant resolution

Kleem supports several entry profiles because embedded commerce support, a hosted support portal, and enterprise workforce access have different identity sources. They converge only after provider-specific validation. OAuth 2.0 implicit and resource-owner-password flows are prohibited; hosted clients use Authorization Code with PKCE following OAuth security best practice [S1].

Entry	V1 profile	Required binding
Hosted customer portal	OIDC Authorization Code + PKCE; secure HttpOnly same-site session at Edge/BFF	Registered issuer/client/audience; nonce/state; callback allowlist; short access token
Embedded commerce widget	Tenant backend issues short-lived signed session-exchange JWT to registered audience	Issuer/JWKS, tenant channel, customer subject, nonce, max 5-minute lifetime; no browser-held integration secret
Guest public support	Anonymous random session cookie created by Edge	General public knowledge only; no account/order data; tenant from trusted host
Tenant workforce	Enterprise OIDC or SAML federation through the configured identity broker; MFA required	Organization connection plus active Kleem membership; workforce token cannot act as a customer
Platform workforce	Corporate OIDC; phishing-resistant MFA; managed-device/risk checks	No tenant scope until JIT/break-glass grant is resolved
Webhook/integration	Signed webhook, mTLS, OAuth client, or API key at Integration Gateway	Integration identity is not an interactive principal; exact connector policy belongs to Part 7

4.1 External JWT validation profile

Area	Validation	Reason
Header	Reject unknown typ/crit; allow only configured asymmetric algorithms; require kid when several active keys exist	Prevents algorithm confusion and cross-JWT substitution [S2]
Signature/key	Resolve only from registered issuer metadata/JWKS; cache with bounded stale window; never follow token-provided URL	Prevents SSRF and attacker-selected key
Issuer/audience	Exact issuer and expected resource audience; authorized-party/client check when required	Prevents accepting a valid token intended for another API
Time	Require exp and iat; validate nbf; maximum 60-second skew; provider-specific maximum token age	Bounds replay and stale identity
Session	Validate nonce/state for login; PKCE verifier; rotate Edge session on login/step-up	Prevents code/session substitution
Subject	Use exact issuer plus sub mapping; no email/username identity key	Stable identity despite profile changes
Tenant	Treat org/tenant claim as a routing hint only; prove it against host/channel and current Kleem membership	Prevents claim-based tenant switching

4.2 Tenant resolution algorithm

1	Resolve **RouteCandidate** from the TLS host, public client/channel ID, deployment environment, and current routing snapshot. An arbitrary X-Tenant-ID is ignored and logged as an attack signal.
2	Validate the external credential using the adapter registered to that route. A token from any other issuer/audience/client is rejected before membership lookup.
3	Map exact issuer plus subject to an internal principal. For an embedded customer token, also map the registered external customer subject to one internal customer under the route tenant.
4	Load active memberships for the route tenant and environment. There must be exactly one eligible result. Suspended, expired, deleted, or wrong-environment membership returns a generic denial.
5	Compare token organization/channel binding, route tenant, resolved membership tenant, and customer/account ownership. Any mismatch is a security event; no fallback tenant search occurs.
6	Load tenant home region/cell and routing epoch from the signed regional routing snapshot. A stale or non-home route can serve only the specifically allowed redirect/read behavior.
7	Return a TrustedIdentitySeed to Context Authority. The browser never receives internal tenant, membership, route, role, or workload claims.

4.3 Hosted session rules

- The browser receives an opaque Secure, HttpOnly, SameSite cookie. Provider tokens remain encrypted at Edge; localStorage and JavaScript-readable refresh tokens are prohibited.
- CSRF applies to state changes. CORS and Origin are allowlisted per tenant channel but never replace authentication.
- Default idle timeout is 30 minutes for workforce and 60 for customers; workforce absolute session is 8 hours. Rotate after login, step-up, role change, and takeover.
- Logout invalidates the Edge session and its revocation marker. Provider logout is best effort.
- Provider introspection is adapter-specific; cryptographic validity never overrides a current Kleem membership suspension.

5. Customer verification and step-up

Kleem uses product-specific verification levels rather than claiming formal NIST assurance certification. The model borrows the separation between identity proofing, authentication, and federation described by NIST SP 800-63-4, but each tenant must assess its own assurance requirements [S4].

Level	Evidence	Permitted use
V0 ANONYMOUS	No customer identity; Edge-issued public session only	Public general policy/knowledge; request human; no account/order lookup
V1 IDENTIFIED	A candidate customer/order reference exists but ownership is not proven	Collect minimum verification evidence; no protected data disclosure
V2 AUTHENTICATED	Active authenticated commerce/hosted account or approved OTP/order verification	Own account/order reads and ordinary journey setup, subject to policy
V3 STEP_UP	Recent stronger proof bound to subject and purpose; default freshness 10 minutes	Sensitive address/account change, refund/replacement/cancellation confirmation when policy requires
V4 MANUAL_VERIFIED	Authorized human completed configured exceptional verification and recorded evidence/reason	Only the exact case/resource/purpose approved; not a reusable stronger session

5.1 Verification object

```
type VerificationContext = Readonly<{
  level: "V0" | "V1" | "V2" | "V3" | "V4";
  subjectCustomerId?: CustomerId;
  methods: readonly VerificationMethodCode[];
  verifiedAt?: Timestamp;
  validUntil?: Timestamp;
  purpose: PurposeCode;
  resourceBindings: readonly ResourceUrn[]; // order/account/case if narrow
  evidenceRefs: readonly OpaqueEvidenceRef[]; // no OTP or secret
  verificationVersion: string;
}>;
```

5.2 Step-up rules

Operation	Minimum identity	Additional invariant
Protected account/order read	V2; ownership/resource match	Generic 401/403 challenge without revealing whether resource exists
Create return	V2; V3 if tenant policy/risk requires	Exact confirmation still required; verification does not authorize action
Replacement/refund/cancel	V2 plus current journey policy; V3 for sensitive/high-risk conditions	Approval, confirmation, policy, and Action Capability remain separate
Address change	V3 bound to customer/address-change purpose; re-verify when replacement address differs	Material preview change invalidates confirmation
Workforce admin change	MFA session plus recent re-authentication; explicit confirmation	Self-elevation and forbidden cross-environment grants denied
Manual verification	Eligible case-bound representative plus required evidence/reason	V4 expires with case/resource and cannot be reused elsewhere

When the PDP returns STEP_UP_REQUIRED, Edge emits a stable challenge code, required verification class, maximum age, purpose, and continuation handle. The client completes the provider flow and resumes with that handle. The server re-runs authorization and business policy; a stronger login never resumes an already stale preview automatically. RFC 9470 supplies the standard pattern for communicating insufficient authentication context, although Kleem's public API details are frozen in Part 4 [S11].

5.3 Privacy and anti-enumeration

- Verification prompts request the minimum data necessary. The model never receives OTP values, recovery answers, full government identifiers, provider tokens, or raw authentication factors.
- Unknown order, wrong tenant, wrong customer, and unauthorized resource return the same external not-found/verification response where disclosure would enable enumeration.
- Attempt counters are tenant, channel, session, subject hint, and network-risk scoped. Rate limits do not confirm whether an account exists.
- Verification evidence references are opaque and access-controlled. Raw evidence follows the product's classification, retention, and deletion policy.
- Customer identity mismatch triggers bounded clarification or handoff. The system never searches other tenants for a matching email/order.

6. Trusted TenantContext contract

MANDATORY CONTEXT

TrustedTenantContextV1 is a branded, immutable server-side type that application code cannot construct directly. Only verified inbound adapters or Context Authority clients return it. Every protected command, query, repository method, retrieval request, workflow activity, tool call, and human view requires it or a narrower authorized derivative.

```
type TrustedTenantContextV1 = Readonly<{
  contextVersion: "1";
  contextId: ContextId;                 // unique, non-secret
  tenant: {
    tenantId: TenantId;
    environmentId: EnvironmentId;
    environmentClass: "DEV" | "STAGING" | "PRODUCTION";
  };
  actor: ActorRef;
  subject?: CustomerSubjectRef;
  delegation: DelegationRef;
  verification: VerificationContext;
  membership?: {
    membershipId: MembershipId;
    roleIds: readonly RoleId[];
    queueIds: readonly QueueId[];
    membershipEpoch: bigint;
    tenantAccessEpoch: bigint;
  };
  purpose: PurposeCode;
  route: {
    homeRegion: RegionId;
    homeCell: CellId;
    routingEpoch: bigint;
  };
  provenance: {
    issuer: ContextIssuerId;
    audience: ServiceAudience;
    authenticationEventId: AuthenticationEventId;
    issuedAt: Timestamp;
    expiresAt: Timestamp;               // at most 120 seconds
    parentContextId?: ContextId;
  };
  request: {
    requestId: RequestId;
    traceId: TraceId;
    channelId: ChannelId;
  };
}>;
```

6.1 Field semantics

Field	Meaning	Invariant
tenantId/environmentId	Authoritative isolation scope from route plus verified membership	Required everywhere; environment is not inferred from deployment alone
actor	Original interactive actor or trusted system continuation	Never replaced by executing workload; preserves accountability
subject	Customer/account affected by the operation	Required for customer-specific work; separate from workforce actor
delegation	Self, case-bound workforce action, or system continuation	CASE_BOUND requires authorized case and reason; cannot be set by browser/model
verification	Current customer verification evidence and narrow resource/purpose bindings	Not equivalent to authorization, approval, or confirmation
membership	Current workforce binding snapshot and revocation epochs	Absent for pure customer/workload contexts; PDP may require fresh read
purpose	Customer support, tenant admin, knowledge admin, evaluation, or incident break-glass	Prevents a context collected for one purpose from being reused broadly
route	Home region/cell and writer fencing epoch	Every protected write compares current authoritative routing epoch
provenance	Signer, intended audience, auth event, issue/expiry, parent exchange	Supports cryptographic verification and delegation-chain audit
request	Correlation only	Trace IDs are not security decisions and contain no customer PII

6.2 Internal Context Assertion profile

Property	Decision	Reason
Format	Compact JWS JWT; typ=kleem-tenant-context+jwt; context_version=1	Distinct type and validation path prevents cross-JWT confusion [S2]
Algorithm	ES256 baseline; exact allowlist; separate signing key family from OAuth, Action Capability, and artifact signatures	A valid token from another Kleem subsystem is still rejected
Issuer	Cell-local Context Authority URI; issuer/region/cell must match trusted configuration	No issuer discovery from token input
Audience	Exactly one release-unit/service audience	No wildcard or all-services audience
Lifetime	Maximum 120 seconds; no refresh token	Long work reconstructs context; bearer replay window stays narrow
Claims	IDs, enums, epochs, hashes, and timestamps only	No email, phone, transcript, OTP, provider token, secret, or full permission set
Binding	Used only over authenticated mTLS; receiver compares current SPIFFE caller against operation call graph	Bearer possession alone is insufficient

6.3 Construction and type safety

```
declare const TRUSTED_CONTEXT: unique symbol;
type Trusted<T> = T & { readonly [TRUSTED_CONTEXT]: true };

interface ContextVerifier {
  verifyInbound(
    assertion: string,
    expectedAudience: ServiceAudience,
    caller: VerifiedWorkloadIdentity
  ): Promise<Trusted<TrustedTenantContextV1>>;
}

type Permission = `${string}:${string}`;
type AuthorizedContext<P extends Permission> = Readonly<{
  tenantContext: Trusted<TrustedTenantContextV1>;
  decision: AllowDecision<P>;
}>;

interface OrderRepository {
  getById(
    ctx: AuthorizedContext<"order:read">,
    orderId: OrderId
  ): Promise<Order | NotFound>;
}
```

Compile-time branding prevents casual construction but is not a security boundary. Runtime signature verification, caller authentication, PDP evaluation, resource tenant checks, and datastore isolation remain mandatory. Test helpers create signed development contexts through the identity stub, never object casts.

7. Issuance, exchange, propagation, and durable continuation

A single broad context token is not forwarded through the entire graph. Context Authority implements constrained exchange inspired by OAuth Token Exchange [S10]. Each exchange produces a new assertion for one audience, links to the parent context ID, preserves immutable identity/scope, and can only reduce purpose, resource bindings, or verification validity.

```
service ContextAuthorityV1 {
  rpc IssueFromIdentitySeed(IssueRequest) returns (ContextAssertion);
  rpc ExchangeForAudience(ExchangeRequest) returns (ContextAssertion);
  rpc ReconstructForContinuation(ContinuationRequest)
      returns (ContextAssertion);
}

message ExchangeRequest {
  string parent_assertion = 1;
  string requested_audience = 2;
  string operation = 3;
  string resource_handle = 4;  // optional and opaque
  string request_id = 5;
}
```

7.1 Issue and exchange rules

Operation	Caller	Checks	Result
IssueFromIdentitySeed	Edge authentication adapter only	Verified route, identity, membership, verification, purpose, and current routing snapshot	Assertion for immediate service audience
ExchangeForAudience	Allowed runtime workload over SPIFFE mTLS	Valid parent assertion; caller permitted by call graph; target audience reachable; claims unchanged or narrowed	Shorter/equal expiry and linked parent ID
ReconstructForContinuation	Approved event/Temporal adapter	Durable tenant scope, actor snapshot, workflow/event provenance, current workload, current epochs, intended operation	SYSTEM_CONTINUATION assertion; no user session resurrection

7.2 Headers and transport

- Internal gRPC metadata uses **authorization: KleemContext <assertion>**. Public HTTP never accepts that scheme. External Bearer tokens are accepted only by Edge public authentication adapters.
- Trace, request, deadline, and idempotency metadata are separate from TenantContext. A client cannot override context fields by sending duplicate business headers.
- Transport adapters remove all incoming internal context headers at the public edge, verify external identity, and inject a newly issued context.
- Services do not serialize the assertion into domain commands, logs, Kafka, Temporal, object metadata, or model inputs. They pass the branded in-memory context to application code.
- Outbound provider and model clients use explicit allowlisted request DTOs. Context headers are stripped by default; gateway interceptors reject attempts to forward them outside the trust domain.

7.3 Durable ActorSnapshot

```
type DurableActorSnapshotV1 = Readonly<{
  snapshotVersion: "1";
  tenantId: TenantId;
  environmentId: EnvironmentId;
  actor: Omit<ActorRef, "sessionId">;
  subject?: CustomerSubjectRef;
  delegation: DelegationRef;
  verificationLevelAtEvent: VerificationContext["level"];
  verificationEventId?: VerificationEventId;
  originalPurpose: PurposeCode;
  authenticationEventId: AuthenticationEventId;
  capturedAt: Timestamp;
}>;
```

The snapshot is evidence of who caused a durable operation; it is not current authorization. On resume, the executing workload authenticates again, Context Authority verifies workflow/event provenance, the resource owner evaluates current policy and routing, and sensitive work may require a new customer confirmation or step-up. External tokens and expiring Internal Context Assertions are never stored in Temporal history.

7.4 Continuation behavior

Continuation	Context source	Critical rule
Kafka event	Trusted consumer validates topic/schema/producer and reconstructs from envelope scope plus owner data	Do not trust tenant solely because bytes contain tenant_id
Temporal Activity	Workflow carries TenantScope and ActorSnapshot; worker reconstructs assertion for the Activity audience	Workflow code remains deterministic and contains no token exchange/network call
Scheduled job	Control-plane assignment enumerates explicit tenant/environment partitions; job workload reconstructs SYSTEM_CONTINUATION	No platform-wide unscoped scan
Human takeover	New CASE_BOUND context records workforce actor while preserving customer subject and prior actor snapshot	Representative is never impersonated as customer

8. Workload identity and service authorization

TWO KEYS TO EVERY INTERNAL CALL

The receiving service requires both a valid workload certificate and a valid TenantContext assertion. Workload identity answers 'which code is calling?'; TenantContext answers 'for which tenant, actor, subject, purpose, and route?'. Neither substitutes for the other.

8.1 SPIFFE and cloud identity

Layer	Decision	Use
mTLS identity	SPIRE-issued X.509 SVID; trust domain per environment; short lifetime and automatic rotation	Service-to-service authentication and encryption [S6]
SPIFFE ID	spiffe://kleem./region//cell//ns//sa/	Exact workload role, region, cell, namespace, and service account
Kubernetes	One service account per workload role, not per release unit	API, worker, consumer, reconciliation, and admin roles do not share ambient rights
AWS APIs	IRSA role bound to service account with least privilege	S3/KMS/Secrets/MSK/etc. access independent from mTLS [S7]
External provider	Gateway-owned OAuth/API/mTLS credential from secret reference	Never forward SPIFFE certificate, internal context assertion, or customer token

8.2 Call-graph authorization

Caller	Allowed internal targets	Forbidden authority
Edge/API	Conversation, Human API, Control API admin endpoints, Context Authority	Cannot call model provider, datastore owner repository, or Integration action endpoint
Conversation Runtime	Agent Runtime, Workflow start/signal, Human API, Context Authority	Cannot call consequential connector or policy datastore directly
Agent Runtime	Conversation read projection, Model/Retrieval gateways, approved Integration read projection, Context Authority	No Action Capability, action execution, credential broker, tenant-admin writes
Workflow Workers	Policy module, Integration Gateway, Human API, Conversation response/progress, Context Authority	No model provider/credential access from deterministic Workflow code
Integration Gateway	Approved external providers, Action Safety Store, Context Authority	No prompt, transcript assembly, broad customer search, or tenant admin
Human Operations	Conversation projection, Workflow signals/queries, Context Authority	No direct workflow datastore edit or connector credential
Control/Knowledge	Publication channels, identity administration stores, approved knowledge sources	No synchronous normal-turn resource mutation or customer action

8.3 Receiver checks

1	Terminate mTLS and verify SPIFFE trust domain, certificate chain, validity, and exact caller ID. Network source IP is not identity.
2	Verify Internal Context Assertion type, signature, algorithm, issuer, audience, issued/expiry time, context version, and route.
3	Check the static call graph: this caller workload may invoke this RPC/method. A broad service certificate cannot reach an undeclared method.
4	Compare caller region/cell with context route. For protected writes, verify current tenant home cell and routing epoch through the owner safety adapter.
5	Build authorization input from trusted context, caller workload, intended action, owner-generated resource attributes, and current identity epochs.
6	Evaluate OPA; require ALLOW; enforce all obligations; then call a repository whose interface also requires AuthorizedContext.
7	Audit the decision and outcome using IDs/hashes only. Never log certificate, token, full PDP input, or customer content by default.

8.4 Workload failure behavior

- Missing, expired, untrusted, or wrong-environment SVID returns UNAUTHENTICATED and emits a high-severity workload-identity signal.
- SPIRE outage does not invalidate still-valid certificates. Readiness turns false before the certificate renewal safety margin is exhausted; new protected work stops if identity cannot be renewed.
- IRSA failure blocks the specific AWS-backed operation. It never falls back to node credentials, shared static keys, or another workload's role.
- Certificate rotation is automatic and does not require pod restart. Trust-bundle change is canaried and supports controlled overlap; unknown roots fail closed.
- A compromised workload identity cannot broaden tenant context because Context Exchange validates parent context and call graph. Resource owners still enforce tenant/resource scope.

9. Authorization engine and policy contract

Kleem selects OPA 1.x and Rego for platform authorization. Each protected workload has a local PDP sidecar or same-pod authorization process with a signed immutable policy bundle. Control and Knowledge publishes reviewed bundles asynchronously; runtime cells evaluate locally and have no synchronous dependency on the control plane. OPA supports context-aware API authorization and signed bundle distribution [S8].

9.1 Authorization versus business policy

Authority	Question answered	Output
Platform Authorization (OPA)	May this actor/workload perform action X on resource Y in tenant/environment Z for this purpose now?	ALLOW/DENY/STEP_UP_REQUIRED ; obligations; authz policy version
Business Policy Service	Is this return/refund/replacement/cancellation eligible and what thresholds/evidence/approval apply?	Eligible/ineligible/needs facts/approval; policy version and obligations
Workflow state	Is this transition valid now and are preview/confirmation/approval current?	Accepted/rejected transition and new durable state
Action Capability	May Integration Gateway execute exactly this confirmed effect once?	Opaque single-use authority defined in Part 7

COMPOSITION RULE

A consequential operation proceeds only when platform authorization allows, business policy allows, workflow state allows, required verification/confirmation/approval are current, routing epoch is current, and Integration Gateway redeems the exact Action Capability. One allow can never override another authority's deny.

9.2 Permission and resource naming

```
Permissions:
  conversation:create        conversation:read        conversation:join
  order:read                 customer:read            case:read
  case:assign                case:takeover            case:release
  approval:decide            queue:manage             knowledge:publish
  release:promote            identity:role-bind       tenant:configure
  tool:read:<operation>      action:request:<class>

Resource URNs:
  kleem://tenant/<tenant>/env/<env>/conversation/<id>
  kleem://tenant/<tenant>/env/<env>/customer/<id>
  kleem://tenant/<tenant>/env/<env>/order/<id>
  kleem://tenant/<tenant>/env/<env>/case/<id>
  kleem://tenant/<tenant>/env/<env>/knowledge-release/<id>
```

Resource URNs are server-generated typed descriptors, not authorization by obscurity. The owner loads the resource under tenant scope and supplies authoritative attributes to OPA. A browser or model-provided ID is only a lookup candidate and cannot change the tenant encoded by the owner.

9.3 Decision API

```
type AuthorizationInputV1 = Readonly<{
  inputVersion: "1";
  tenant: TenantScope;
  actor: ActorRef;
  subject?: CustomerSubjectRef;
  executor: VerifiedWorkloadIdentity;
  delegation: DelegationRef;
  membership?: MembershipSnapshot;
  verification: VerificationContext;
  purpose: PurposeCode;
  action: Permission;
  resource: AuthorizedResourceDescriptor;
  environment: EnvironmentAttributes;
  route: RoutingContext;
  request: { time: Timestamp; channelId: ChannelId; riskClass: RiskClass };
}>;

type AuthorizationDecisionV1 =
  | { effect: "DENY"; decisionId: DecisionId; reasonCodes: string[];
      policyBundleVersion: string }
  | { effect: "STEP_UP_REQUIRED"; decisionId: DecisionId;
      requiredLevel: VerificationLevel; maxAgeSeconds: number;
      purpose: PurposeCode; reasonCodes: string[]; policyBundleVersion: string }
  | { effect: "ALLOW"; decisionId: DecisionId; permission: Permission;
      resourceUrn: ResourceUrn; obligations: Obligation[];
      expiresAt: Timestamp; policyBundleVersion: string };
```

9.4 Obligations

Obligation	Enforcement	Example
FILTER_FIELDS	Return only approved named fields	Order lookup omits payment/contact fields not required for tracking
FILTER_ROWS	Owner injects declared scope predicate	Assigned queue/case/customer relationship
REDACT	Apply classification-aware masking before response/log/export	Email, phone, address, payment reference
REQUIRE_STEP_UP	Return a typed challenge before access	Address change or sensitive account view
REQUIRE_APPROVAL	Workflow creates/awaits approval; does not execute	Refund/replacement authority threshold
READ_ONLY	Reject writes even when read resource is allowed	Platform break-glass diagnostic access
AUDIT_LEVEL	Emit standard or enhanced decision audit	Admin, approval, break-glass, identity changes
MAX_AMOUNT_CLASS	Narrow approver authority class	Exact business amount still comes from Policy Service

9.5 Policy bundle lifecycle and caching

- Authorization source is versioned Rego, JSON schemas, tests, examples, and metadata. CI builds a signed OPA bundle with content digest.
- After security review, cell watchers verify signature, target environment, compatibility, and revocation before atomic activation.
- Every decision records policy bundle version. A bundle is immutable; rollback selects a previous approved version and increments authorization revocation epoch.
- No protected service becomes ready without a valid, non-revoked baseline bundle. If current bundle freshness exceeds its safety window, protected operations fail closed.
- ALLOW may cache for at most 30 seconds and DENY for 5. Keys include every decision-input dimension and expire before assertion, membership, routing, bundle, or revocation validity; STEP_UP_REQUIRED is never an allow.

10. Workforce delegation, approval, and break-glass

A representative never impersonates the customer. The customer remains the subject, the representative remains the actor, and the service workload remains the executor. Human access is case-, queue-, environment-, purpose-, and resource-scoped. Browser UI state cannot grant takeover or approval.

10.1 Case-bound delegation

1	Human Operations assigns or exposes a case according to queue and role policy. Opening a guessed case URL performs no data read before owner authorization.
2	The representative explicitly accepts/takes over the case. Workflow records the transition; Conversation Runtime changes control mode and suppresses automatic replies.
3	Context Authority issues CASE_BOUND TenantContext containing workforce actor, customer subject, case/delegation ID, environment, purpose, role/queue epoch, and route.
4	Each customer/order/conversation read still requires owner OPA authorization and a case-to-resource relation. Assignment alone is not permission to query unrelated tenant data.
5	Representative operations create new audit events with human actor and exact decision. They do not reuse customer confirmation as workforce approval.
6	Release-back or reassignment revokes the delegation epoch. Existing pages lose access on the next request and live sessions receive a control-change event.

10.2 Approval authority

Stage	Rule	Owner
Eligibility	Approver role, tenant/environment, queue/action class, amount class, active membership, no conflict of interest	OPA authorization
Business threshold	Policy Service says approval is required and supplies threshold/obligations	Business Policy Service
Current state	Workflow is awaiting this approval and proposal/preview version is unchanged	Temporal Workflow
Decision	Approve or reject with reason; immutable actor, time, authority and policy references	Human Operations plus Workflow signal
Execution	Approval alone cannot call connector; current confirmation and Action Capability still required	Workflow and Integration Gateway

10.3 Administrative changes

- Identity connections, production role bindings, approval authority, retention, credential scope, residency, and break-glass policy are high-impact.
- High-impact changes require a server preview, recent re-authentication, typed confirmation, immutable audit, and second approval when policy requires.
- A principal cannot approve a change they initiated when separation-of-duties policy requires another actor.
- Tenant Admin cannot grant Platform Operator, disable audit/isolation, alter reserved roles, or assign outside owned tenant/environment; removals and emergency suspensions take effect through epochs immediately.

10.4 Platform break-glass

Area	Decision
Prerequisite	Open incident ID, phishing-resistant MFA, managed device, named tenant/environment, reason, requested operations, and independent approver
Grant	Opaque JIT grant with exact tenant/environment/resources, READ_ONLY default, issue/expiry, no delegation, maximum 30 minutes
Execution	Dedicated support path; PLATFORM_WORKFORCE actor; CASE/INCIDENT_BOUND purpose; enhanced audit; 100-percent session and decision review
Forbidden	Customer business action, role self-elevation, credential export, disabling audit/isolation, unrestricted tenant search, copying content to unmanaged systems
End	Automatic expiry/revocation, open sessions terminated, access report attached to incident, tenant notification according to contract/policy

11. Tenant isolation enforcement

ISOLATION INVARIANT

Every query, row/item, event, workflow, cache key, object path, retrieval candidate, model request, tool invocation, human view, export, trace, and evaluation sample is bound to exactly one tenant and environment before protected data is accessed. Cross-tenant exposure is severity zero and a release-blocking failure.

11.1 Edge and data isolation matrix

Layer	Required control	Hard boundary
Edge	Trusted host/channel plus verified identity membership; remove external internal-context headers; per-tenant origin/rate/upload policy	Tenant mismatch never falls back to lookup
Application	Mandatory branded TenantContext and AuthorizedContext; no optional tenant parameter; owner PEP at every protected handler	Missing context fails before domain code
PostgreSQL	Tenant/environment columns, composite ownership constraints, transaction-local scope, RLS and non-bypass application role	RLS is backstop; owner query still filters [S9]
DynamoDB	Tenant/environment in partition key and conditional access; owner-generated key builders; no Scan in online path	Cross-tenant key cannot be represented by repository API
Kafka	Tenant/environment in signed/validated envelope and partition key; producer allowlist and consumer authorization	Consumer cross-checks owner resource; no topic per tenant
Redis	Typed key builder includes environment/tenant/purpose/schema; per-tenant quota; short TTL	No shared semantic answer or durable sensitive truth

11.2 Workflow, AI, and operations isolation matrix

Layer	Required control	Hard boundary
S3	Owner creates tenant/environment/class prefix or access point and opaque resource handle; short signed URL	Caller cannot supply arbitrary bucket/key
OpenSearch	Server injects tenant/environment/audience/ACL/product/region/locale/effective/revocation filters during candidate generation	No post-search tenant filtering
Temporal	TenantScope and actor snapshot in workflow input; owner task queues/namespaces per environment; each Activity reconstructs context	No external token in history
Model	Only authorized facts/evidence; one tenant per request; context assertion and internal IDs excluded	No cross-tenant prompt/semantic-response cache
Human UI	Backend-generated opaque resource links; assignment/role/resource checks per request and subscription	Route or hidden UI control is not authorization
Telemetry	Tenant pseudonym plus environment and safe IDs; redaction before export; purpose-limited lookup	No raw token, role list, prompt, transcript by default

11.3 Repository enforcement pattern

```
async function loadOrder(
  authz: AuthorizedContext<"order:read">,
  orderId: OrderId
): Promise<OrderView | NotFound> {
  const { tenantId, environmentId } = authz.tenantContext.tenant;
  return db.transaction(async tx => {
    await setLocalTenantScope(tx, tenantId, environmentId);
    return orderRepo.getScoped(tx, {
      tenantId,
      environmentId,
      orderId,
      customerId: authz.tenantContext.subject?.customerId
    });
  });
}
```

Part 3 will define SQL and keys. Part 2 freezes the call shape: caller cannot omit tenant/environment, resource owner derives scope from TrustedTenantContext, and database session scope is set inside the transaction. The application database role must not have BYPASSRLS or table-owner behavior in the normal path.

11.4 Resource-handle rules

- Cross-service payloads use typed opaque resource handles that resolve only through the owning service under TenantContext. Raw S3 keys, search index names, database keys, and provider IDs are not general-purpose client inputs.
- A handle contains or references tenant/environment and resource type under owner integrity protection. Resolution compares handle scope to TenantContext before any provider/datastore call.
- Not-found and unauthorized are intentionally indistinguishable at external edges where existence is sensitive. Internal audit retains the actual reason code.
- Batch operations declare one tenant/environment. Mixed-tenant batches are rejected rather than split implicitly.
- Background administration iterates explicit tenant partitions from a trusted assignment. There is no universal 'all tenants' TenantContext.

11.5 High-isolation tenant tier

The default tier uses pooled services with logical isolation proven above. A configured high-isolation tier may add dedicated databases, OpenSearch domains, KMS keys, AWS accounts, or cells, but it uses the same TenantContext, authorization, audit, and negative tests. Physical separation supplements rather than replaces application enforcement.

12. Workflow, event, AI, retrieval, and tool binding

Later LLDs inherit the identity contract through explicit projections. They may add fields but cannot accept a weaker source, forward credentials into untrusted systems, or treat identity confidence as authorization.

Subsystem	Identity projection	Forbidden shortcut
Conversation	TenantScope, actor, customer subject, verification level/event, purpose, route, message/request IDs	No external/internal bearer token in record
Kafka event	Tenant/environment, event ID/version, resource owner/ref, actor snapshot ref, producer workload, classification	Consumer validates producer and owner; event claim alone is not access
Temporal workflow	TenantScope, DurableActorSnapshot, journey/resource IDs, routing epoch reference	Activity reauthorizes; Workflow code performs no token exchange
Retrieval	Tenant/environment, purpose, audience/ACL, authorized subject/resource handles, knowledge release and revocation epoch	Filters apply before candidate scoring
Model request	Redacted actor class, authorized facts/evidence, journey state, allowed capability projection	No token, raw roles, membership, secret, tenant-private unrelated content
Business read tool	AuthorizedContext with exact read permission, object/field/purpose obligations, caller workload	Agent proposal cannot widen the owner authorization
Consequential action	Workflow/current identity references are inputs to Action Capability issuance	TenantContext alone cannot execute; model never receives capability
Human handoff	Customer subject, original actor snapshot, assigned workforce actor, case/delegation, queue/resource authorization	AI summary remains draft; backend links authoritative data

12.1 Agent Runtime boundary

- Agent Runtime receives a sanitized **AgentIdentityProjection**, not TenantContext assertion. It may know tenant-local behavior, locale, verification class, and authorized resource handles required for the turn.
- The model cannot select tenant, environment, role, verification, purpose, routing epoch, policy version, customer ownership, or allowed tool operations.
- Specialists receive the same or narrower projection. There is no cross-tenant agent memory or autonomous discovery of additional principals/resources.
- Read-tool proposals are schema-validated and sent to an owner endpoint with the original TrustedTenantContext held by Agent Runtime code, not emitted into the prompt.
- A high confidence model output cannot bypass STEP_UP_REQUIRED, DENY, policy, workflow, confirmation, approval, or Action Capability checks.

12.2 Action Capability identity bindings

Binding	Requirement carried into Part 7
Tenant/environment	Exact TenantScope from trusted workflow context
Actor/subject	Original customer or workforce actor, affected customer/account, and delegation
Verification	Required verification event/level and validity at confirmation
Workflow/action	Workflow, action ledger record, state version, target/effect, preview/confirmation/approval references
Executor	Integration Gateway operation and approved connector identity
Route	Current routing epoch/home writer; stale epoch rejected at redemption and datastore

Exact capability format, cryptography, single redemption, and Action Safety storage remain Part 7. This Part freezes that capability issuance cannot use a model-created identity claim or an expired/unverified TenantContext.

13. Revocation, failures, and regional recovery

Cryptographic validity is necessary but insufficient. A token may remain correctly signed after a role is removed, a tenant is suspended, a route is fenced, or a policy bundle is revoked. Kleem therefore uses short assertion lifetimes plus explicit epochs and kill switches.

Epoch	Invalidates	Owner
session_revocation_epoch	Login/logout/compromise state for one interactive session	Edge session and Context Authority
membership_epoch	Role, queue, status, environment, approval authority for one membership	Tenant identity administration
tenant_access_epoch	Tenant-wide emergency access revocation or identity-connection change	Tenant identity administration/security
authorization_epoch	OPA policy bundle revocation/rollback	Control and Knowledge/security
routing_epoch	Active home-cell writer fence	Regional routing/fencing authority
resource_acl_epoch	Owner-specific relation/ACL change	Resource owner

13.1 Revocation objectives

Revocation	Enforcement objective
Interactive session logout/compromise	New assertions denied immediately after Edge state update; active internal assertion expires within 120 seconds
Membership suspension/role removal	Identity event to regional projections; protected writes require current epoch; target propagation objective under 30 seconds
Tenant emergency block	100-percent protected operations denied through tenant-access kill switch; target under 10 seconds
Authorization bundle revoke	Watcher rejects version and swaps to approved prior bundle or fail-closed baseline; target under 30 seconds
Routing fence	Protected writes and capabilities compare authoritative current epoch at owner boundary; stale cell cannot write

These are internal engineering objectives, not external SLAs. Part 10 must load-test and exercise them before production commitment.

13.2 Failure matrix

Failure	Behavior	Forbidden
External JWKS unavailable	Use previously validated key only within configured cache/freshness window; deny new/unknown kid or expired cache	No unverified token acceptance
Identity projection unavailable	Existing low-risk request may continue only if membership/access epoch snapshot is within operation TTL; all admin, protected write, approval, and step-up paths fail closed	Do not treat token role as fallback
Context Authority unavailable	Reject new/exchanged protected calls; preserve durable message/workflow; offer retry/handoff	No service self-mints context
OPA missing/unhealthy/stale	Protected handler unavailable; public health and explicitly public knowledge only	No default-allow policy
SPIRE/certificate renewal failure	Drain before certificate safety margin; current valid connections only until expiry	No fallback to unauthenticated cluster traffic
Route epoch stale	Reads according to owner policy; writes return FENCED_ROUTE and retry through home cell	No application-only bypass
Membership revoked mid-session	Next request/exchange/owner sensitive check denies; revoke takeover/delegation	Page visibility does not preserve access
Step-up expires	Return STEP_UP_REQUIRED; invalidate pending confirmation when identity obligation was material	No silent reuse of old verification
Authorization timeout	DENY/DEPENDENCY_UNAVAILABLE according to operation; never reuse unrelated cached allow	No broader degraded access

13.3 Regional recovery gate

- Context Authority keys and trust bundles are region/cell scoped. A recovery cell may verify historical assertions for audit but cannot mint writable context until routing and identity-readiness gates pass.
- Tenant routing snapshot includes home region, home cell, routing epoch, status, residency, and recovery state. Context issuance never invents a route from deployment environment.
- Before a recovery cell serves protected writes: old writer is fenced, new routing epoch is durable, Action Safety continuity is proven, identity and policy bundles are current, SPIFFE trust is healthy, and queued/unknown actions are reconciled.
- An assertion minted before the fence carries the old routing epoch and is rejected at every protected owner and Action Capability redemption.
- Recovery does not erase actor, decision, confirmation, approval, submission, or audit records. It changes future execution authority.

14. Audit, privacy, and observability

Identity telemetry must prove decisions without becoming a credential warehouse or customer-data replica. Full tokens, certificates, authentication factors, raw PDP inputs, and customer content are never logged. Correlation uses opaque IDs and hashes.

Event	Allowed fields	Excluded
Authentication	authentication_event_id, adapter/issuer ID, principal ID, result/reason, verification level, risk class, session hash, time	No token, OTP, email/phone by default
Tenant resolution	tenant/environment, route source, membership ID/epoch, routing epoch, result/mismatch code	No list of alternative tenant matches
Context	context ID, issuer cell, audience, parent context ID, actor/subject IDs, purpose, expiry, verification class	No serialized assertion

Event	Allowed fields	Excluded
Workload	caller SPIFFE ID, receiver service, mTLS result, certificate serial hash, trust-bundle version	No private key/certificate body
Authorization	decision ID/effect/reason, permission, resource URN hash, policy version, obligations, epochs, latency	No full owner record or raw Rego input
Human	case/delegation, workforce actor, action, approval/break-glass grant, reason, outcome	No customer impersonation or shared actor
Security	mismatch type, suspected replay, invalid audience/type/signature, rate signal, cross-tenant canary	Restricted access and retention

14.1 Correlation contract

```
tenant_pseudonym       environment_id       request_id
trace_id               conversation_id      workflow_id
message_id             authentication_event_id
context_id             parent_context_id    decision_id
principal_id           subject_customer_id  membership_epoch
caller_spiffe_id       policy_bundle_version
routing_epoch          action_id/tool_call_id (when applicable)
```

Tenant IDs may be pseudonymized before export to shared observability systems. Direct tenant lookup is available only through an audited correlation service. Metrics use bounded labels; customer, conversation, resource, and principal IDs never become high-cardinality metric labels.

14.2 Security metrics and alerts

Signal	Required observation
Invalid context assertions	By reason/audience/caller/region; alert on burst or novel issuer/type
Tenant mismatch	Host/token/membership/resource mismatch; immediate security triage
Authorization deny/step-up	By permission/reason/policy version, without customer identifiers
PDP bundle freshness	Active version, age, signature status, revoked-version attempts
Membership/routing epoch lag	Publisher-to-cell and cell-to-enforcement delay
SPIFFE health	SVID age, renewal failures, trust-bundle convergence, unknown caller
Cross-tenant canaries	Any access success is severity zero and automatic release/traffic stop
Break-glass	Every request, grant, use, denial, expiry, and review completion

14.3 Retention and access

- Authentication, authorization, administrative, approval, delegation, and break-glass audits have tenant-configurable retention with legal/security minimums defined by policy; exact tables and lifecycle are Part 3.
- Security logs use separate access roles from customer support operations. Support agents cannot search platform security events by principal or tenant beyond their case needs.
- Identity data export/deletion follows legal obligations but preserves immutable security/action evidence where retention policy requires it, using pseudonymization when identity linkage is no longer needed.
- Production content capture for debugging requires purpose, time-bound approval, redaction, and audit. It is never enabled globally by a developer flag.

15. Implementation modules and delivery sequence

Part 2 fits within the seven Part 1 release units. It adds shared contracts and enforcement libraries, but identity administration, runtime context issuance, authorization enforcement, and workload PKI retain distinct owners.

Owner	Modules	Boundary
Edge/API	authentication-adapters; hosted-session; tenant-resolver; context-authority-api; identity-read-projection; step-up; public auth errors	Issues first context; no business repository
Each runtime release	context-verifier; context-exchange-client; workload-identity interceptor; authorization PEP client; obligations; tenant-key/resource-handle helpers	Cannot mint or bypass context
Control/Knowledge	identity-administration; role/policy catalog; OPA bundle build/publication/revocation; federation registration	No synchronous normal-turn dependency
Human Operations	case-bound delegation; takeover/release; approval authority projection; break-glass UI integration	No customer impersonation
Workflow Workers	actor-snapshot contracts; continuation reconstruction Activity; fresh authz/verification checks	No bearer tokens in history
Integration Gateway	read/action PEP; connector workload identity; tenant credential scope; action identity binding	Capability execution remains Part 7
Platform/infra	SPIRE server/agent; trust bundles; service-account/IRSA bindings; mesh mTLS; network policies	No shared workload identity

15.1 Canonical repository additions

```
contracts/
  identity/v1/{principal,verification,tenant-context,actor-snapshot}.proto
  authorization/v1/{input,decision,obligation}.proto
  identity/json-schema/{context-assertion,policy-input}/v1/
  generated/{typescript,python}/identity/

packages/platform/
  identity-context/        workload-identity/       authorization-pep/
  tenant-resource-handle/  tenant-key-builders/     auth-errors/

deployables/edge-api/src/modules/
  authentication/          tenant-resolution/       context-authority/
  identity-read-model/     sessions/                step-up/

deployables/control-knowledge/src/modules/
  identity-administration/ authorization-policy/    federation-registry/

policies/authorization/
  rego/                    schemas/                  tests/
  bundles/                 metadata/

tests/
  isolation/               identity-conformance/    policy/
  workload-identity/       context-exchange/        break-glass/
```

15.2 CI and architecture gates

Gate	Release-blocking proof
Contract	Protobuf/JSON Schema lint, generated TS/Python drift, cross-language canonical claim fixtures
JWT/profile	Wrong typ/alg/iss/aud/kid/time, duplicate claims, malformed JSON, token substitution, key rotation
Policy	OPA fmt/check/test, strict schema, default deny, reason/obligation coverage, signed-bundle verification
Dependencies	Only Edge Context Authority can access signing port; external token libraries forbidden outside auth adapters; Agent code cannot import action capability
Workload	Every workload role has unique service account/SPIFFE ID/IRSA; call graph and mTLS tests
Isolation	Repository/cache/search/object/event/workflow/model/human negative suites with two or more synthetic tenants
Revocation	Membership, session, policy, route, and tenant-access epoch convergence and stale-cache tests
Supply chain	Signed identity/policy artifacts, SBOM, secret scan, image scan, provenance

15.3 Implementation sequence

Step	Deliverable	Proof
1	Contracts and testkit	Principal/verification/context/decision schemas; canonical fixtures; identity stub; two-tenant negative harness
2	Workload foundation	SPIRE/SVID, mTLS, unique service accounts/IRSA, caller verification, call-graph policy
3	Edge authentication	Hosted OIDC PKCE, commerce-session adapter, anonymous profile, tenant resolver, secure session
4	Context Authority	Issue/exchange/reconstruct, KMS-backed keys, rotation, audience graph, assertion conformance
5	Authorization	OPA sidecars, signed bundles, PEP library, obligations, decision audit, fail-closed readiness
6	Owner enforcement	Conversation then Integration-read repositories, cache/object handle builders, cross-tenant suites
7	Workforce	SSO/MFA membership, roles/queues, case delegation, approvals, admin confirmation, break-glass
8	Durable context	Kafka/Temporal actor snapshots and reconstruction; stale identity/routing/verification behavior
9	Journey proof	Order tracking identity slice first; then eligibility and each consequential journey before Part 10 load/chaos

15.4 Required ADRs

ADR	Decision to record
ADR-009	External identity profiles and first provider adapter
ADR-010	Internal Context Assertion JWS profile, keys, lifetime, and Context Exchange
ADR-011	SPIFFE/SPIRE mTLS plus Kubernetes service-account/IRSA workload identity
ADR-012	OPA 1.x cell-local PDP, signed bundles, and fail-closed behavior
ADR-013	Actor/subject/executor separation and durable ActorSnapshot
ADR-014	RBAC plus ABAC role catalog, obligations, and authorization/business-policy separation
ADR-015	Tenant isolation enforcement interfaces and resource handles
ADR-016	Membership, policy, session, and routing revocation epochs

16. Tests, acceptance, and freeze decision

Tenant isolation and unauthorized action tests are invariants, not average quality metrics. Any observed cross-tenant read/write, context forgery, stale-route protected write, customer impersonation, unauthorized human view, or default-allow authorization blocks release.

Suite	Required cases
Authentication	Wrong issuer/audience/type/algorithm/key/time; ID token used as API token; PKCE/nonce/state; JWKS rotation/outage; commerce-token replay
Tenant resolution	Host-token-org-membership disagreement; ambiguous membership; suspended tenant; environment mismatch; attacker tenant header
Context	Missing/expired/wrong-audience/modified assertion; parent broadening; illegal audience exchange; replay to another service; key rotation
Workload	Unknown SPIFFE ID, wrong trust domain/cell/service account, expired SVID, call-graph violation, IRSA privilege boundary
Authorization	Default deny, explicit deny precedence, stale/unsigned/revoked OPA bundle, obligation omission, cache-key dimension removal, PDP timeout
Resource isolation	Tenant A requests Tenant B IDs across PostgreSQL, DynamoDB, Redis, S3, OpenSearch, Kafka, Temporal, model, human UI, exports, logs
Verification	Expired/wrong-purpose/wrong-resource step-up; manual verification reuse; order/account enumeration; session rotation
Workforce	Unassigned case, wrong queue/env, self-elevation, conflict-of-interest approval, takeover revocation, platform ambient access, break-glass expiry
Durable work	Expired request token in history prohibited; resumed workflow reauthz; revoked membership; stale routing epoch; event producer spoof
AI/tool	Prompt asks to change tenant/role/verification; malicious tool/resource ID; context leakage to model/provider; action attempt with context only

16.1 Property and concurrency tests

- For any generated pair of distinct tenants, no resource created under tenant A is observable or mutable using any valid principal/context from tenant B.
- Exchanging a context never changes tenant, environment, actor, subject, delegation, routing epoch, or verification upward; target expiry is never later than parent expiry.
- Changing any authorization cache dimension changes the cache key. Removing one dimension in mutation testing must make the isolation suite fail.
- A membership/route/policy revoke racing with an allow decision cannot authorize a protected write after the owner observes the new epoch.
- Two simultaneous takeovers cannot both become active; a released delegation cannot continue reading through an existing SSE/subscription.
- No logged event, trace, error, event payload, workflow history, or model request contains a recognized token/certificate/OTP/secret pattern.

16.2 Journey identity proof

Journey	Part 2 proof
Order tracking	Anonymous can ask general help; V2 required for protected order; ownership and field obligations; mismatch is non-enumerating
Eligibility	V2 subject and order facts; OPA access then business Policy; knowledge only explains current authorized result
Create return	Current verification, exact subject/order, confirmation, workflow, and later capability; context alone cannot create
Damaged replacement	Evidence handle scoped; address change V3; human case delegation/approval; no model identity judgment
Refund	V2/V3 policy, approver authority, subject/payment target, current route, no customer/representative identity collapse
Cancellation	Verified subscription ownership, effect preview, step-up when required, material change re-confirmation
Human escalation	Anonymous marked V0; verified subject when available; queue/assignment/case delegation; automation suppressed on takeover

16.3 Acceptance checklist

Result	Criterion
PASS	Every principal, actor, subject, executor, tenant, environment, purpose, verification, route, and revocation source has one defined owner.
PASS	External credentials stop at Edge and cannot be confused with internal context or Action Capability tokens.
PASS	TenantContext has an exact immutable schema, issuer/audience/lifetime profile, constrained exchange, and durable-continuation replacement.
PASS	Every workload role has cryptographic identity, declared call graph, and least-privilege cloud identity.
PASS	OPA authorization and Business Policy Service answer different questions and compose by all-required allow.
PASS	V1 roles, delegation, approvals, self-elevation prevention, and platform break-glass are explicit.
PASS	All datastore, event, workflow, retrieval, model, tool, human, and telemetry paths inherit tenant/environment enforcement.
PASS	Revocation and failure behavior are fail-closed for protected operations and preserve durable work for retry/handoff.
PASS	Cross-tenant, context forgery, stale-epoch, and unauthorized-human tests are hard release gates.
PASS	The design preserves all seven journeys, HLD amendments A1-A10, and Part 1 dependency/authority boundaries.

16.4 Decisions frozen in Part 2

- Identity, TenantContext, workload identity, authorization, business policy, confirmation, approval, and Action Capability remain separate authorities.
- External access and identity tokens are verified only at Edge and are never propagated into internal services or durable state.
- Tenant route plus current internal membership determines tenant/environment; caller/model-provided tenant selection is forbidden.
- TrustedTenantContextV1, Internal Context Assertion JWS profile, two-minute maximum lifetime, exact audience, constrained exchange, and ActorSnapshot are frozen.
- SPIFFE/SPIRE X.509 SVID mTLS plus unique workload service accounts and IRSA are the V1 workload identity baseline.
- OPA 1.x with signed cell-local bundles is the platform authorization engine; resource owners enforce decisions and obligations.

- RBAC plus ABAC, the seven workforce roles, customer verification ladder V0-V4, case-bound delegation, and JIT break-glass are frozen.
- Short-lived assertions are not durable authority; events, Temporal Activities, and jobs reconstruct current execution context.
- Tenant/environment enforcement is mandatory at every data, event, workflow, retrieval, model, tool, human, export, and telemetry boundary.
- Any missing/stale/unverifiable protected identity or authorization dependency fails closed; routing fences and revocation epochs override cached allows.

16.5 Part 2 freeze decision

PART 2 FROZEN	Identity and TenantContext are complete enough for all later LLDs to inherit without developer interpretation. No unresolved identity decision blocks Part 3. Exact persistence structures, indexes, constraints, RLS SQL, sessions, identity epochs, audit storage, keys, and migrations are the next design.

16.6 Next LLD

LLD Part 3 will define database and storage schemas: tenant and environment records, principals and memberships, role bindings, conversations/messages, workflows and projections, policy/authorization audit, action safety references, outbox/inbox, PostgreSQL keys/RLS, DynamoDB items, Redis keys, S3 paths, OpenSearch mappings, migrations, retention, and backup/restore constraints.

References

[I1] Customer Agent OS Lite - V1 Product Contract, July 2026.

[I2] Customer Agent OS Lite - Combined High-Level Design v1.0, July 2026.

[I3] Customer Agent OS Lite - Integrated HLD Validation and Architecture Freeze v1.1, July 21, 2026.

[I4] Customer Agent OS Lite - LLD Part 1: Codebase and Deployable Structure v1.0, July 21, 2026.

[S1] RFC 9700, Best Current Practice for OAuth 2.0 Security. https://www.rfc-editor.org/info/rfc9700/

[S2] RFC 8725, JSON Web Token Best Current Practices. https://www.rfc-editor.org/info/rfc8725/

[S3] OpenID Connect Core 1.0 incorporating errata set 2. https://openid.net/specs/openid-connect-core-1_0.html

[S4] NIST SP 800-63-4, Digital Identity Guidelines. https://pages.nist.gov/800-63-4/

[S5] NIST SP 800-207, Zero Trust Architecture. https://csrc.nist.gov/pubs/sp/800/207/final

[S6] SPIFFE, An overview of the SPIFFE specification. https://spiffe.io/docs/latest/spiffe-about/overview/

[S7] AWS EKS, IAM roles for service accounts. https://docs.aws.amazon.com/eks/latest/userguide/iam-roles-for-service-accounts.html

[S8] Open Policy Agent, HTTP API authorization and bundle management. https://www.openpolicyagent.org/docs/http-api-authorization.html ; https://www.openpolicyagent.org/docs/management-bundles

[S9] PostgreSQL 18, Row Security Policies. https://www.postgresql.org/docs/current/ddl-rowsecurity.html

[S10] RFC 8693, OAuth 2.0 Token Exchange. https://www.rfc-editor.org/info/rfc8693/

[S11] RFC 9470, OAuth 2.0 Step Up Authentication Challenge Protocol. https://www.rfc-editor.org/info/rfc9470/

[S12] AWS Prescriptive Guidance, Multi-tenant SaaS authorization and API access control. https://docs.aws.amazon.com/prescriptive-guidance/latest/saas-multitenant-api-access-authorization/

AUTHORITY

If Part 2 conflicts with the V1 Product Contract or Integrated HLD Freeze, the higher-precedence frozen document controls and Part 2 must be corrected. Part 2 may specialize Part 1 interfaces but cannot weaken Part 1 dependency, workflow, streaming, tenant, regional, or action-safety boundaries.

KLEEM AI / LOW-LEVEL DESIGN

Customer Agent OS Lite

LLD Part 3: Database and Storage Schemas

Exact authoritative schemas, tenant keys, indexes, row security, action-safety transactions, projections, cache keys, object paths, search mappings, migrations, retention, deletion, backup, restore, and cross-store correctness for the frozen V1 architecture.

STATUS	PART 3 FROZEN - READY FOR LLD PART 4
CORE DECISION	Aurora PostgreSQL for relational truth; Aurora DSQL for consequential action safety; DynamoDB, Redis, S3, OpenSearch, ClickHouse, Kafka, and Temporal only within declared authority
SECURITY INVARIANT	Every stored row, item, key, object, document, projection, event, workflow reference, backup, and deletion marker is tenant-and-environment scoped
VERSION	1.0 - July 23, 2026

Design rule: choose a store for its consistency and access pattern, then constrain it to one authority. A cache, projection, index, workflow history, or analytics copy never becomes business truth.

1. Part 3 decision

FINAL PART 3 DECISION

Adopt a polyglot persistence model with explicit truth ownership. Aurora PostgreSQL 17.7 LTS is the pooled relational baseline. Aurora DSQL is a narrow, separate Action Safety Store for confirmation, approval, capability redemption, submission identity, and reconciliation. DynamoDB global tables hold rebuildable low-latency projections only. Redis is disposable. S3 owns large immutable content. OpenSearch and ClickHouse are rebuildable derived stores. Temporal owns workflow history; Kafka carries facts; neither is a general database.

1.1 Outcomes frozen by this LLD

- Application-generated UUIDv7 identifiers are canonical; database sequences are not distributed resource identity.
- Every tenant-owned relational table carries tenant_id and environment_id; every foreign key repeats both, preventing a cross-tenant relation from being representable.
- Normal Aurora PostgreSQL application roles are non-owner and have no BYPASSRLS. Repository predicates and PostgreSQL RLS both enforce scope; RLS is a backstop, not a substitute.
- A message, aggregate update, and outbox fact commit in one PostgreSQL transaction. An inbox marker commits with each consumer-owned mutation.
- Consequential action transitions use strongly consistent Aurora DSQL transactions. A 64-shard tenant fence makes route changes conflict with every in-flight action transaction.
- No MREC DynamoDB table is authoritative for capability redemption, confirmation, provider submission, or completion.
- All raw customer text and selected PII use envelope encryption. Secrets remain secret references; they are not database values, object metadata, events, indexes, or logs.
- Retention is implemented with classification, retain-until, legal-hold, deletion-ledger, and downstream erasure state. Restore always reapplies the deletion ledger.
- All schema changes use owner-scoped, forward-compatible expand/migrate/contract releases. Runtime startup checks compatibility but never auto-migrates production.
- Cross-tenant, stale-epoch, duplicate-action, false-completion, and unreconciled-restore tests are hard release gates.

1.2 Scope and deferments

Class	Boundary
Included	Store authority, relational schemas, identifiers, keys, constraints, indexes, RLS, transactions, Action Safety Store, DynamoDB items, Redis keys, S3 paths, OpenSearch mappings, ClickHouse baseline, outbox/inbox, encryption metadata, migrations, retention, deletion, backup and restore.
Deferred to Part 4	Public/admin/realtime API payloads, endpoint names, pagination cursors, error mapping, and SSE protocol.
Deferred to Part 5	Complete Kafka topic catalog and exact event payloads. Part 3 freezes only persisted envelope/outbox/inbox fields.
Deferred to Part 6-9	Temporal state machines, Action Capability cryptographic format, connector contracts, agent internals, and complete knowledge ingestion/retrieval algorithms.
Deferred to Part 10	Final capacity numbers, SLOs, production DR runbooks, chaos schedules, telemetry/evaluation schemas, and compliance approval.
Not reopened	Seven release units, TenantContext, OPA, one active writer per tenant, Temporal journey authority, validated-only delivery, no model write tools, and HLD amendments A1-A10.

1.3 Language continuity

The physical schemas are runtime-neutral. TypeScript repository adapters and migration verification are the V1 reference path for online owners. Python workers may access only Control/Knowledge-owned ingestion and evaluation contracts through generated clients or explicitly granted schemas. No Python job receives a broad application database role. Go is not introduced in V1.

2. Storage portfolio and truth ownership

Figure 2.1 - The store does not determine authority; the owning service and frozen contract do.

Store	Authoritative for	Never authoritative for	Consistency / recovery role
Aurora PostgreSQL	Tenant/environment directory, identity membership, conversation ledger, human cases, control metadata, outbox/inbox	External commerce outcome; action redemption; Temporal history	Single home-region writer; Multi-AZ; cross-region replica; PITR and snapshots
Aurora DSQL	Consequential action safety chain and routing fence	Conversation text, knowledge, tenant administration, workflow history	Strongly consistent multi-Region peer endpoints plus witness; transactional fencing
DynamoDB MREC	No business truth; rebuildable runtime read models and release snapshots	Action, confirmation, completion, membership source	Low-latency regional projections; last-writer-wins accepted only because source is elsewhere
Redis / Valkey	No durable truth	Identity, permission, rate-limit evidence when safety requires durable enforcement, workflow/action state	Short-lived cache, token bucket, lock hint, connection metadata; safe loss
S3	Original/derived knowledge objects, attachments/evidence, exports, immutable audit archive	Authorization, workflow state, action status	Versioned large-object durability; class-specific replication and Object Lock
OpenSearch	No source truth; published search generation only	ACL, policy, source ownership, action status	Rebuildable from S3 plus PostgreSQL manifests
Temporal	Workflow execution history and deterministic journey state	Conversation DB, object store, retrieval index, action result	Durable replay; payload references rather than large/private bodies
Kafka / MSK	Ordered durable facts in retention window	Aggregate source of truth or exactly-once business effect	At-least-once delivery with transactional outbox and consumer inbox
ClickHouse	No transactional truth; redacted analytics/evaluation copy	Authorization, customer-visible status, audit source	Rebuildable analytical projections with bounded retention

2.1 Database placement

- Each runtime cell has one Aurora PostgreSQL cluster and one non-owner application role per release unit. A tenant's operational writes go only to its home cell.
- Aurora Global Database secondaries may serve explicitly stale-tolerant projections, but unplanned failover can lose unreplicated writes; no lost relational record may be interpreted as proof that an external effect did not occur [A2].
- The Action Safety Store is deployed as Aurora DSQL multi-Region peer clusters with a witness. A tenant belongs to one DSQL cohort; both peer endpoints expose one strongly consistent logical database [D1][D2].
- Across three runtime regions, cohort pairs are deployed as needed (A+B with witness C, B+C with witness A, C+A with witness B). A tenant's active cell must be a data peer for its assigned cohort before protected writes are admitted.
- High-isolation tenants may receive dedicated clusters, accounts, indexes, and KMS keys, but schema contracts and negative tests remain identical.

3. Global schema conventions

3.1 Identifier contract

Identifier	Physical type	Generation	Rule
Resource IDs	uuid	Application UUIDv7	Opaque externally; never derived from email, slug, provider ID, or tenant
Event / request / trace IDs	uuid or 16-byte trace	Ingress/owner	Idempotency and correlation only; not authorization
External provider IDs	text ciphertext or HMAC lookup	Provider	Never a public Kleem resource ID; tenant-scoped mapping required
Sequence number	bigint	Owner transaction	Monotonic only inside one aggregate, such as a conversation
Epoch / version	bigint	Authoritative owner	Starts at 1; compare-and-swap; no timestamp-based concurrency
Hashes	bytea / char(64)	SHA-256 or HMAC-SHA-256	Canonicalization version recorded; hash is evidence, not secrecy
Object handle	uuid + owner/type	Owner service	Resolves under TenantContext; raw S3/database/search key never crosses boundary

UUIDv7 follows RFC 9562 and improves time-order locality without a central sequence [U1]. IDs remain opaque: clients must not parse timestamps or infer authorization from their structure.

3.2 Required tenant columns

```
-- Required on every pooled tenant-owned row.
tenant_id       uuid        NOT NULL,
environment_id  uuid        NOT NULL,
record_version  bigint      NOT NULL DEFAULT 1 CHECK (record_version >= 1),
created_at      timestamptz NOT NULL,
updated_at      timestamptz NOT NULL,
created_by      uuid,
updated_by      uuid

-- Every primary/unique/foreign relationship repeats tenant and environment.
PRIMARY KEY (tenant_id, environment_id, resource_id)
FOREIGN KEY (tenant_id, environment_id, parent_id)
  REFERENCES owner.parent (tenant_id, environment_id, parent_id)
```

- All times are UTC timestamptz. A locale or customer timezone is display configuration, never stored by dropping timezone.
- Money is numeric(19,4) plus ISO-4217 currency code. Floating point is prohibited for policy, preview, and action amounts.
- Canonical JSON uses RFC 8785-compatible deterministic serialization before hashing; the canonicalization version is stored beside each digest.
- Resource state is explicit. Absence, soft deletion, retention expiry, legal hold, and security suspension are distinct.
- Mutable aggregates use record_version compare-and-swap. Append-only records have immutable content plus a separate supersession/revocation record.

3.3 Schema ownership

PostgreSQL schema	Owning release unit	Examples
platform	Control and Knowledge / platform	tenant, environment, routing projection, retention, key metadata
identity	Edge/API + Control/Knowledge	principal, external identity, membership, role, session, verification
conversation	Conversation Runtime	conversation, participant, message, encrypted payload, attachment, turn record
workflow	Workflow Workers	workflow projection, actor snapshot, policy decision, non-action preview
human	Human Operations	case, queue, assignment, delegation, note, approval projection
control	Control and Knowledge	agent release, integration registration, policy bundle, rollout
knowledge	Control and Knowledge	source, source version, chunk manifest, knowledge release, revocation
integration	Integration Gateway	connector registration projection, read audit, reconciliation projection
events	Each local owner	transactional outbox and inbox
audit	Audit pipeline owner	append-only decision/security/action references before WORM archive

4. Tenant, environment, routing, and retention schemas

4.1 Core directory tables

Table	Primary key	Important fields / constraints	Owner / use
platform.tenants	tenant_id	canonical_slug unique; status; isolation_tier; tenant_access_epoch; residency_policy_id; no customer content	Control-plane tenant identity
platform.environments	(tenant_id, environment_id)	environment_class unique per tenant; status; home_region; home_cell; routing_epoch; authorization_epoch	Trusted route/environment source
platform.routing_history	(tenant_id, environment_id, routing_epoch)	old/new cell; reason; fence status; initiated/activated actor; action-safety readiness reference	Immutable route-change evidence
platform.retention_policies	(tenant_id, environment_id, retention_policy_id)	data_class; retain_days; security_min_days; archive behavior; deletion mode; legal basis code	Lifecycle configuration
platform.key_bindings	(tenant_id, environment_id, key_purpose, key_version)	KMS/key reference; status; created/retired; public thumbprint where applicable; no private key bytes	Encryption/signature metadata
platform.deletion_requests	(tenant_id, environment_id, deletion_request_id)	subject/resource scope; policy version; legal hold check; state; requested/approved/completed timestamps	Authoritative erasure coordinator
platform.deletion_targets	(tenant_id, environment_id, deletion_request_id, target_store, target_ref)	expected version; attempt; outcome; tombstone hash; completed_at	Verifiable downstream propagation

4.2 Representative DDL

```
CREATE TABLE platform.environments (
  tenant_id            uuid        NOT NULL,
  environment_id       uuid        NOT NULL,
  environment_class    text        NOT NULL
    CHECK (environment_class IN ('DEV','STAGING','PRODUCTION')),
  status               text        NOT NULL
    CHECK (status IN ('PROVISIONING','ACTIVE','SUSPENDED','DELETING')),
  home_region_id       text        NOT NULL,
  home_cell_id         text        NOT NULL,
  routing_epoch        bigint      NOT NULL CHECK (routing_epoch >= 1),
  authorization_epoch  bigint      NOT NULL CHECK (authorization_epoch >= 1),
  knowledge_rev_epoch  bigint      NOT NULL CHECK (knowledge_rev_epoch >= 1),
  retention_policy_id  uuid        NOT NULL,
  record_version       bigint      NOT NULL DEFAULT 1,
  created_at           timestamptz NOT NULL,
  updated_at           timestamptz NOT NULL,
  PRIMARY KEY (tenant_id, environment_id),
  UNIQUE (tenant_id, environment_class)
);
```

ROUTING AUTHORITY Aurora PostgreSQL holds the control-plane directory and signed runtime projection. It does not by itself fence a consequential action. Protected action fencing is independently enforced inside Aurora DSQL in the same transaction as every action state transition.

5. PostgreSQL isolation and connection contract

5.1 Transaction-local scope

```
BEGIN;
SELECT set_config('app.tenant_id',      $1::text, true);
SELECT set_config('app.environment_id', $2::text, true);
SELECT set_config('app.request_id',     $3::text, true);

-- Owner query repeats scope even though RLS is active.
SELECT *
FROM conversation.conversations
WHERE tenant_id = $1
  AND environment_id = $2
  AND conversation_id = $4;
COMMIT;
```

Scope is set inside every transaction with is_local=true, so a pooled connection cannot leak tenant state into the next request. Repository adapters reject execution outside the scoped transaction wrapper. Direct pool access is forbidden outside the storage adapter.

5.2 RLS policy template

```
CREATE FUNCTION security.current_tenant_id()
RETURNS uuid LANGUAGE sql STABLE PARALLEL SAFE AS $$
  SELECT NULLIF(current_setting('app.tenant_id', true), '')::uuid
$$;

CREATE FUNCTION security.current_environment_id()
RETURNS uuid LANGUAGE sql STABLE PARALLEL SAFE AS $$
  SELECT NULLIF(current_setting('app.environment_id', true), '')::uuid
$$;

ALTER TABLE conversation.conversations ENABLE ROW LEVEL SECURITY;
ALTER TABLE conversation.conversations FORCE ROW LEVEL SECURITY;

CREATE POLICY tenant_environment_scope
ON conversation.conversations
FOR ALL TO kleem_conversation_app
USING (
  tenant_id = security.current_tenant_id()
  AND environment_id = security.current_environment_id()
)
WITH CHECK (
  tenant_id = security.current_tenant_id()
  AND environment_id = security.current_environment_id()
);
```

- Missing settings evaluate to NULL and match no row. The adapter converts this to an internal isolation error before returning externally.

- The migration owner is separate from every application role. Application roles are neither table owners nor superusers and cannot set BYPASSRLS.
- Views over protected tables use security_invoker=true where available. Security-definer functions require explicit security review and fully qualified names.
- TRUNCATE, COPY FROM, extension install, DDL, and role management are unavailable to runtime roles.
- CI enumerates every tenant table and fails if tenant/environment columns, FORCE RLS, policy, or a leading scoped index are missing.

PostgreSQL documents that superusers, BYPASSRLS roles, and normally table owners bypass row security; FORCE ROW LEVEL SECURITY subjects the owner too [P1]. Kleem therefore uses both role separation and FORCE RLS.

5.3 Index rule

```
-- Good: scope is leading, then the actual access path.
CREATE INDEX messages_by_conversation
ON conversation.messages
  (tenant_id, environment_id, conversation_id, sequence_number);

-- Good: idempotent client retry in exactly one conversation.
CREATE UNIQUE INDEX messages_client_idempotency
ON conversation.messages
  (tenant_id, environment_id, conversation_id, client_message_id);

-- Prohibited in a pooled runtime table:
-- CREATE INDEX ON conversation.messages (conversation_id);
```

6. Identity and TenantContext persistence

6.1 Identity directory tables

Table	Key and critical columns	Invariants
identity.principals	(tenant_id, environment_id, principal_id); kind; status; profile_payload_id	Internal stable identity. Customer, tenant workforce, and platform workforce never collapse.
identity.external_identities	(tenant, env, adapter_id, subject_lookup_hmac) unique; principal_id; subject_cipher_ref; status	Exact issuer/subject mapping. Email or username is never identity key.
identity.customers	(tenant, env, customer_id); principal_id; provider_customer_hmac; pii_payload_id	One tenant-scoped customer. Provider lookup cannot search other tenants.
identity.memberships	(tenant, env, membership_id); principal_id; status; membership_epoch; valid_from/until	Role/queue change increments membership_epoch. Suspended/expired fails closed.
identity.roles	(tenant, env, role_id); role_code; reserved; permissions_version	Seven frozen role codes plus tenant roles constrained by policy. Role is not a token claim.
identity.role_bindings	(tenant, env, binding_id); membership_id; role_id; constraints_json; valid_from/until	Composite FKs prevent cross-scope binding; self-elevation approval reference when required.
identity.queue_memberships	(tenant, env, queue_id, membership_id); authority_class; valid window	Queue assignment is an ABAC input, not broad case access.
identity.hosted_sessions	(tenant, env, session_id); session_hash; principal/membership; idle/absolute expiry; revocation_epoch	Opaque cookie lookup only; provider tokens encrypted in Edge-owned payload store.
identity.authentication_events	(tenant, env, authentication_event_id); adapter; result/reason; risk; occurred_at	Append-only metadata; no token, OTP, email, phone, or certificate body.
identity.verification_events	(tenant, env, verification_event_id); customer; level; methods; purpose; valid_until	Evidence references only. V4 is case/resource bound and not reusable.
identity.context_key_metadata	(issuer_cell, key_id); KMS ref; algorithm; public JWK thumbprint; valid/revoked dates	No private key material. Context assertion itself is never stored.

6.2 External subject lookup

```
subject_lookup_hmac =
  HMAC-SHA256(identity_lookup_key_version,
              canonical_issuer || 0x00 || exact_external_subject)

UNIQUE (
  tenant_id,
  environment_id,
  identity_adapter_id,
  subject_lookup_hmac
)
```

- The lookup HMAC supports exact matching without indexing raw external subject. The key version is stored and rotation supports dual lookup during a bounded migration.
- If recovery or provider unlink requires the raw subject, its encrypted payload is separately access-controlled and audited.
- One external subject may not silently map to several active principals inside one tenant/environment/adapter. Collision or duplicate state blocks authentication.

6.3 Membership epoch transaction

A role, queue, approval authority, status, environment, or validity change updates the binding and increments memberships.membership_epoch in one transaction. The same transaction inserts an identity audit event and outbox record. Consumers never calculate the epoch from event arrival order.

```
UPDATE identity.memberships
SET membership_epoch = membership_epoch + 1,
    record_version   = record_version + 1,
    updated_at       = now()
WHERE tenant_id       = $tenant
  AND environment_id  = $env
  AND membership_id   = $membership
  AND record_version  = $expected_version
RETURNING membership_epoch, record_version;
```

7. Conversation and message schemas

7.1 Core tables

Table	Key fields	Important state / constraints
conversation.conversations	PK (tenant, env, conversation_id)	channel; customer subject; status; control_mode AI/HUMAN/QUEUED; version; next_sequence; active workflow/release refs
conversation.participants	PK (tenant, env, conversation_id, participant_id)	principal/customer/workforce kind; joined/left; role in conversation; no bearer tokens
conversation.messages	PK (tenant, env, conversation_id, sequence_number); unique message_id	client_message_id unique per conversation; sender kind/ref; payload_id; status; reply_to; committed version
conversation.message_payloads	PK (tenant, env, payload_id)	INLINE_ENCRYPTED or S3_OBJECT; ciphertext/object handle; content hash; encryption key version; classification
conversation.attachments	PK (tenant, env, attachment_id)	object handle; upload/scan state; size; MIME; checksum; S3 version ID; retention; quarantine reason
conversation.response_validations	PK (tenant, env, validation_id); unique response_message_id	schema version; evidence refs; claim check; guardrail result; validator release; no raw hidden reasoning
conversation.turn_executions	PK (tenant, env, turn_id)	actual agent/workflow/prompt/model/knowledge/policy/tool/eval versions used; cost/latency refs; status
conversation.summaries	PK (tenant, env, conversation_id, summary_version)	encrypted structured summary; through_sequence; evidence refs; superseded_by; release used

7.2 Message commit transaction

1. Lock or compare-and-swap the conversation row at the expected conversation version.
2. Check the unique client message ID. If it exists, return the previously committed message identity.
3. Allocate sequence_number = next_sequence_number; increment next sequence and conversation version.
4. Insert encrypted payload and message metadata.
5. Insert the outbox fact with the same aggregate sequence.
6. Commit. Kafka publication, model execution, and SSE delivery occur only after this durable point.

```
UPDATE conversation.conversations
SET next_sequence_number = next_sequence_number + 1,
    record_version       = record_version + 1,
    updated_at           = $now
WHERE tenant_id          = $tenant
  AND environment_id     = $env
  AND conversation_id    = $conversation
  AND record_version     = $expected_version
RETURNING next_sequence_number - 1 AS allocated_sequence,
          record_version;
```

STALE MODEL OUTPUT

A model response does not reserve a future version. Response commit repeats the expected conversation version and control mode. If the customer sent another material message or a human accepted ownership, the response is stored as discarded telemetry, not shown as customer truth.

7.3 Payload rule

- UTF-8 text up to 32 KiB is envelope-encrypted in message_payloads.ciphertext. Larger or binary content is an S3 object handle.
- The message row stores content type, length, plaintext SHA-256, classification, and payload reference, but no searchable raw customer text.
- Human notes use a separate encryption purpose and authorization path. They are never mixed into customer-visible message payloads.
- A deletion may remove or crypto-shred payload content while retaining a tombstoned message ledger and action/audit references required by policy.

8. Workflow, policy, preview, and human-operation schemas

8.1 Temporal and relational boundary

Temporal is authoritative for the journey state machine. PostgreSQL stores queryable projections and authoritative records owned outside Temporal, such as deterministic policy decisions, human cases, and non-consequential previews. Large payloads and tokens never enter Workflow history.

Table	Role	Truth status
workflow.actor_snapshots	DurableActorSnapshotV1 without session/token; immutable cause evidence	Authoritative actor evidence
workflow.instances	workflow/run/type/journey, state projection, version, routing epoch, current resource handles	Rebuildable projection of Temporal
workflow.fact_snapshots	Typed facts plus authoritative source/version/freshness references	Accepted workflow evidence
workflow.policy_decisions	Input hash, selected policy version, effect, obligations, reason codes, decided_at	Authoritative Policy Service output
workflow.previews	Canonical non-action or eligibility preview; hash; material fields; state version; expiry	Authoritative unless consequential

Table	Role	Truth status
workflow.timers_projection	Named deadline and current visibility for UI/operations	Rebuildable projection
human.cases	Case lifecycle, queue, subject, linked workflow, priority, version	Authoritative Human Operations state
human.assignments	Case/queue/membership, accepted/released, delegation epoch	Authoritative representative ownership
human.notes	Encrypted internal note payload plus classification and author	Authoritative human record
human.approval_projection	Queryable copy of DSQL action approval	Projection only for consequential actions

8.2 Canonical preview

```
canonical_preview_v1 = {
  schema_version,
  tenant_id,
  environment_id,
  workflow_id,
  workflow_state_version,
  action_class,
  resource_handles,
  amount: { value_decimal, currency },
  destination_or_effect,
  fees_and_deductions,
  policy_decision_id,
  policy_version,
  expires_at
}

preview_hash = SHA256(
  "kleem.preview.v1" || RFC8785(canonical_preview_v1)
)
```

- Customer wording is not the hash input. The exact material fields are rendered from the canonical object.
- Any amount, item, quantity, address, timing, destination, consequence, policy, state version, or required approval change supersedes the preview and invalidates confirmation.
- Consequential previews and their confirmation/approval chain are copied into the Action Safety Store before capability issuance.

8.3 Human concurrency

At most one active accepted assignment exists per case. A partial unique index covers (tenant_id, environment_id, case_id) WHERE released_at IS NULL AND accepted_at IS NOT NULL. Taking ownership increments the case control version and delegation epoch in one transaction, emits an outbox event, and causes Conversation Runtime to reject automated response commits.

9. Control and knowledge metadata schemas

Table	Key fields	Freeze invariant
control.agent_releases	release_id; manifest_digest; compatibility range; status; approved_by/at; revoked_at	Immutable after publication; active selection is a separate versioned routing record.
control.policy_bundles	bundle_version; digest; target env; signature ref; activation/revocation	Every OPA decision records exact version; revoke increments authorization epoch.
control.integration_registrations	integration_id; connector type; tool contract releases; credential_ref; state	Credential is a secret reference only; AI sees per-turn approved projection.
knowledge.sources	source_id; owner; authority class; ACL policy; region; sync rule; status	Source ownership and ACL cannot be overridden by derived chunk metadata.
knowledge.source_versions	source_version_id; source_id; object handle/version; checksum; effective dates; scan status	Original object is immutable; supersession and revocation are explicit.
knowledge.chunk_manifests	chunk_id; source_version; locator; content hash; S3 derived handle; metadata digest	PostgreSQL/S3 authoritative; OpenSearch document is rebuildable.
knowledge.knowledge_releases	knowledge_release_id; manifest object/digest; embedding model; index generation; status	Turn selects one published, non-revoked release; emergency revocation always wins.
knowledge.release_members	knowledge_release_id + source_version/chunk; ACL/effective digest	Immutable manifest membership; no partially published corpus.
knowledge.revocations	revocation_id; scope source/version/chunk/release; epoch; reason; effective_at	Runtime lookup must observe current revocation epoch or fail closed.

9.1 Publication transaction

1. Write original and derived objects to S3; verify checksum, scan result, classification, and object version.
2. Build a new OpenSearch generation using only manifest members and exact embedding model.
3. Run coverage, ACL, revocation, citation, conflict, and retrieval evaluation gates.
4. Insert immutable knowledge release and member records with the index generation and digests.
5. Publish the active release pointer and outbox fact in one PostgreSQL transaction.
6. Regional consumers install the signed snapshot. Queries require both the selected release and current revocation epoch.

10. Transactional outbox and inbox

10.1 Outbox schema

```
CREATE TABLE events.outbox (
  tenant_id          uuid        NOT NULL,
  environment_id     uuid        NOT NULL,
  event_id           uuid        NOT NULL,
  owner_release_unit  text       NOT NULL,
  aggregate_type     text        NOT NULL,
  aggregate_id       uuid        NOT NULL,
  aggregate_version  bigint      NOT NULL,
  event_type         text        NOT NULL,
  event_schema_version integer   NOT NULL,
  payload_json       jsonb,
  payload_object_handle uuid,
  classification     text        NOT NULL,
  occurred_at        timestamptz NOT NULL,
  publish_after      timestamptz NOT NULL,
  published_at       timestamptz,
  attempts           integer     NOT NULL DEFAULT 0,
  last_error_code    text,
  PRIMARY KEY (tenant_id, environment_id, event_id),
  UNIQUE (tenant_id, environment_id, aggregate_type,
          aggregate_id, aggregate_version, event_type),
  CHECK ((payload_json IS NULL) <> (payload_object_handle IS NULL))
);
```

Each deployable owns its own physical outbox table or partition under the same contract. A dispatcher claims rows with FOR UPDATE SKIP LOCKED, publishes the versioned envelope, and marks publication. A crash may publish twice; consumers must deduplicate.

10.2 Inbox schema

```
CREATE TABLE events.inbox (
  tenant_id          uuid        NOT NULL,
  environment_id     uuid        NOT NULL,
  consumer_name      text        NOT NULL,
  event_id           uuid        NOT NULL,
  event_type         text        NOT NULL,
  producer_id        text        NOT NULL,
  received_at        timestamptz NOT NULL,
  processed_at       timestamptz,
  outcome_code       text,
  source_payload_hash bytea      NOT NULL,
  retain_until       timestamptz NOT NULL,
  PRIMARY KEY (tenant_id, environment_id, consumer_name, event_id)
);
```

- A consumer validates topic, schema, producer workload, tenant envelope, classification, and owner reference before opening its scoped transaction.
- The inbox insert and local state mutation commit together. A duplicate primary-key conflict returns the previously recorded outcome.
- Inbox retention is at least Kafka replay retention plus seven days. Safety-relevant action deduplication remains in DSQL and is not aged out by this rule.
- Large, secret, or restricted payloads never travel through outbox JSON. The event contains a scoped opaque handle and integrity hash.

11. Consequential Action Safety Store

WHY A SEPARATE STORE	An ordinary Aurora Global Database secondary can miss unreplicated transactions during an unplanned failover [A2], and DynamoDB MREC uses last-writer-wins. Neither may decide whether a refund capability was redeemed. Aurora DSQL provides strongly consistent multi-Region endpoints and ACID transactions for this small safety domain [D1][D2].

11.1 Scope

- Only consequential return creation, replacement, refund, subscription cancellation, address-change effect, and provider-backed human action records enter this store.
- The store contains hashes, IDs, exact material action fields, and result references - not conversation text, images, prompts, knowledge content, or credentials.
- Integration Gateway and the narrow capability/confirmation services receive distinct DSQL IAM database roles. Agent Runtime has no network route, credential, schema grant, or generated client.
- Aurora DSQL does not currently provide PostgreSQL row-level security [D5]. Kleem compensates with schema-level grants, separate workload roles, mandatory composite tenant keys, parameterized repository methods, and cross-tenant database tests.

11.2 Safety tables

Table	Primary key	Critical fields
safety.route_fence_shards	(tenant_id, environment_id, shard_no)	routing_epoch; writer_region/cell; mode ACTIVE/FENCING/BLOCKED; fence_version; updated_at
safety.action_intents	(tenant_id, environment_id, action_id)	workflow/state version; action class; resource/argument/preview hashes; amount/effect; policy; route; state/version
safety.confirmations	(tenant, env, action_id, confirmation_id)	preview hash; actor/subject snapshot hash; verification event; confirmed/expires/invalidated; workflow version
safety.approvals	(tenant, env, action_id, approval_id)	approver membership; authority snapshot; decision; amount class; policy; preview hash; immutable time
safety.capabilities	(tenant, env, capability_ref_hash)	action; bound-claims hash; issue/expiry; state; redemption_count; redeemed workload/time
safety.submissions	(tenant, env, action_id, attempt_no)	provider; idempotency key; request hash; state; lease; submitted_at; external result/reference hashes
safety.reconciliation_attempts	(tenant, env, action_id, reconciliation_no)	reason; provider query; observed result; next attempt; owner; immutable timestamps
safety.action_events	(tenant, env, action_id, action_version)	transition; actor/executor; previous/new state; decision refs; canonical record hash; append-only

11.3 Action state

```
PROPOSED
  -> PREVIEW_READY
  -> CONFIRMED
  -> APPROVED_OR_NOT_REQUIRED
  -> CAPABILITY_ISSUED
  -> SUBMISSION_RESERVED
  -> SUBMITTED
  -> SUCCEEDED | FAILED_FINAL

SUBMITTED -> RESULT_UNKNOWN -> RECONCILING
RECONCILING -> SUCCEEDED | FAILED_FINAL | RECONCILING

Any pre-submission state -> INVALIDATED | EXPIRED | DENIED
```

SUCCEEDED requires an authoritative external result reference or a reconciliation result. RESULT_UNKNOWN blocks a second submission. A provider timeout never maps directly to FAILED_FINAL.

12. Atomic fencing, redemption, and submission

12.1 Sixty-four fence shards

Each tenant/environment has exactly 64 fence rows. An action deterministically selects shard_no = first_6_bits(SHA-256(action_id)). Every action transition updates that fence row in the same DSQL transaction. This avoids one hot tenant row while making a route change conflict with all action transactions.

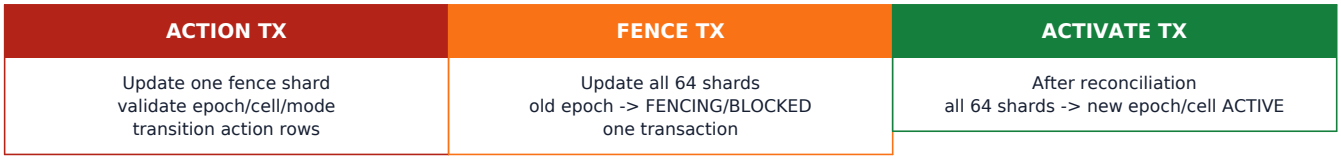


Figure 12.1 - Concurrent old-writer action and fence transactions touch the same shard and cannot both commit.

12.2 Redemption transaction

```
BEGIN;

UPDATE safety.route_fence_shards
SET fence_version = fence_version + 1,
    updated_at    = $now
WHERE tenant_id      = $tenant
  AND environment_id = $env
  AND shard_no       = $shard
  AND routing_epoch  = $expected_epoch
  AND writer_cell_id = $caller_cell
  AND mode           = 'ACTIVE';
-- Require exactly one row.

UPDATE safety.capabilities
SET state             = 'REDEEMED',
    redemption_count  = 1,
    redeemed_at       = $now,
    redeemed_by       = $caller_spiffe
WHERE tenant_id           = $tenant
  AND environment_id      = $env
  AND capability_ref_hash = $capability_hash
  AND action_id            = $action
  AND state                = 'ISSUED'
  AND redemption_count     = 0
  AND expires_at           > $now
  AND bound_claims_hash    = $canonical_claims_hash;
-- Require exactly one row.

UPDATE safety.action_intents
SET state          = 'SUBMISSION_RESERVED',
    action_version = action_version + 1,
    updated_at     = $now
WHERE tenant_id       = $tenant
  AND environment_id  = $env
  AND action_id        = $action
  AND state            = 'CAPABILITY_ISSUED'
  AND action_version   = $expected_action_version
  AND routing_epoch    = $expected_epoch;
-- Require exactly one row.

INSERT INTO safety.submissions (... attempt_no, idempotency_key,
  request_hash, state, lease_until ...)
VALUES (..., 1, $stable_key, $request_hash, 'RESERVED', $lease_until ...);

INSERT INTO safety.action_events (...);
COMMIT;
```

Aurora DSQL uses optimistic concurrency control and reports serialization failures with SQLSTATE 40001; the entire transaction may be retried only from immutable canonical input [D3]. The retry re-reads current state. It never assumes the prior transaction failed before commit.

12.3 Provider call and crash windows

Failure point	Persisted state	Recovery
Before reservation commit	Capability remains ISSUED	Safe to retry the whole redemption transaction.
After reservation, before provider request	SUBMISSION_RESERVED with stable idempotency key and lease	Lease owner retries the same provider request; no new capability or key.
Provider accepted, response received	Transition SUBMITTED then SUCCEEDED with external reference	Customer completion only after DSQL commit and authoritative result.
Timeout after possible submission	RESULT_UNKNOWN	Do not submit again. Reconcile by provider idempotency key/reference.
Worker crashes with expired lease	Reserved/unknown record remains	Reconciler conditionally acquires a new lease and continues the same action identity.
Region fenced mid-call	Old action transaction conflicts or later result commit sees stale epoch	New region reconciles provider before activating related writes.

12.4 Regional fencing transaction

1. Recovery authority proves old runtime ingress and Integration Gateway write identity are fenced.
2. In one DSQL transaction, update all 64 shard rows from old ACTIVE epoch to FENCING at new_epoch. Any in-flight action touching a shard conflicts or later fails the epoch predicate.
3. Query every RESERVED, SUBMITTED, RESULT_UNKNOWN, and RECONCILING action for the tenant/environment. Reconcile each external provider.
4. Verify Temporal and relational projections, current secrets, non-revoked releases, identity/OPA readiness, and provider idempotency behavior.
5. In one DSQL transaction, update all 64 shards to the new writer cell and ACTIVE mode.
6. Only then publish the signed routing snapshot and admit protected writes in the new cell.

DSQL LIMITS The fence transaction mutates 64 rows, well below Aurora DSQL's documented 3,000-row / 10 MiB transaction limits [D4]. DDL and DML are never mixed. The safety repository retries 40001 conflicts with bounded jitter and an absolute deadline.

13. DynamoDB projection schemas

DynamoDB uses MREC Global Tables for low-latency projections because the sources are authoritative elsewhere and one home cell writes each tenant. Last-writer-wins is acceptable only for a rebuildable current view. It is explicitly prohibited for Action Safety.

Table	Key	Item types / access	Recovery
runtime_read_models_v1	PK T#tenant#E#env#TYPE#aggregate; SK CURRENT or segment	conversation summary, identity/membership projection, workflow/human projection; GetItem/Query only	Rebuild from Aurora/outbox; projection_version rejects older home-writer events
release_snapshots_v1	PK T#tenant#E#env; SK AGENT#ACTIVE / KNOWLEDGE#ACTIVE / POLICY#ACTIVE	Signed manifest digest, compatibility, revocation epoch, published version	Rebuild from Control PostgreSQL and S3 manifest; protected use requires fresh revocation
derived_job_dedup_v1	PK T#tenant#E#env#CONSUMER#name; SK event_id	Only indexing/analytics jobs with no local transactional DB mutation	TTL after replay horizon; loss may repeat an idempotent derived job

13.1 Canonical projection item

```
{
  "pk": "T##E##CONV#",
  "sk": "CURRENT",
  "schema_version": 1,
  "tenant_id": "",
  "environment_id": "",
  "owner": "conversation-runtime",
  "source_aggregate_version": 184,
  "source_event_id": "",
  "routing_epoch": 27,
  "projection": { "...allowlisted non-secret fields..." },
  "source_updated_at": "2026-07-23T21:00:00Z",
  "projected_at": "2026-07-23T21:00:00.120Z"
}
```

- Key builders accept a branded TenantScope and typed resource ID; arbitrary strings are not valid repository keys.
- Online paths prohibit Scan and PartiQL without a complete partition key. Batch jobs iterate trusted tenant assignments.
- Conditional update accepts a projection only when source_aggregate_version is greater than the current value.
- GSIs repeat tenant/environment in the partition key. A global cross-tenant GSI is forbidden.
- MRSC is not used here. AWS documents that MRSC does not support transaction APIs or TTL and requires exactly three Regions [DB1]; the safety domain instead uses DSQL transactions.

14. Redis / Valkey keyspace

14.1 Typed key grammar

```
v1:{t::e::p:<00-63>}:::

p = first_6_bits(SHA256(resource_or_principal_id))

Examples:
v1:{t:...:e:...:p:17}:authz:v1:
v1:{t:...:e:...:p:04}:membership:v1:
v1:{t:...:e:...:p:31}:sse:v1:
v1:{t:...:e:...:p:22}:rate:v1:
```

Purpose	TTL / dimensions	Outage behavior
Authorization ALLOW	At most 30 s and before context/membership/routing/bundle/epoch expiry; key includes every PDP dimension	Re-evaluate locally; protected operation fails closed if PDP unavailable
Authorization DENY	At most 5 s; includes policy/revocation epochs	Safe to re-evaluate; never promote stale DENY to ALLOW
Membership / route projection	Shorter than propagation objective; explicit epoch in value	Use authoritative owner or fail closed for protected access
Rate / abuse bucket	Window-specific; tenant/channel/principal/network-risk dimensions	Safety-sensitive class fails closed; low-risk public traffic uses reduced emergency quota
SSE connection	Heartbeat plus grace; no message content	Client reconnects and replays from durable conversation sequence
Safe prompt prefix	Agent release/model/policy/locale scoped; no customer content	Cache miss only affects cost/latency

- No cache entry contains an external token, Internal Context Assertion, OTP, credential, reusable Action Capability, raw prompt, private transcript, or durable action truth.
- Cross-tenant semantic response caching and customer-specific generated answer reuse are prohibited.
- Each tenant has byte/key/operation quotas. High-cardinality keys use HMAC identifiers, not email, phone, IP, or display name.
- Locks are advisory optimization only. Correctness uses PostgreSQL/DSQL versions, unique constraints, and idempotency records.

15. S3 object schemas and paths

15.1 Bucket classes

Bucket class	Objects	Controls
uploads-quarantine	Untrusted customer/source uploads	No human/model/retrieval reads; malware/decompression scan; short lifecycle
evidence-clean	Approved evidence versions	Tenant/env prefix; versioning; KMS; signed access; case/resource authorization
knowledge-original	Immutable source versions	Versioning; source owner/ACL metadata in PostgreSQL; replication by residency policy
knowledge-derived	Parsed pages, chunks, OCR, normalized tables	Rebuildable; checksum and source-version linkage; no authority elevation
exports	Tenant-requested export packages	Per-request key, short expiry, download audit, no public access
audit-worm	Signed audit segments/manifests	Versioning + Object Lock; legal hold; restricted security/audit role
eval-artifacts	Approved de-identified datasets and release reports	Purpose-limited; no automatic raw conversation copy

15.2 Object key grammar

```
s3://-/v1/
  tenant//
  environment//
  class//
  year//month//day//
  object//
  version/
```

- Clients never choose bucket or key. The owner allocates one opaque upload target after authorization.
- Original filename, customer identifier, order number, case title, and email never appear in a key or unencrypted metadata.
- Allowed metadata is limited to schema version, classification, content type, checksum, retention class, and pseudonymous tenant correlation.
- Every database object record stores bucket class, region, key hash, exact S3 version ID, ETag/checksum, encryption key reference, scan status, retain-until, and legal-hold state.
- Presigned URLs are time-limited capabilities generated by the owning service [S2]. V1 defaults: upload <= 5 minutes, human evidence read <= 2 minutes, export download <= 10 minutes.
- Versioning protects against overwrite. The owner seals the first verified upload version; later versions at the same key are quarantined and never replace the sealed object.

S3 Object Lock requires Versioning and supports retention periods and legal holds on exact object versions [S1]. Kleem uses governance or compliance mode according to approved data class; ordinary application roles cannot bypass retention or legal hold.

15.3 Evidence upload state

ALLOCATED -> UPLOADING -> UPLOADED -> SCANNING -> CLEAN or REJECTED / QUARANTINED. Only CLEAN produces a customer/human/retrieval resource handle. The model never receives a quarantine key or presigned URL.

16. OpenSearch knowledge mapping

AUTHORIZATION BEFORE SCORING	Tenant, environment, knowledge release, ACL/audience, effective time, source status, region, locale, product/journey, and revocation filters are injected by the Retrieval Gateway in the hybrid/k-NN query. Post-filtering is prohibited for authorization because an unauthorized candidate must never influence scores, counts, snippets, or timing.

16.1 Index generation

- Physical index: kleem-knowledge-v1-<region>-g<generation>. A generation fixes mapping, analyzer, and embedding dimension.
- Many tenants share a generation, but every document carries exact tenant/environment/release/ACL fields. High-isolation tenants use a dedicated domain/index with the same mapping.
- A PostgreSQL knowledge release becomes visible only after every manifest member is indexed and validation passes. Queries always filter one published release ID.
- Revocation is a second independent filter: document.revocation_epoch <= request.allowed_epoch is insufficient; the gateway also excludes revoked source/version/chunk IDs from the current revocation set.

16.2 Mapping

```
{
  "dynamic": "strict",
  "properties": {
    "schema_version":       { "type": "integer" },
    "tenant_id":            { "type": "keyword" },
    "environment_id":       { "type": "keyword" },
    "knowledge_release_id": { "type": "keyword" },
    "source_id":            { "type": "keyword" },
    "source_version_id":    { "type": "keyword" },
    "chunk_id":             { "type": "keyword" },
    "acl_ids":              { "type": "keyword" },
    "audiences":            { "type": "keyword" },
    "locale":               { "type": "keyword" },
    "product_codes":        { "type": "keyword" },
    "journey_codes":        { "type": "keyword" },
    "region_scope":         { "type": "keyword" },
    "authority_rank":       { "type": "short" },
    "trust_label":          { "type": "keyword" },
    "effective_from":       { "type": "date" },
    "effective_until":      { "type": "date" },
    "revocation_epoch":     { "type": "long" },
    "content_hash":         { "type": "keyword" },
    "citation_locator":     { "type": "keyword" },
    "title":                { "type": "text" },
    "passage_text":         { "type": "text" },
    "embedding": {
      "type": "knn_vector",
      "dimension": "",
      "method": { "engine": "lucene", "name": "hnsw" }
    }
  }
}
```

OpenSearch supports hybrid pre-filtering and efficient k-NN filtering [O1][O2]. Retrieval tests insert cross-tenant canaries and verify that unauthorized documents do not appear in hits, scores, aggregations, suggestions, snippets, or model evidence.

16.3 Rebuild and rollback

1. Create a new generation with strict mapping and fixed model dimension.
2. Read authorized chunk manifests from PostgreSQL and derived content from S3.
3. Bulk index by explicit tenant partitions; validate counts, hashes, ACLs, and citations.
4. Run release retrieval suites. Publish the generation in the immutable knowledge release.
5. Rollback selects a prior non-revoked release; it never edits the old release in place.
6. Delete obsolete generations only after retention, legal hold, and active-release references permit.

17. ClickHouse analytics baseline

ClickHouse receives redacted asynchronous events. It is never queried to authorize access, decide workflow state, reconstruct a capability, or tell a customer that an action completed.

Table	Partition / order	Allowed content
analytics.runtime_events_v1	PARTITION BY toYYYYMM(event_time); ORDER BY (tenant_pseudonym, event_date, event_type, event_time, event_id)	Safe IDs, release/model/tool codes, status class, latency, tokens, cost, region/cell; no raw prompt/transcript
analytics.journey_outcomes_v1	Month; tenant pseudonym, journey, outcome, time	Workflow/result references, handoff reason class, completion latency, repeat-contact signal
analytics.evaluation_results_v1	Evaluation run month; release, dataset, metric, sample hash	Scores, reason codes, cohort labels approved for analysis; content stored only by separate eval artifact policy

- Tenant IDs are HMAC-pseudonymized before export. Direct lookup requires an audited correlation service.
- Customer, principal, conversation, workflow, and resource IDs are not metric labels. Exact IDs may exist only in restricted event columns with retention controls.
- Materialized views are versioned. A corrected source event produces a new correction event; analytics never updates transactional truth.
- Part 10 will freeze capacity, TTLs, sampling, and evaluation schemas. Part 3 freezes store boundaries and prohibited content.

18. Encryption, tokenization, and secret references

18.1 Data protection pattern

1. Classify the field/object before storage: PUBLIC, INTERNAL, CONFIDENTIAL, RESTRICTED_PII, SECRET, or EVIDENCE.
2. Generate a random data-encryption key (DEK) for the payload or bounded record group.
3. Encrypt content with an approved AEAD mode and authenticated context containing tenant, environment, object type, object ID, schema version, and key purpose.
4. Wrap the DEK with the tenant/environment/purpose KMS key. Store ciphertext, nonce, wrapped DEK, algorithm, and key version - never plaintext.
5. Decrypt only in the owning service after current authorization. Redact before logs, model requests, analytics, exports, or human views.

Data	Storage representation	Search / display
Email, phone, address	Encrypted payload; HMAC lookup only when exact lookup is required	Masked projection; no plaintext index
External customer/order/provider ID	Encrypted value or tenant-scoped HMAC mapping	Opaque Kleem handle outside owner
Conversation / human note	Envelope-encrypted inline payload or S3 version	Decrypted for authorized view/context only
Credential / OAuth refresh token	Secrets Manager/Vault reference only	Never database/S3/event/model/log
Internal Context Assertion / access token	In-memory only; session stores opaque hash/metadata	Never persisted or exported
Action Capability	Opaque reference hash and bound-claims hash in DSQL	Raw reference exists only during narrow invocation
Audit content	IDs, hashes, reason codes; restricted encrypted segment when approved	Purpose-limited security/audit access

18.2 Key rotation

- Key metadata has ACTIVE, DECRYPT_ONLY, RETIRED, and COMPROMISED states. A compromised key triggers incident-specific re-encryption and access review.
- New writes use one active key version per tenant/environment/purpose. Reads select by stored key version.
- Lookup HMAC rotation uses dual-compute/dual-read during migration, followed by backfill and old-key retirement.
- KMS deletion is not the ordinary data-deletion mechanism because it may destroy unrelated records. Resource deletion removes ciphertext/DEK references according to the deletion ledger.

19. Retention, deletion, and legal hold

19.1 Retention classes

Class	Initial engineering default	Examples	Rule
R0 TRANSIENT	Seconds to 24 hours	Redis cache, upload allocation, short leases	Loss is safe; never durable truth
R1 SESSION	Expiry plus 7 days metadata	Hosted session metadata, auth challenge	Token/secret is not retained
R2 OPERATIONAL	365 days after closure	Conversation payload, case note, workflow projection	Tenant configurable within approved bounds
R3 EVIDENCE	180 days after case/action closure	Clean attachment and verification evidence	Policy may require shorter/longer; holds override
R4 SECURITY_AUDIT	400 days online, then approved archive	Authentication, authorization, admin, break-glass	Restricted; content-minimized
R5 ACTION_SAFETY	7 years baseline	Confirmation, approval, capability, submission, reconciliation	Never removed before provider replay/legal/safety window
R6 KNOWLEDGE	Active plus 90 days superseded	Source/chunk/release versions	Revocation blocks use immediately; retention supports audit/rollback
R7 ANALYTICS_EVAL	90-180 days by dataset	ClickHouse and approved eval samples	De-identified/purpose-limited; raw conversation not automatic

These are initial engineering defaults, not legal conclusions or customer SLAs. The effective retain_until is computed from product policy, tenant configuration, legal/security minimum, jurisdiction, object state, and legal hold. A shorter tenant value cannot override a mandatory minimum.

19.2 Deletion state machine

```
REQUESTED
  -> POLICY_CHECKED
  -> BLOCKED_BY_HOLD | APPROVED
  -> TOMBSTONED
  -> PROPAGATING
  -> VERIFIED
  -> COMPLETED

Any target failure -> PARTIAL -> retry / incident review
```

- The coordinator creates one target record for Aurora, DSQL, DynamoDB, Redis, S3 versions, OpenSearch, ClickHouse, Kafka-derived projections, and export/evaluation copies that actually apply.
- Redis is evicted immediately. OpenSearch and DynamoDB delete by exact tenant key/manifest, never broad user input.
- S3 deletion enumerates exact version IDs and waits for retention/legal hold. A delete marker alone is not proof of erasure.
- Action and security records under mandatory retention are pseudonymized and delinked rather than falsely reported as erased.
- Backups age out according to policy. Every restore runs the deletion ledger before the restored environment may serve traffic.

- Completion requires per-store evidence and a signed deletion manifest; a queue acknowledgment is not completion.

19.3 Legal hold

A legal hold is immutable evidence referencing exact resource/object versions, scope, authority, reason code, issue time, and release authority. It prevents lifecycle deletion without widening read access. Removing a hold requires a separate authorized event and never rewrites prior audit history.

20. Backup, restore, and regional recovery constraints

Store	Backup / recovery	Restore admission rule
Aurora PostgreSQL	Continuous automated backup with 35-day PITR; daily snapshots; encrypted cross-region copy by residency; Global Database replicas	Restore into new isolated cluster, validate schema/RLS, replay deletion ledger and outbox, compare epochs; never infer external action absence
Aurora DSQL	Strong multi-Region peers plus witness for live safety; scheduled AWS Backup full backups and cross-region copies	Catastrophic restore remains BLOCKED; reconcile every non-final action with provider before activating any fence shard
DynamoDB	PITR plus infrastructure definition; global replicas	Projection version/source event validation; rebuild preferred; no promotion to truth
S3	Versioning; class-specific replication; Object Lock for audit; inventory/checksum	Rebuild manifests; preserve version/hold state; no raw bucket exposure
OpenSearch	Snapshots plus deterministic rebuild from PostgreSQL/S3	New generation only; run ACL/revocation/citation tests before publish
Redis	No correctness backup requirement	Warm from source; protected paths fail closed until required PDP/epoch state is ready
ClickHouse	S3-backed snapshots/exports according to analytics policy	Rebuild from redacted events; never block transactional recovery
Temporal	Provider-supported durability/failover plus replay-tested workflow code	Action Safety reconciliation precedes consequential progress after regional recovery

Aurora supports continuous incremental backups and PITR, while unplanned Global Database failover can lose writes not yet replicated [A2][A3]. Therefore the restored relational cluster may rebuild conversation/workflow projections but cannot decide that a provider write did not happen.

20.1 Recovery gates

- No environment serves protected traffic until identity projections, OPA bundle, routing epoch, key metadata, deletion ledger, and schema compatibility are current.
- No consequential write until DSQL fence shards are active for the new cell and all RESERVED/SUBMITTED/UNKNOWN actions are reconciled.
- No knowledge answer until the selected release manifest, index generation, ACL projection, and revocation epoch pass validation.
- Restore tests include two synthetic tenants and prove absence of cross-tenant rows, keys, objects, search hits, analytics correlation, and deleted payloads.
- Quarterly restore drills measure time and correctness; Part 10 freezes final RPO/RTO objectives and runbooks.

21. Migration and schema evolution

21.1 Migration ownership

- Each release unit owns only its schema and migrations. No deployment applies another owner's DDL.
- Migrations are ordered immutable SQL files plus a manifest containing ID, owner, digest, minimum/maximum application compatibility, lock/statement risk, backfill plan, and rollback disposition.
- Production startup checks schema compatibility and refuses an incompatible version. It never runs migrations automatically.
- One dedicated migration workload identity may execute DDL. It is disabled outside a controlled release window and has no application/session token.
- Cross-store migrations use a durable migration job record and explicit tenant partitions; they are idempotent and resumable.

21.2 Expand / migrate / contract

Phase	Allowed work	Release gate
EXPAND	Add nullable column/table/index, new item/doc version, dual-read-compatible contract	Old and new applications both pass
MIGRATE	Backfill by tenant partitions; dual write; verify counts/hashes; shadow read comparison	No cross-tenant or missing rows; resumable
SWITCH	Select new read path via release config; continue compatibility telemetry	Canary and rollback path proven
CONTRACT	Enforce not-null/constraint, stop old writes, later remove old field/index	No supported release depends on old schema; restore tested

21.3 Store-specific evolution

Store	Evolution rule
Aurora PostgreSQL	Additive SQL first; indexes created safely; validate constraint before enforcement; partition DDL owned and tested
Aurora DSQL	One DDL statement per transaction; DDL separate from DML; handle asynchronous catalog/index state and SQLSTATE 40001 [D3]
DynamoDB	Every item carries schema_version; writers emit new version; readers support bounded prior versions; backfill by exact partition assignments
Redis	Version in key prefix; never mutate meaning of an existing key; old keys expire
S3	Immutable object/schema version; derived object rebuilt; manifest selects version
OpenSearch	New generation for mapping/analyzer/dimension change; reindex and publish; never mutate published release in place
ClickHouse	Versioned table/view; backfill from redacted source; swap consumer after comparison

21.4 Partitioning

- High-volume append tables - messages, outbox, inbox, auth decisions, audit events - use time partitions only when measurement justifies them. Tenant/environment remain in every partition key/index.
- Partition creation is automated ahead of time; a missing partition fails readiness instead of inserting into an unbounded default partition.
- Dropping a partition is allowed only after retention, legal hold, deletion ledger, archive, and active reference checks.
- A partition is operational placement, not a tenant security boundary.

22. Transactions, concurrency, and repository rules

Operation	Atomic unit	Concurrency rule
Create message	Conversation version + payload + message + outbox	Client idempotency unique; expected conversation version
Commit response	Validation + response message + conversation version + outbox	Expected version and AI control mode; stale response discarded
Membership change	Binding/status + epoch + audit + outbox	Expected membership version; self-elevation approval check
Human takeover	Assignment + case/control version + delegation epoch + outbox	One active accepted assignment partial unique constraint
Publish release	Immutable release + active pointer + audit + outbox	Expected active pointer version; revoked dependency cannot activate
Redeem action	Fence shard + capability + action + submission + event in DSQL	Exact hashes/epoch/version; one redemption
Reconcile action	Fence shard + lease/result + action/event in DSQL	Stable action/idempotency key; provider observation required
Consume event	Inbox marker + consumer-owned mutation + local outbox	Duplicate returns prior outcome
Delete resource	Deletion request + targets; per-store idempotent tasks	Holds and retention prevent premature completion

22.1 Repository interface

```
interface ScopedTransaction {
  readonly tenantId: TenantId;
  readonly environmentId: EnvironmentId;
  readonly requestId: RequestId;
  readonly db: TransactionOnlyClient;
}

interface ConversationRepository {
  appendMessage(
    tx: ScopedTransaction,
    command: AppendMessageCommand,
    expectedVersion: bigint
  ): Promise;
}

// There is no:
// getById(id)
// queryAllTenants(...)
// rawPool.query(...)
```

- The scope comes from TrustedTenantContext and is installed by the transaction wrapper. A handler cannot pass a different tenant argument.
- Batch methods accept one scope and reject mixed-tenant inputs. Platform administration iterates explicit assignments, never a universal TenantContext.
- Every mutation returns the new record/aggregate version. Time is metadata, not concurrency control.
- Retries are operation-specific. PostgreSQL serialization/deadlock retries and DSQL 40001 retries use deterministic idempotent commands. External provider calls are never hidden inside a database retry closure.

23. Audit and cross-store reconciliation

23.1 Audit record

```
AuditRecordV1 {
  audit_event_id,
  tenant_id / tenant_pseudonym,
  environment_id,
  actor_ref,
  executor_spiffe_id,
  action_code,
  resource_urn_hash,
  decision_id / policy_version,
  workflow_id / action_id,
  previous_record_hash,
  canonical_record_hash,
  outcome_code,
  occurred_at,
  retention_class,
  classification
}
```

- Audit rows are append-only in PostgreSQL/DSQL owner transactions, then segmented, signed, and archived to S3 Object Lock.
- The per-scope hash chain detects omission/reordering inside an exported segment but does not replace database constraints or external provider reconciliation.
- No raw token, credential, OTP, certificate, prompt, hidden reasoning, unrestricted resource body, or customer attachment is copied into audit.
- Authorization audit records effect/reason/obligations/version and resource hash. Enhanced admin/action/break-glass records require explicit reason and actor.

23.2 Reconciliation jobs

Reconciler	Compares	Correction behavior
Outbox	Committed owner version vs published event	Republish same event ID; never synthesize new aggregate version
Projection	Aurora aggregate/event version vs DynamoDB/OpenSearch/ClickHouse version	Rebuild exact tenant partition; source remains authoritative
Object manifest	PostgreSQL object/version/checksum vs S3 inventory	Quarantine orphan; recreate derived object; incident on missing authoritative original
Knowledge	Release manifest vs OpenSearch docs and revocation state	Remove/rebuild generation; block release if mismatch
Action	DSQL submission/idempotency/external ref vs provider status	RESULT_UNKNOWN remains until authoritative observation; no blind retry
Deletion	Deletion target ledger vs each store	Retry or PARTIAL incident; completion only after evidence
Audit archive	Database segment count/hash vs WORM manifest	Re-export missing segment; preserve chain and reason

24. Journey-to-storage traces

Journey	Relational / workflow truth	Action safety / derived storage
Order tracking	Conversation/message + authorized read audit; Temporal only if durable escalation	No action record. DynamoDB projection may accelerate; response versions recorded; no guessed result.
Eligibility	Workflow fact snapshot + Policy Service decision/version + explanatory knowledge release	No capability. OpenSearch evidence is derived; source/manifest remains PostgreSQL/S3.
Create return	Temporal journey + canonical preview + policy; conversation visible state	DSQL action/confirmation/capability/submission /result; label follow-up is a separate substate, not a second return.
Damaged replacement	Evidence metadata in PostgreSQL, image version in clean S3, human case/approval if required	DSQL exact item/quantity/address/effect and provider reservation; unknown blocks duplicate.
Refund	Policy amount/threshold + workflow + human approval projection	DSQL exact amount/currency/destination, confirmation, approval, capability, payment submission and reconciliation.
Cancellation	Subscription read facts + preview + policy + workflow intent	DSQL effective date/effect/capability/submission. Existing scheduled cancellation is reconciled, not duplicated.
Human escalation	Temporal pause + Human case/queue/assignment + Conversation control mode	No action capability for handoff itself. Representative later actions receive their own DSQL record if consequential.

24.1 Release-version recording

A1 is implemented by conversation.turn_executions and safety.action_intents. The active conversation pins agent/workflow compatibility, while every turn/action records the actual prompt bundle, resolved model route, knowledge release, policy decision/version, tool contract, guardrail, and evaluation version used. Knowledge may advance between turns; a turn never mixes releases.

24.2 Action amendments

Amendment	Part 3 enforcement
A4 Action Capability	Opaque ref hash, canonical claims hash, one redemption count, expiry, exact action/version/epoch in DSQL
A5 Action Safety Store	Aurora DSQL strong transactions, 64-shard fence, provider idempotency, unknown-result ledger
A6 Revocation cache	Revocation epoch in PostgreSQL release, Redis key, DynamoDB snapshot, and OpenSearch authorization filter
A10 Recovery gate	DSQL FENCING/BLOCKED modes, all-action reconciliation query, activation transaction before routing publish

25. Failure behavior

Failure	Required storage behavior	Forbidden conclusion
Aurora writer unavailable	Stop relational writes; buffer nothing unbounded; retry idempotent command after recovery	External action did not occur
Aurora unplanned regional failover	Restore consistent relational state; replay outbox/deletion; consult DSQL/provider for actions	Secondary contains every last write
DSQL 40001 conflict	Retry full deterministic transaction with bounded jitter and fresh reads	Prior transaction definitely failed before commit
DSQL peer unavailable	Use healthy peer endpoint only when routing/workload policy permits; strong database remains one logical state	Bypass fence or create local ledger
DynamoDB conflict/stale replica	Compare source version; rebuild projection	Projection is newer business truth
Redis unavailable	Cache miss/degraded quota/fail closed according to operation	Cached allow or action state may be invented
S3 scan/object mismatch	Quarantine exact version; block handle; incident on authoritative original loss	Latest key version is automatically trusted
OpenSearch unavailable	Use permitted revocation-checked evidence cache or truthful handoff	General model knowledge substitutes for tenant policy
Kafka duplicate/outage	Inbox dedup; outbox retry; admission control	Exactly-once infrastructure removes idempotency need
Deletion target failure	PARTIAL state and retry/incident	User request is fully complete

26. Security, correctness, and recovery tests

26.1 Schema and RLS gates

- Enumerate catalog tables and require tenant_id/environment_id, scoped primary/unique/foreign constraints, FORCE RLS, policy, and scoped indexes.
- Connect as each application role and prove it cannot SET ROLE to owner, bypass RLS, access another schema, install extension, copy/truncate, or read system secrets.
- Mutation testing removes one tenant predicate, one RLS clause, one cache-key dimension, one object prefix, and one OpenSearch filter; the isolation suite must fail each mutation.
- Property test any two distinct tenants: no valid context from tenant B can observe, count, time-infer, mutate, export, restore, or delete tenant A data.

26.2 Action safety gates

- One hundred concurrent redemption attempts for the same capability produce exactly one SUBMISSION_RESERVED record and redemption_count=1.
- Concurrent action transition and 64-shard route fence: either the action commits before the fence and is reconciled, or it aborts/fails the new epoch; no stale commit after fence activation.
- Crash after reservation but before provider call resumes with the same action ID and provider idempotency key.
- Timeout after possible submission becomes RESULT_UNKNOWN; repeated customer messages, worker replay, and failover do not create a second provider request identity.
- Changed preview, policy, approval, verification, action arguments, amount, destination, workflow version, or route invalidates the old capability hash.
- Restored DSQL backup cannot activate fence shards until provider reconciliation and deletion-ledger application pass.
- Agent Runtime image/network/role tests prove no DSQL client, schema grant, Action Capability type, connector write library, or credential is present.

26.3 Storage conformance matrix

Suite	Required proof
PostgreSQL	Transactions, CAS, unique idempotency, RLS default deny, composite FK, deadlock/serialization retry
DSQL	40001 retry, fence conflict, action state invariants, multi-peer consistency, backup restore block
DynamoDB	No Scan, exact key grammar, conditional source version, cross-tenant GSI absence, rebuild
Redis	Typed key dimensions, TTL upper bounds, eviction/loss, secret scanner, quota isolation
S3	Allocated key only, checksum/version seal, scan quarantine, signed URL expiry, Object Lock, all-version deletion
OpenSearch	Strict mapping, pre-score filters, cross-tenant canaries, revoked source, release rollback
ClickHouse	Pseudonymization, forbidden-field scan, TTL, correction events, no transactional dependency
Outbox/inbox	Crash/retry/duplicate/reorder, aggregate sequence, payload handle classification
Deletion/restore	Hold, partial target, backup age-out, deletion-ledger replay, tombstone evidence

27. Implementation modules and delivery sequence

27.1 Repository additions

```
contracts/storage/v1/
  identifiers.proto  object-handle.proto  retention.proto
  action-safety.proto  deletion.proto

packages/platform/
  scoped-transaction/ tenant-key-builders/ envelope-crypto/
  object-handles/ retention-policy/ migration-runtime/

deployables//migrations/postgresql/
deployables/integration-gateway/migrations/dsql/

infra/terraform/modules/
  aurora-postgresql/ aurora-dsql-action-safety/
  dynamodb-projections/ redis/ s3-data-classes/
  opensearch-knowledge/ clickhouse/

tests/
  storage-conformance/ rls/ isolation/ action-safety/
  migration/ deletion/ restore/ reconciliation/
```

27.2 Delivery sequence

Step	Deliverable	Release proof
1	Identifier, TenantScope, object-handle, retention, and migration contracts	Cross-language fixtures; no arbitrary tenant/key construction
2	Aurora clusters, roles, TLS, RLS template, scoped transaction wrapper	Two-tenant repository negative suite
3	Tenant/environment and identity directory schemas	Membership epoch/revocation/session tests
4	Conversation/message/payload/outbox/inbox vertical slice	Idempotent order-tracking message and validated response commit
5	Temporal projections, Policy decisions, Human case/assignment	Takeover suppresses automated response
6	S3 quarantine/evidence/knowledge object lifecycle	Checksum, scan, sealed version, access and deletion
7	Knowledge metadata and OpenSearch generation	ACL/revocation/citation/canary suite
8	Aurora DSQL Action Safety schema and 64-shard fence	Concurrent redeem/failover/provider-unknown chaos suite
9	DynamoDB/Redis/ClickHouse derived paths	Loss/rebuild and forbidden-content tests
10	Retention, deletion, backup, restore and reconciliation	Full two-tenant restore and deletion-ledger exercise
11	All seven journey storage traces	No duplicate effect, false completion, cross-tenant record, or stale-epoch action

27.3 Required ADRs

ADR	Decision
ADR-017	Aurora PostgreSQL 17.7 LTS, pooled tenant schemas, composite scope keys, FORCE RLS, and non-owner roles
ADR-018	Application UUIDv7 identifiers, aggregate sequence/version, canonical JSON hashing
ADR-019	Conversation payload envelope encryption and S3 large-object threshold
ADR-020	Transactional outbox/inbox and at-least-once consumer idempotency
ADR-021	Aurora DSQL Action Safety Store, regional peer cohorts, and 64-shard transactional fencing
ADR-022	DynamoDB projection-only global tables and prohibited authority
ADR-023	Redis typed key grammar, TTL maxima, quotas, and loss behavior
ADR-024	S3 bucket classes, version sealing, Object Lock, retention, legal hold, and deletion
ADR-025	OpenSearch immutable generation, strict mapping, pre-score authorization, and rebuild
ADR-026	Retention/deletion ledger, backup restore admission, and downstream erasure evidence

28. Acceptance and freeze decision

28.1 Acceptance checklist

Result	Criterion
PASS	Every persisted data class has exactly one authoritative owner and a declared derived-store relationship.
PASS	Every pooled tenant row/key/object/document and relationship is tenant-and-environment scoped.
PASS	Aurora PostgreSQL repository predicates, composite constraints, FORCE RLS, and non-owner roles compose as defense in depth.
PASS	Conversation sequencing, idempotency, expected-version commit, outbox, and inbox have exact atomic boundaries.
PASS	Identity, membership, sessions, verification, epochs, decision audit, and key metadata satisfy Part 2.
PASS	Consequential confirmation, approval, capability, submission, and reconciliation use a strongly consistent DSQL safety domain.
PASS	The 64-shard fence makes regional route changes conflict with every in-flight action transaction.
PASS	DynamoDB, Redis, OpenSearch, ClickHouse, and workflow projections cannot become action or business truth.
PASS	S3 paths, versions, scanning, encryption, retention, legal hold, and exact-version deletion are defined.
PASS	Migration, backup, restore, deletion replay, and reconciliation preserve all hard invariants.
PASS	All seven V1 journeys and amendments A1-A10 have concrete storage enforcement.

28.2 Decisions frozen in Part 3

- Aurora PostgreSQL 17.7 LTS is the V1 relational baseline; schema SQL remains forward-compatible with managed patch upgrades.
- Aurora DSQL is the V1 Action Safety Store. It is not a general platform database.
- The Action Safety Store uses a 64-shard transactional route fence and one exact state machine for capability, submission, unknown result, and reconciliation.
- Application UUIDv7, tenant/environment composite relationships, record versions, and canonical hashes are mandatory conventions.
- RLS is mandatory on pooled tenant PostgreSQL tables; normal application roles cannot own tables or bypass RLS.
- Conversation content is envelope-encrypted; attachments and large payloads are versioned S3 objects behind opaque handles.
- DynamoDB stores only projections; Redis stores only disposable state; OpenSearch and ClickHouse are derived and rebuildable.
- Outbox/inbox, migration, retention, deletion, backup, restore, audit, and reconciliation contracts are frozen.
- No unresolved persistence decision blocks Part 4.

PART 3 FROZEN Database and storage schemas are complete enough for APIs, events, workflows, tools, agent runtime, knowledge/retrieval, and production-validation LLDs to inherit without developer interpretation. The next design is LLD Part 4 - Conversation, Admin, and Real-time APIs.

References

[I1] Customer Agent OS Lite - V1 Product Contract, July 2026.

[I2] Customer Agent OS Lite - Integrated HLD Validation and Architecture Freeze v1.1, July 21, 2026.

[I3] Customer Agent OS Lite - LLD Part 1: Codebase and Deployable Structure v1.0, July 21, 2026.

[I4] Customer Agent OS Lite - LLD Part 2: Identity and TenantContext v1.0, July 23, 2026.

[P1] PostgreSQL 18 - Row Security Policies. https://www.postgresql.org/docs/current/ddl-rowsecurity.html

[P2] PostgreSQL 18 - Constraints. https://www.postgresql.org/docs/current/ddl-constraints.html

[A1] AWS - Aurora PostgreSQL LTS releases; PostgreSQL 17.7 released March 25, 2026.
https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/AuroraPostgreSQL.Updates.LTS.html

[A2] AWS - Switchover or failover in Aurora Global Database; unplanned failover can lose unreplicated writes.
https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/aurora-global-database-disaster-recovery.html

[A3] AWS - Aurora backups and point-in-time restore.
https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/Aurora.Managing.Backups.html

[D1] AWS - Amazon Aurora DSQL: strongly consistent, active-active distributed SQL. https://aws.amazon.com/rds/aurora/dsql/

[D2] AWS - Aurora DSQL multi-Region clusters: two peer endpoints and a witness Region.
https://docs.aws.amazon.com/aurora-dsql/latest/userguide/multi-region-aws-cli.html

[D3] AWS - Aurora DSQL optimistic concurrency control and SQLSTATE 40001.
https://docs.aws.amazon.com/aurora-dsql/latest/userguide/working-with-concurrency-control.html

[D4] AWS - Aurora DSQL quotas: 3,000 mutated rows and 10 MiB per transaction.
https://docs.aws.amazon.com/aurora-dsql/latest/userguide/CHAP_quotas.html

[D5] AWS - Aurora DSQL CREATE VIEW; row-level security is not currently supported.
https://docs.aws.amazon.com/aurora-dsql/latest/userguide/create-view.html

[DB1] AWS - DynamoDB Global Tables design: MRSC lacks transaction APIs and requires exactly three Regions.
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-global-table-design.html

[S1] AWS - S3 Object Lock, retention, legal hold, and Versioning requirement.
https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html

[S2] AWS - S3 presigned URLs provide time-limited upload/download access.
https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html

[O1] OpenSearch - Hybrid search filtering; pre-filtering removes documents before scoring.
https://docs.opensearch.org/latest/vector-search/ai-search/hybrid-search/index/

[O2] OpenSearch - Efficient k-NN filtering. https://docs.opensearch.org/latest/vector-search/filter-search-knn/efficient-knn-filtering/

[U1] RFC 9562 - Universally Unique IDentifiers, including UUID Version 7. https://www.rfc-editor.org/rfc/rfc9562.html

AUTHORITY If Part 3 conflicts with the V1 Product Contract, Integrated HLD Freeze, or LLD Part 2 identity invariants, the higher-precedence frozen document controls and Part 3 must be corrected. Part 3 specializes storage implementation; it cannot weaken authority, isolation, confirmation, action-safety, validated-delivery, or regional-fencing boundaries.

KLEEM AI / LOW-LEVEL DESIGN

LLD Part 4

Conversation, Admin, Internal, and Real-time APIs

Exact interaction boundaries for customer chat, support operations, configuration, internal service calls, and validated real-time updates.

STATUS	**PART 4 CONCEPTUAL BASELINE**
CORE DECISION	Use resource-oriented REST at the edge, typed HTTP/gRPC internally, asynchronous acceptance for AI work, and SSE for server-to-browser updates.
CORE INVARIANT	No client-provided TenantContext, no duplicate mutation after retry, and no unvalidated model content is delivered to a customer.
VERSION	1.0 - July 24, 2026

DESIGN RULE

The model may interpret, retrieve, and propose. Trusted services own identity, policy, state transitions, authorization, execution, reconciliation, and final truth.

1. Part 4 decision and scope

Part 4 defines the doors through which people, browsers, administrators, and services communicate. It freezes API ownership, interaction semantics, authentication boundaries, idempotency behavior, error contracts, and the real-time event model.

Interface class	Primary consumer	Transport	Owner
Conversation API	Customer web chat and human workspace	HTTPS REST	Conversation Runtime
Real-time stream	Customer web chat and human workspace	SSE; WebSocket only for justified bidirectional features	Conversation Runtime
Admin API	Tenant administrator and operations manager	HTTPS REST	Control and Knowledge
Workflow command API	Trusted services	Typed HTTP/gRPC plus Temporal signal	Workflow Workers
Internal service API	Cell-local workloads	mTLS gRPC or typed HTTP	Owning release unit

1.1 Trusted request context

External clients send authentication credentials, resource identifiers, an idempotency key when required, and ordinary request data. The Edge validates the external identity and obtains a server-issued TrustedTenantContextV1. Clients never submit tenant authority, roles, policy decisions, routing epoch, or workload identity as trusted fields.

```
POST /v1/conversations/{conversationId}/messages
Authorization: Bearer <external access token>
Idempotency-Key: msg-client-456
Content-Type: application/json

{
  "content": {"type": "text", "text": "Where is my order?"},
  "clientMessageId": "browser-456"
}
```

1.2 Standard response envelope

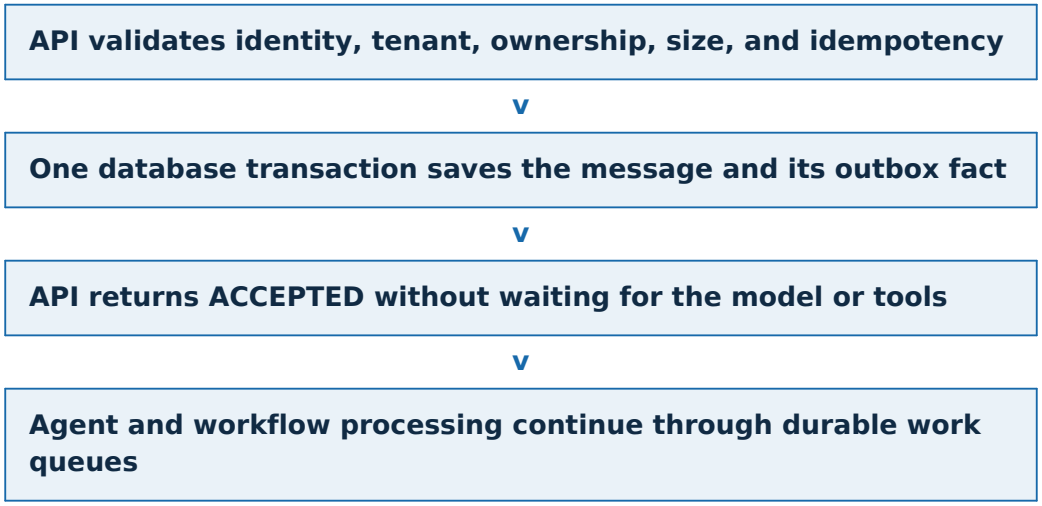
```
{
  "data": {
    "messageId": "msg_789",
    "conversationId": "con_123",
    "status": "ACCEPTED"
  },
  "meta": {
    "requestId": "req_456",
    "apiVersion": "2026-07-24"
  }
}
```

The public response exposes stable product identifiers and status, not internal database keys, topology, provider credentials, raw policy input, or internal tokens.

2. Conversation API

Operation	Purpose	Important rule
POST /v1/conversations	Create a conversation pinned to an agent release	Idempotent; server chooses tenant, environment, release, and home cell
POST /v1/conversations/{id}/messages	Accept a customer or representative message	Return after durable commit; AI work continues asynchronously
GET /v1/conversations/{id}	Read summary, participants, status, and active journey	Owner and tenant checks at the Conversation Runtime
GET /v1/conversations/{id}/messages	Page through committed messages	Opaque cursor; stable ordering by aggregate sequence
POST /v1/conversations/{id}/cancel	Request cancellation of cancellable processing	Does not imply an external business action was reversed
POST /v1/conversations/{id}/handoff	Request human support	Starts or signals the handoff workflow

2.1 Asynchronous acceptance



A successful message response means the message was durably accepted. It does not mean an AI answer, refund, return, replacement, or provider operation has completed.

2.2 Idempotency

Input	Behavior
Same tenant, endpoint, idempotency key, and same canonical request	Return the original result
Same key but different canonical request	Reject with IDEMPOTENCY_CONFLICT
No key on a required mutation	Reject before business processing
Expired key	Treat according to the documented endpoint retention window; never silently duplicate a consequential action

Idempotency keys are scoped by tenant, environment, operation, and caller class. The server stores the canonical request hash, result reference, state, and expiry.

3. Real-time SSE protocol

The browser opens a resumable SSE connection. SSE is the default because most live traffic is server to browser. WebSocket is reserved for a feature that truly requires continuous bidirectional communication.

```
GET /v1/conversations/con_123/stream
Accept: text/event-stream
Last-Event-ID: stream-event-900

event: progress.updated
id: stream-event-901
data: {"code":"CHECKING_ORDER","text":"Checking your order..."}

event: response.completed
id: stream-event-902
data: {"messageId":"msg_900","delivery":"chunked-approved-content"}
```

3.1 Event catalog

Event	Meaning	Customer content rule
progress.updated	Deterministic progress from trusted workflow state	Safe template only
response.completed	Validated assistant message is committed	Approved content may be delivered in chunks
confirmation.required	Exact preview awaits customer confirmation	Contains preview ID and display-safe facts
approval.pending	A human approval is required	No private approver details
handoff.updated	Human queue, assignment, or takeover changed	Only customer-visible status
journey.updated	Durable journey state changed	Never claims completion before authoritative confirmation
processing.failed	Current work could not safely complete	Truthful retry or handoff guidance

VALIDATED-ONLY DELIVERY

V1 does not stream raw model tokens. It may stream deterministic progress. Generated content is grounded, safety-checked, committed, and only then delivered through SSE.

3.2 Resume and connection behavior

- Each stream event has a monotonically increasing conversation-local event identifier.
- The client reconnects with Last-Event-ID; the server replays retained events or sends a resync-required instruction.
- Heartbeats keep intermediaries from silently closing idle streams.
- SSE is a delivery channel, not source truth; clients can always reload committed conversation state.
- Per-tenant and per-principal connection limits protect the cell during reconnect storms.

4. Confirmation, approval, and workflow APIs

Operation	Effect
POST /v1/previews/{previewId}/confirm	Confirms the exact immutable preview; sends a signal to the owning workflow
POST /v1/previews/{previewId}/reject	Rejects or cancels the proposed action
POST /v1/approvals/{approvalId}/decisions	Records approve, reject, or bounded modification by an authorized workforce principal
GET /v1/workflows/{workflowId}	Returns a customer-safe or workforce-safe projection, never raw Temporal history

Confirmation never carries an editable amount or target. It references the immutable preview. The workflow revalidates expiry, state, actor, policy version, routing epoch, and any required approval before requesting an Action Capability.

5. Admin and release APIs

Resource	Representative operations	Lifecycle
Agents	Create draft, update behavior, validate	DRAFT -> VALIDATED -> PUBLISHED -> RETIRED
Knowledge sources	Register, sync, inspect, revoke	QUARANTINED -> INDEXED_DRAFT -> PUBLISHED -> REVOKED
Integrations	Register connector, test health, approve tools	DRAFT -> VERIFIED -> ENABLED -> SUSPENDED
Policies	Create bundle, test, publish	DRAFT -> TESTED -> PUBLISHED -> SUPERSEDED
Releases	Assemble, evaluate, canary, promote, rollback	CANDIDATE -> APPROVED -> CANARY -> ACTIVE -> ROLLED_BACK

Draft and published resources are separate. A configuration edit cannot change production behavior until validation, authorization, evaluation, and release promotion succeed. Long-running workflows stay pinned to compatible versions.

6. Internal contracts

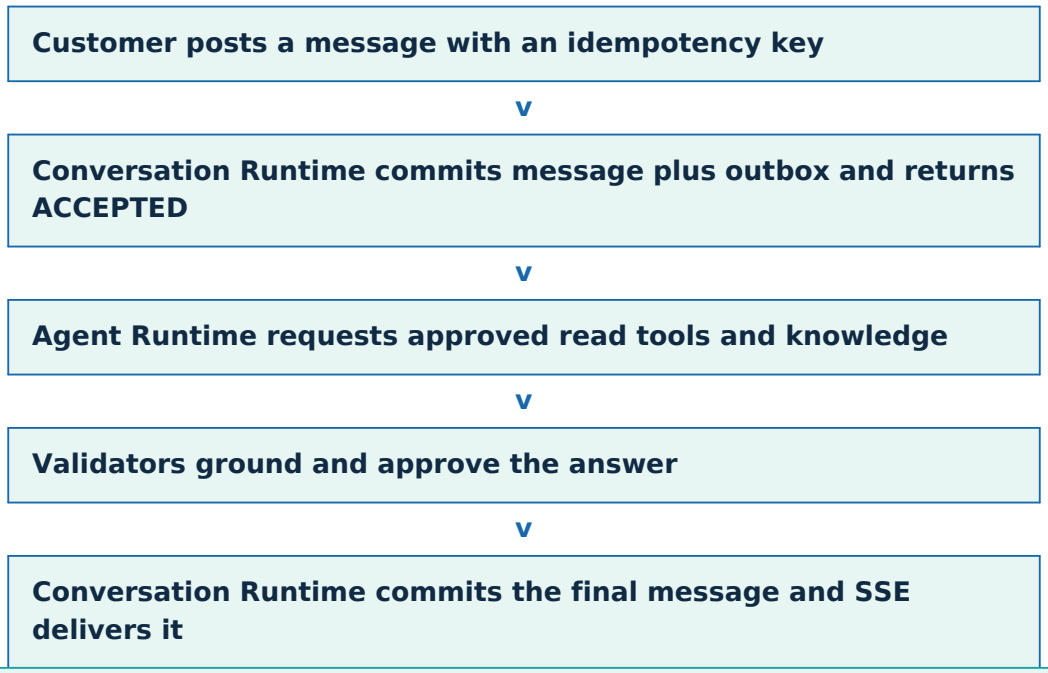
Internal communication uses generated clients from versioned schemas. Every request carries a narrow internal context assertion plus mTLS workload identity. The receiving owner verifies audience, caller, tenant, route, expiry, purpose, and current routing epoch.

Contract concern	Rule
Timeout	Short and explicit; no infinite request waits
Retry	Only retry safe reads or idempotent mutations
Authorization	Admission at Edge and enforcement again by the resource owner
Versioning	Backward-compatible additive changes within a version; breaking change creates a new version
Pagination	Opaque signed cursor bound to tenant, filter, sort, and release
Errors	Stable machine code, safe message, retryability, and request ID
Payload size	Bounded inline data; large content uses controlled object handles

6.1 Stable error model

```
{
  "error": {
    "code": "CONFIRMATION_EXPIRED",
    "message": "This preview has expired. Please request a new preview.",
    "retryable": false,
    "requestId": "req_789"
  }
}
```

7. End-to-end order question



PART 4 SUMMARY

Requests enter through typed, authenticated, idempotent APIs. Live updates leave through resumable SSE. Durable state remains authoritative, and generated content is delivered only after validation.

KLEEM AI / LOW-LEVEL DESIGN

LLD Part 5

Kafka Events and Asynchronous Communication

Versioned facts, transactional outbox and inbox processing, partitioning, retries, dead letters, replay, and privacy-safe event design.

STATUS	**PART 5 CONCEPTUAL BASELINE**
CORE DECISION	Services publish facts after committing owned state. Kafka decouples consumers; it does not replace service ownership, databases, or workflows.
CORE INVARIANT	A committed business change eventually emits its fact, and every consumer remains correct when the same event is delivered more than once.
VERSION	1.0 - July 24, 2026

DESIGN RULE

The model may interpret, retrieve, and propose. Trusted services own identity, policy, state transitions, authorization, execution, reconciliation, and final truth.

1. Part 5 decision

Kafka carries durable facts between independently owned components. A command asks an owner to do something; an event states that an owner has already committed something. Commands use typed APIs or Temporal. Facts use Kafka.

Type	Example	Transport	Authority
Command	Start refund workflow	Typed API or Temporal start	Receiver decides and owns transition
Signal	Customer confirmed preview	Temporal signal	Running workflow validates and applies
Event	conversation.message.received.v1	Kafka	Publisher already committed the fact
Projection update	Refresh analytics or search view	Kafka consumer	Derived and rebuildable

1.1 Representative topic families

Topic family	Representative events
conversation	message.received, response.completed, conversation.closed
workflow	journey.started, preview.ready, confirmation.recorded, journey.completed
action	capability.issued, submission.started, result.unknown, action.succeeded
handoff	handoff.requested, case.created, representative.took_over
knowledge	source.synced, release.published, document.revoked
control	agent.release.published, policy.published, routing.changed
audit/analytics	Privacy-safe derived facts; never the only source of a security audit record

2. Canonical event envelope

```
{
  "eventId": "evt_123",
  "eventType": "conversation.message.received.v1",
  "occurredAt": "2026-07-24T16:30:00Z",
  "producer": "conversation-runtime",
  "tenantId": "tenant_10",
  "environmentId": "prod",
  "aggregateType": "conversation",
  "aggregateId": "con_123",
  "aggregateSequence": 18,
  "routingEpoch": 82,
  "traceId": "trace_456",
  "schemaVersion": 1,
  "payload": {"messageId": "msg_789"}
}
```

Field	Purpose
eventId	Globally unique duplicate-detection identifier
eventType/schemaVersion	Stable consumer contract and explicit evolution
tenant/environment	Mandatory scope; never inferred from payload text
aggregateId/sequence	Per-aggregate ordering and gap detection
routingEpoch	Reject or quarantine stale routed work
traceId	Correlate asynchronous processing without exposing credentials

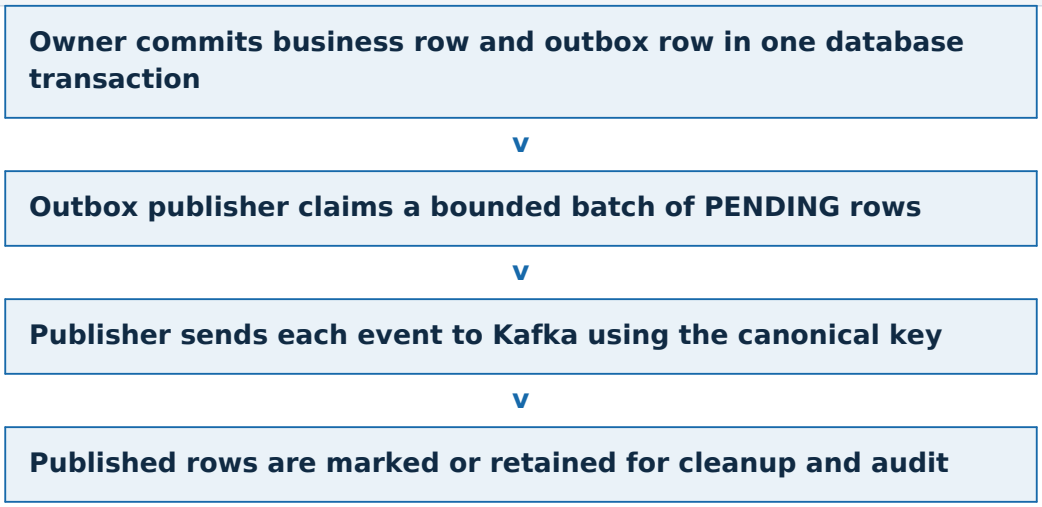
Field	Purpose
payload	Safe references and minimum required facts; avoid raw private conversation content

3. Transactional outbox

The outbox solves the dual-write problem between an owner's database and Kafka. Saving a message and later publishing an event as unrelated operations can leave the system in a contradictory state.

Without outbox	Failure
Save message, then publish	Process crashes after save; the AI never receives message.received
Publish, then save message	Consumer receives an event for data that does not exist
Assume one distributed transaction	PostgreSQL and Kafka do not share a simple application transaction

```
BEGIN;
  INSERT INTO conversation.message (...);
  INSERT INTO events.outbox (
    event_id, tenant_id, environment_id, event_type,
    aggregate_id, aggregate_sequence, payload, status
  ) VALUES (..., 'PENDING');
COMMIT;
```



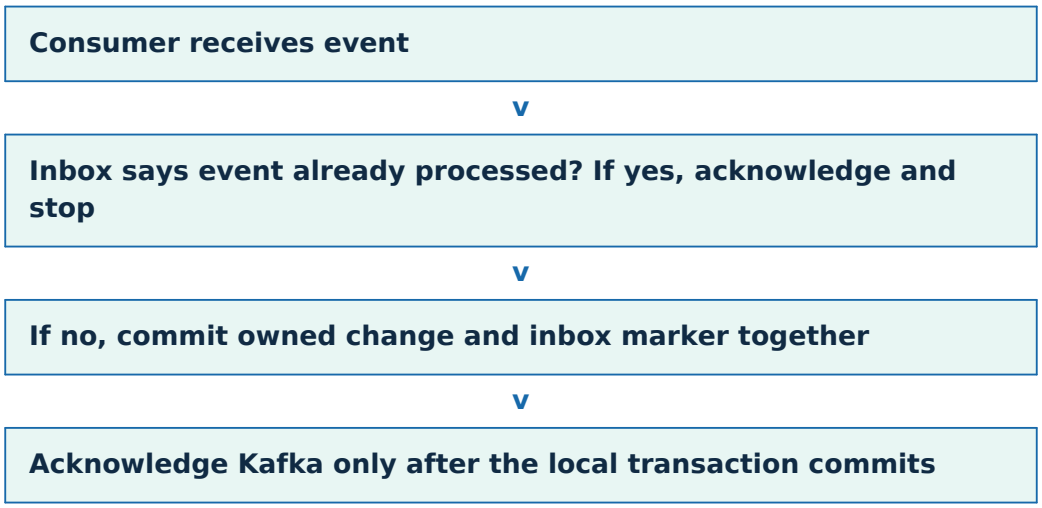
3.1 Publish-crash duplicate

A publisher can send an event successfully and crash before recording PUBLISHED. It will send the event again. This is expected. The design promises at-least-once delivery, not exactly-once business effects.

4. Consumer inbox and idempotency

```
BEGIN;
  SELECT 1 FROM events.inbox
    WHERE consumer = 'agent-runtime' AND event_id = 'evt_123';

  -- If absent, apply the consumer-owned mutation.
  INSERT INTO agent.turn_task (...);
  INSERT INTO events.inbox (consumer, event_id, processed_at) VALUES (...);
COMMIT;
```



The inbox makes duplicate delivery harmless for that consumer. External actions still require the stronger Action Capability, action ledger, provider idempotency, and reconciliation defined in Parts 6 and 7.

5. Partitioning and ordering

Event family	Partition key	Ordering guarantee
Conversation	tenantId + environmentId + conversationId	Message and response facts ordered within one conversation
Workflow	tenantId + environmentId + workflowId	State-transition facts ordered within one workflow
Action	tenantId + environmentId + actionId	Submission and reconciliation facts ordered within one action
Control release	tenantId + environmentId + release resource	Release facts ordered for one resource

The platform does not require global ordering across every tenant or conversation. Consumers use aggregateSequence to reject stale data, detect gaps, and trigger a source-of-truth resync when needed.

6. Retry, dead letters, and replay

Failure class	Handling
Transient dependency failure	Retry with exponential backoff and jitter
Consumer bug or incompatible payload	Stop after bounded attempts; send to dead-letter topic
Authorization or tenant mismatch	Do not retry blindly; quarantine, alert, and investigate
Stale routing epoch	Reject or route to controlled recovery
Poison event	Isolate so one event cannot block its partition indefinitely
Replay request	Authorized operator selects bounded event range and compatible consumer version

6.1 Dead-letter record

- Original event reference and schema version
- Consumer name and deployed release
- Failure class and sanitized error code

- Attempt count and first/last failure time
- Trace, tenant, environment, and aggregate references
- No credential, raw token, hidden model reasoning, or unnecessary private body

Replay is a protected operational action. It is never a generic 'reprocess everything' button. The operator previews scope, validates compatibility, records authority, and observes duplicate-safe results.

7. Schema evolution

Change	Policy
Add optional field	Allowed within the version when old consumers ignore it safely
Change meaning, type, or required field	Publish a new event version
Remove old version	Only after consumer inventory proves migration and retention expires
Consumer deployment	Must declare supported event versions and fail clearly on unsupported input

```
conversation.message.received.v1
conversation.message.received.v2

V1 and V2 may coexist while consumers migrate.
```

8. Security, privacy, and operations

- Kafka authorization is per workload and topic family; network access alone grants nothing.
- Payloads contain stable references and minimum facts. Large or private content stays in the owning store.
- Encryption protects brokers and traffic, but application authorization still applies.
- Lag, retry rate, dead-letter growth, event age, outbox age, and duplicate rate are measured per cell.
- Backpressure protects workflow progress and reconciliation before analytics or batch consumers.
- Kafka retention supports transport and replay; it is not the product's permanent authoritative database.

PART 5 SUMMARY

The outbox prevents lost facts, the inbox makes duplicate delivery safe, partition keys preserve local order, and DLQs isolate failures. Kafka announces facts; it does not own business truth.

KLEEM AI / LOW-LEVEL DESIGN

LLD Part 6

Temporal Workflows and State Machines

Durable multi-step journeys, explicit state machines, activities, signals, timers, approvals, unknown-result reconciliation, and safe workflow evolution.

STATUS	**PART 6 CONCEPTUAL BASELINE**
CORE DECISION	Use a separate bounded Temporal workflow for each V1 journey and shared child workflows/activities for confirmation, evidence, approval, handoff, and reconciliation.
CORE INVARIANT	A journey survives process failure, waits durably, executes each consequential effect through the action-safety chain, and never reports completion before authoritative confirmation.
VERSION	1.0 - July 24, 2026

DESIGN RULE

The model may interpret, retrieve, and propose. Trusted services own identity, policy, state transitions, authorization, execution, reconciliation, and final truth.

1. Part 6 decision

Temporal owns durable coordination for work that spans multiple steps, waits for people or providers, retries, survives crashes, or must reconcile uncertain external results. Kafka carries facts; Temporal owns the in-progress journey.

Use Temporal when	Do not use Temporal as
The journey waits minutes, hours, or days	The conversation message database
A provider call needs retry or reconciliation	A general event bus
Customer confirmation or human approval is required	A replacement for policy or authorization
The process must resume after worker failure	A place for model credentials or arbitrary private payloads
State transitions and timeouts must be explicit	One enormous universal workflow

2. Seven V1 journey workflows

Journey	Workflow responsibility	Consequential effect
Order tracking/investigation	Resolve identity, order, shipment, exceptions, and handoff	Usually none; read-only
Return eligibility	Collect facts and evaluate current deterministic policy	None until return creation
Return creation	Preview items, method, label, fees, and confirmation	Create return
Damaged-item replacement	Collect evidence, check inventory, approval, and confirmation	Create replacement
Refund	Eligibility, exact preview, confirmation, approval, submission, verification	Issue refund
Subscription cancellation	Resolve subscription, terms, preview, confirmation, provider result	Cancel subscription
Human escalation/handoff	Create case, choose queue, summarize, wait for takeover, enforce ownership	Human assignment/takeover

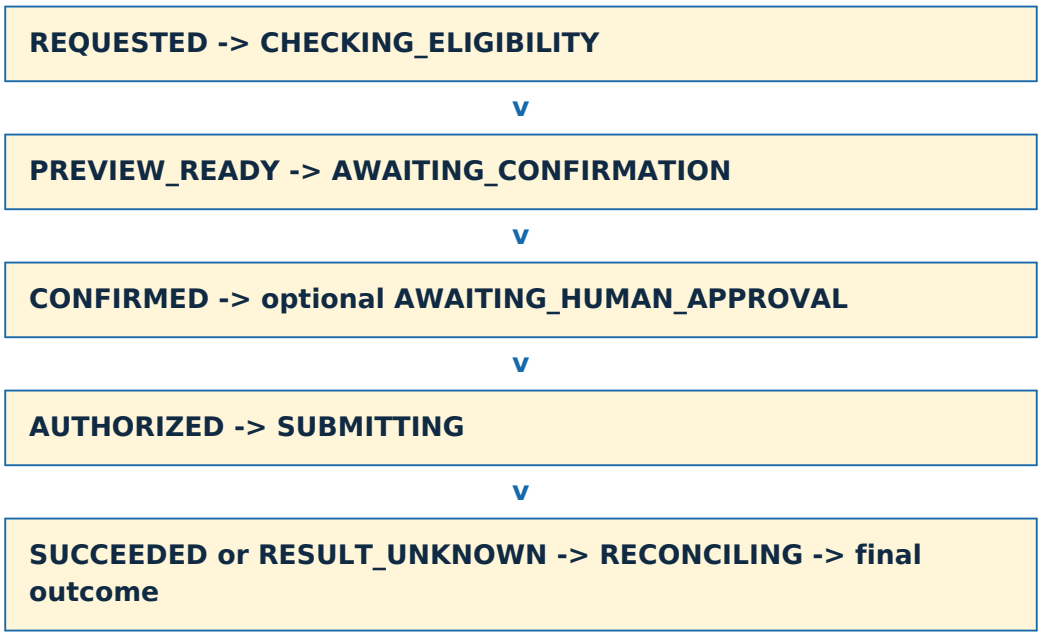
Address change remains a bounded subworkflow rather than an eighth journey. Evidence collection, approval, notification, and reconciliation are reusable child workflows or activities.

3. Universal journey state contract

State class	Representative states
Intake	REQUESTED, COLLECTING_INFORMATION
Decision	CHECKING_ELIGIBILITY, DENIED, NEEDS_REVIEW
Preview	PREVIEW_READY, AWAITING_CONFIRMATION, EXPIRED
Approval	AWAITING_HUMAN_APPROVAL, APPROVED, REJECTED
Execution	AUTHORIZED, SUBMITTING, RESULT_UNKNOWN, RECONCILING
Outcome	SUCCEEDED, CANCELLED, HANDED_OFF, FAILED_FINAL

Each journey defines its own allowed transitions. Shared state names have shared semantics, but a journey cannot jump to a state merely because the name exists.

4. Refund workflow reference



4.1 Exact preview binding

The preview is immutable and contains the target, items, amount, currency, method, fees, policy version, expiry, and display text. Customer confirmation references the preview ID and hash. Editing any material field creates a new preview and requires new confirmation.

4.2 Safe waiting

```
State: AWAITING_CONFIRMATION
Timer: 24 hours

Signal customer_confirmed(previewId, confirmationId)
  -> validate actor, preview hash, expiry, and workflow state
  -> transition to CONFIRMED

Timer fires first
  -> transition to EXPIRED
```

5. Workflow code versus activity code

Workflow code	Activity code
Controls state and allowed transitions	Calls PostgreSQL through owner APIs
Starts timers and waits for signals	Queries policy, retrieval, or business systems
Chooses retry and compensation path	Invokes the Integration Gateway
Must be deterministic and replay-safe	May perform nondeterministic I/O
Stores compact references and decisions	Returns bounded, serializable results

DETERMINISM

Workflow code does not call databases, models, clocks, random generators, or provider APIs directly. Those operations are activities with explicit timeouts and retry policy.

6. Signals, queries, timers, and updates

Mechanism	Use	Example
Signal	Deliver asynchronous information to a running workflow	customer_confirmed, human_approved, provider_callback
Query	Read a safe workflow projection without changing state	current customer-safe status
Update	Validated synchronous state change when a result is required	bounded administrative correction
Timer	Durable deadline or delay	expire preview after 24 hours

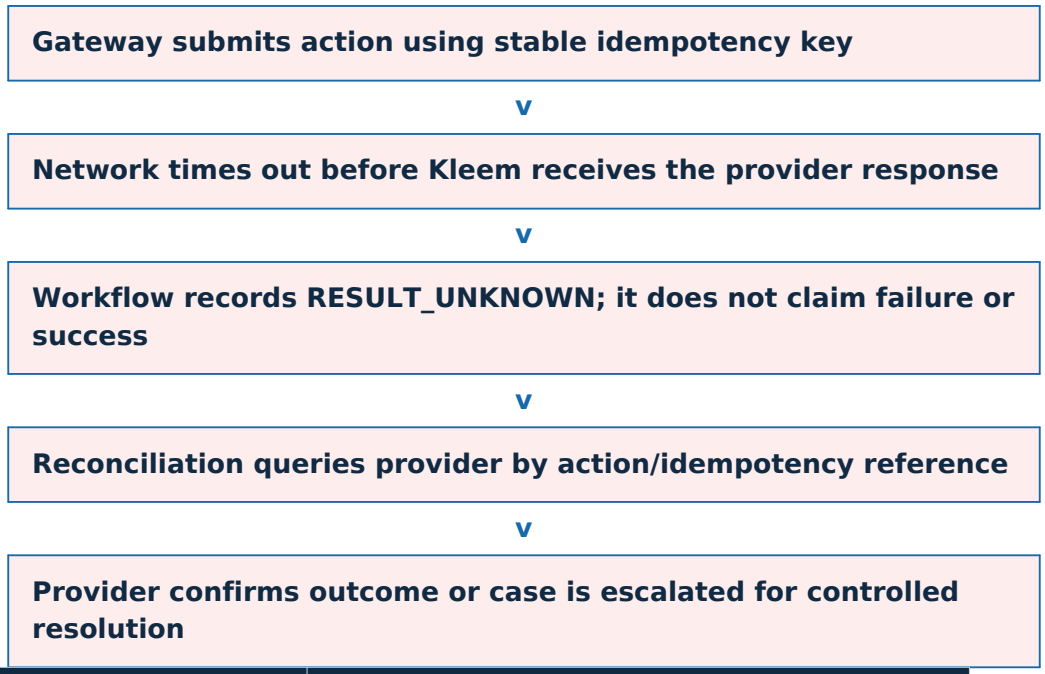
Every signal includes a unique signal ID and expected workflow/reference identifiers. Duplicate signals return the original outcome. A stale or invalid signal is recorded and rejected without changing state.

7. Activities, timeouts, and retries

Activity type	Timeout/retry rule
Read-only internal query	Short timeout; bounded retry with jitter
Knowledge/model request	Budgeted timeout; approved fallback or safe failure
Provider read	Provider-specific timeout and rate-limit handling
Consequential submission	Stable action ID and provider idempotency; do not blindly retry an unknown result
Notification	Retry independently; notification failure does not rewrite business truth

An activity retry policy is chosen by side-effect semantics. A generic 'retry three times' policy is unsafe for external writes.

8. Unknown result and reconciliation



Reconciliation result	Transition
Provider confirms effect exists	SUCCEEDED

Reconciliation result	Transition
Provider confirms request was never accepted	Retry through the same authorized action path when still valid
Provider reports terminal rejection	FAILED_FINAL with truthful customer message
Provider cannot determine result	Remain pending, alert, and hand off; never duplicate to meet a timeout

9. Human approval and takeover

Policy can pause a workflow for human approval. The Human Operations service creates an approval task with the exact preview and reason. The workflow accepts only a signed, authorized decision signal. A representative takeover changes conversation ownership and disables autonomous customer-facing progression until release back to automation.

```
Refund amount > tenant threshold
  -> AWAITING_HUMAN_APPROVAL
  -> approval task created
  -> approver chooses APPROVE or REJECT
  -> workflow verifies authority and exact preview
  -> continue or terminate
```

10. Workflow evolution and operations

- Workflow ID is derived from tenant, environment, journey type, and stable journey identifier.
- New workflow code remains replay-compatible or uses Temporal versioning/worker routing.
- Long-running workflows stay pinned to compatible code and release identifiers.
- Payloads contain references to private content, not large raw documents or credentials.
- Search attributes contain safe operational dimensions for queues and dashboards.
- Operators can pause, retry an activity, repair a bounded state, or terminate only through audited runbooks.
- Every workflow transition emits a privacy-safe fact through an owner outbox when other services need it.

PART 6 SUMMARY

Temporal remembers where a journey is, waits safely, applies explicit transitions, retries according to side-effect semantics, and reconciles uncertain outcomes without risking a duplicate action.

KLEEM AI / LOW-LEVEL DESIGN

LLD Part 7

Tools, MCP, Connectors, and Action Capabilities

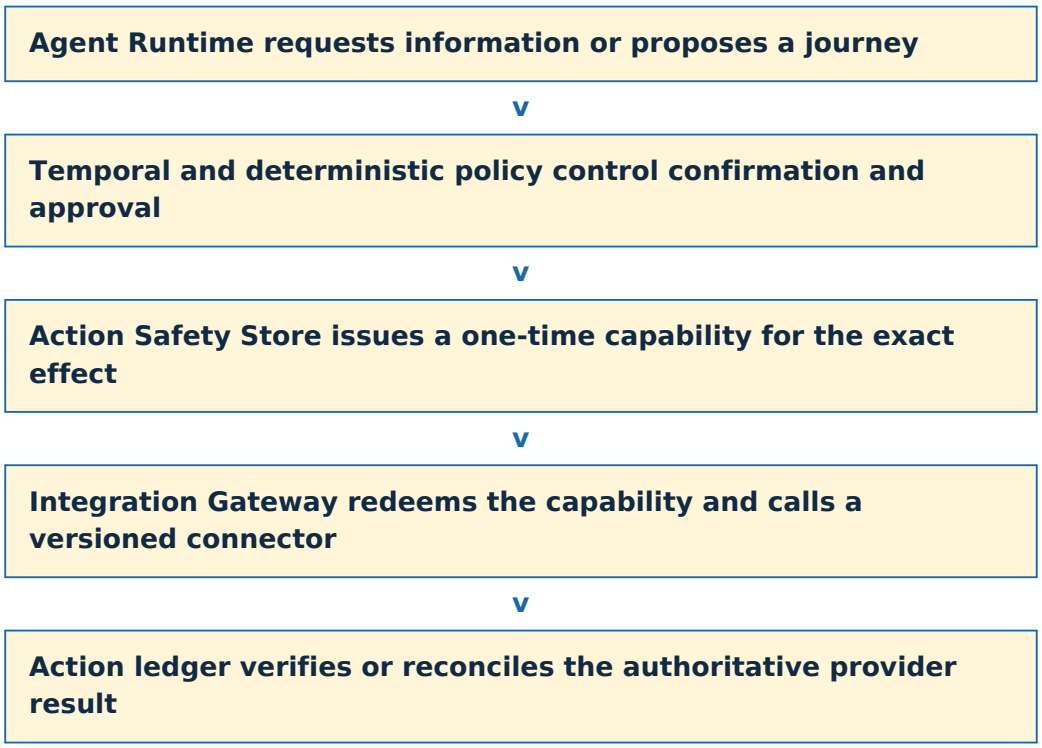
Controlled external-system access, versioned tool contracts, connector isolation, credential handling, one-time action authorization, action ledger, and reconciliation.

STATUS	**PART 7 CONCEPTUAL BASELINE**
CORE DECISION	The Integration Gateway is the only runtime path to external business systems. MCP standardizes tool contracts; it never grants business authority.
CORE INVARIANT	A consequential action executes only when a short-lived Action Capability binds one exact confirmed and approved effect, and the action ledger can prove or reconcile its result.
VERSION	1.0 - July 24, 2026

DESIGN RULE

The model may interpret, retrieve, and propose. Trusted services own identity, policy, state transitions, authorization, execution, reconciliation, and final truth.

1. Part 7 decision



The model never receives provider credentials, broad write permissions, or Action Capabilities. It cannot directly invoke commerce, CRM, shipping, inventory, subscription, or payment APIs.

2. Tool, connector, and gateway boundaries

Concept	Meaning	Example
Tool contract	Provider-neutral operation schema and policy metadata	get_order:v1
Connector	Provider adapter implementing one or more tool contracts	Shopify get_order adapter
Integration Gateway	Trusted service that authorizes, limits, executes, sanitizes, and records calls	Cell-local gateway
MCP	Standard discovery/invocation description for approved tools	Registered MCP server

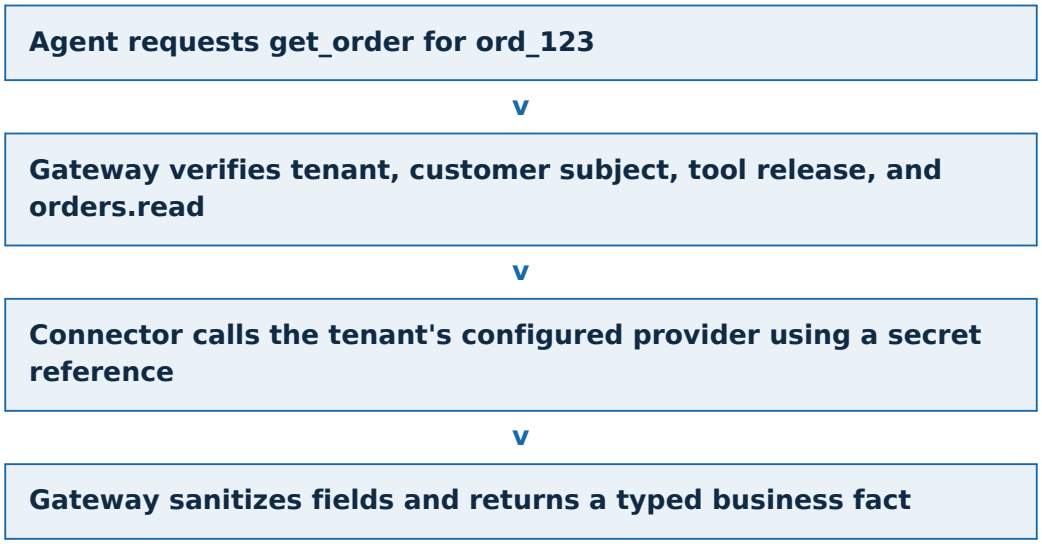
```
{
  "name": "get_order",
  "version": "1.0",
  "operationType": "READ",
  "requiredPermission": "orders.read",
  "inputSchema": {"orderId": "string"},
  "outputSchema": {
    "status": "string",
    "estimatedDelivery": "string"
  },
  "timeoutMs": 3000,
  "dataClass": "CUSTOMER_CONFIDENTIAL"
}
```

3. Read tools versus consequential tools

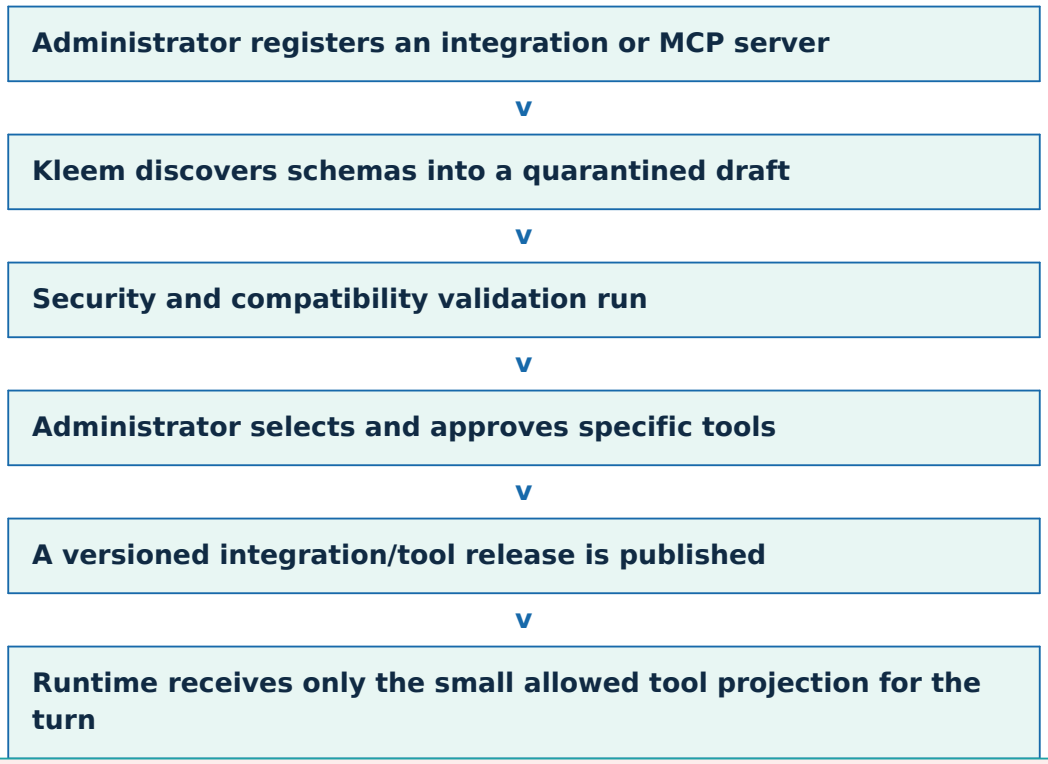
Read tool	Consequential operation
get_order	create_return
get_shipment_status	issue_refund
check_inventory	create_replacement
get_subscription	cancel_subscription
get_refund_status	change_shipping_address

An approved read tool can be projected into the Agent Runtime for the current turn, but the Gateway still checks tenant, subject ownership, purpose, permission, field policy, rate limit, and provider health. Consequential operations are never model tools. They are activities inside a deterministic Temporal journey.

3.1 Read authorization example



4. MCP registration and release



MCP BOUNDARY

MCP describes how a tool can be called. It does not authenticate the customer, grant permission, satisfy policy, confirm an action, or authorize a business effect.

Dynamic production discovery is prohibited. A newly connected server cannot expose arbitrary tools directly to a live agent.

5. Connector SDK and credential isolation

SDK responsibility	Rule
Contract mapping	Translate provider-neutral input/output without changing business meaning
Authentication	Resolve a tenant secret reference at execution time; never return credentials
Timeout/retry	Provider and operation-specific; write retries require stable idempotency
Rate limiting	Per tenant, provider, connector, and operation
Sanitization	Return only declared output fields and safe error classes
Telemetry	Record latency, result class, action/read reference, and provider request ID when safe
Testing	Contract tests, sandbox tests, fault injection, duplicate submission, and unknown-result tests

Credentials live in a managed secret system and are exposed only to the connector execution boundary. The Agent Runtime, prompt, Kafka event, Temporal history, logs, traces, and analytics never contain them.

6. Action Capability - simple meaning

DEFINITION

An Action Capability is a short-lived, single-use permission ticket for one exact confirmed action.

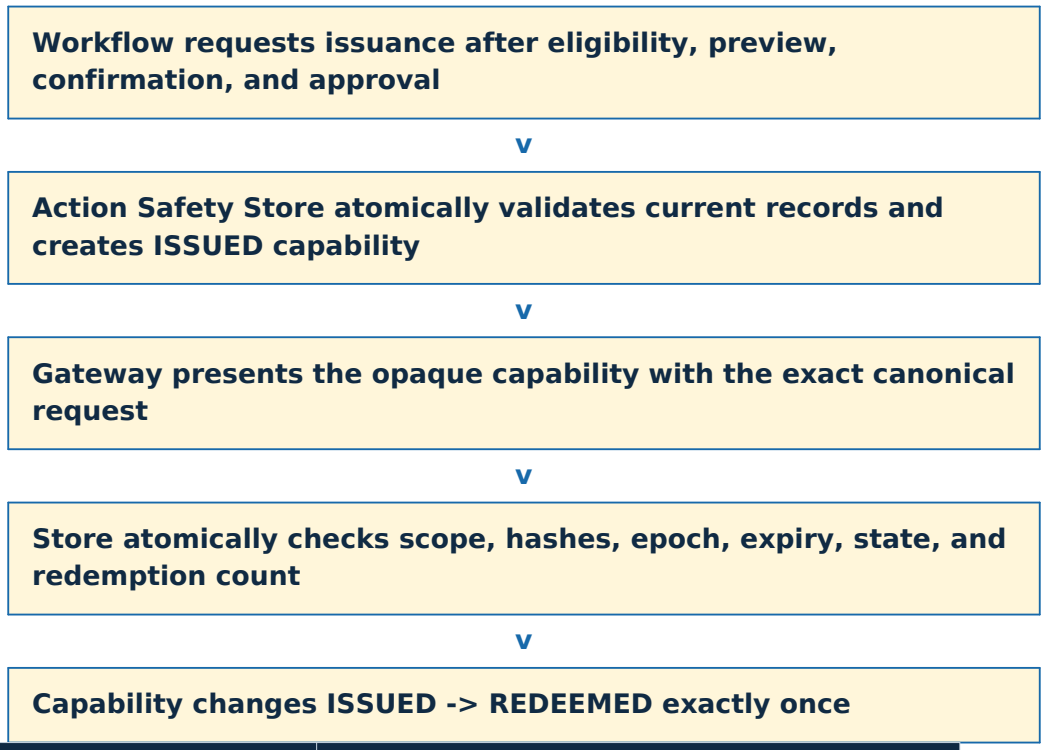
A normal role might say a service can issue refunds. That is too broad for an AI-driven system. The capability says: refund exactly $49.99 for order 123, using this method, under this policy and confirmation, one time, before this expiry.

It binds	Why
Tenant and environment	Cannot cross customer organizations or environments
Workflow and action type	Cannot be moved to another journey or operation
Target and canonical argument hash	Amount, currency, item, method, and recipient cannot be edited
Preview and confirmation	The executed action is the one the customer saw and accepted
Approval and policy version	Required human authority and deterministic decision remain attributable
Routing epoch	Old or split-brain regions cannot execute after fencing
Idempotency key and expiry	One intended effect within a short authorized window

```
Model never sees: cap_7f92b28...

Capability meaning:
  tenant: tenant_10
  workflow: wf_500
  action: ISSUE_REFUND
  target: order_123
  exact arguments: hash(...)
  preview: hash(...)
  confirmation: conf_901
  approval: approval_44 (when required)
  routing epoch: 82
  idempotency key: refund-order123-item2
  expires: 2026-07-24T21:10:00Z
```

7. Capability issuance and redemption

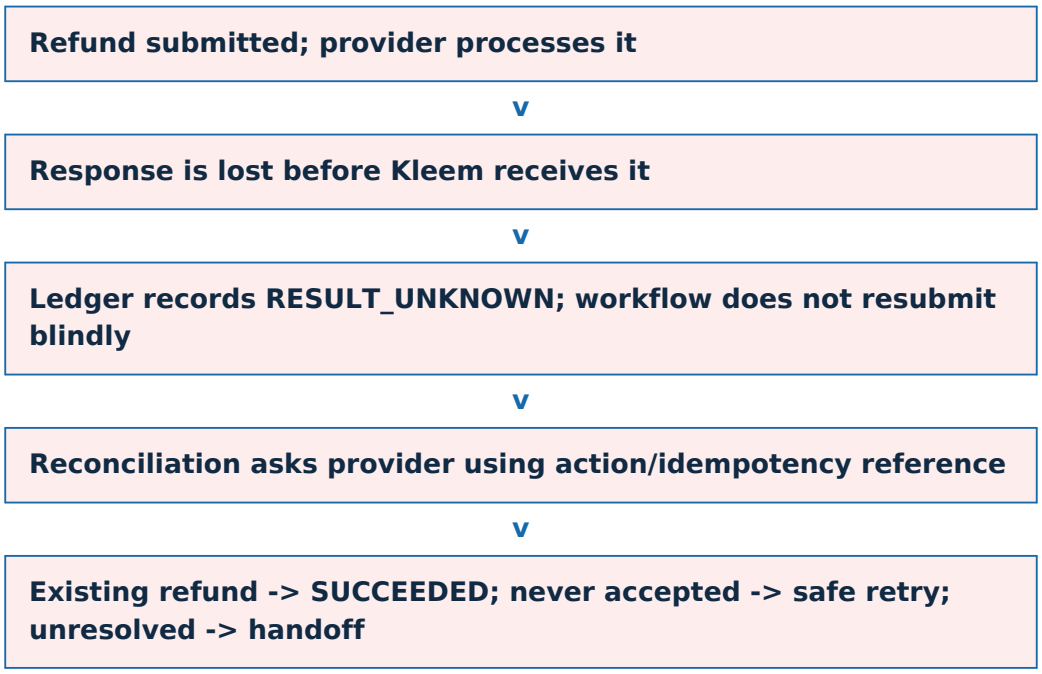


Attempt	Result
Change $49.99 to $499.99	Argument hash mismatch; reject
Use for another order	Target mismatch; reject
Use after route failover	Routing epoch stale; reject
Use after expiry	Expired; reject and require new preview/confirmation
Redeem twice	Already redeemed; return existing action reference, never authorize a second effect

Capability redemption protects authorization. It does not by itself prove the external provider produced one effect. The action ledger and provider idempotency complete that proof.

8. Action ledger and reconciliation

Ledger state	Meaning
AUTHORIZED	Exact action was approved and capability issued
REDEEMED	Gateway consumed the capability
SUBMISSION_STARTED	Provider call began with stable idempotency key
PROVIDER_CONFIRMED	Provider returned authoritative acceptance/result
RESULT_UNKNOWN	Network or provider ambiguity prevents a truthful answer
RECONCILING	System is querying provider or callback evidence
SUCCEEDED / FAILED	Final authoritative outcome recorded



The architecture does not claim infrastructure-level exactly-once delivery. It targets one intended business effect through one-time authorization, stable idempotency, a durable ledger, authoritative verification, and reconciliation.

9. Failure and security matrix

Failure or attack	Safe behavior
Prompt injection asks to call hidden tool	No unapproved tool exists in the turn projection
Connector unavailable	Circuit opens; workflow remains pending or hands off
Provider rate limit	Respect retry-after; preserve journey state
Credential revoked	Suspend integration and fail closed
Tool output contains instructions	Treat as untrusted data; sanitize and validate
Provider output schema changes	Contract validation fails; do not invent fields
Capability store unavailable	No consequential action executes
Result ambiguous	RESULT_UNKNOWN and reconciliation; never false completion

PART 7 SUMMARY

MCP describes approved tools, connectors translate provider APIs, the Gateway controls execution, and an Action Capability authorizes one exact effect once. The ledger and reconciliation prove what actually happened.

KLEEM AI / LOW-LEVEL DESIGN

LLD Part 8

Agent Runtime and Bounded Specialists

One customer-visible agent, one orchestrator, five bounded specialist modules, typed context, structured outputs, budgets, validation, release pinning, and safe fallbacks.

STATUS	**PART 8 CONCEPTUAL BASELINE**
CORE DECISION	Use one Python/FastAPI Agent Runtime that calls centralized Knowledge and Model gateways and coordinates fixed specialist modules. No autonomous internal swarm and no A2A between modules.
CORE INVARIANT	The Agent Runtime interprets and proposes but never owns durable business state, credentials, policy authority, consequential tools, confirmations, approvals, or final provider truth.
VERSION	1.0 - July 24, 2026

DESIGN RULE

The model may interpret, retrieve, and propose. Trusted services own identity, policy, state transitions, authorization, execution, reconciliation, and final truth.

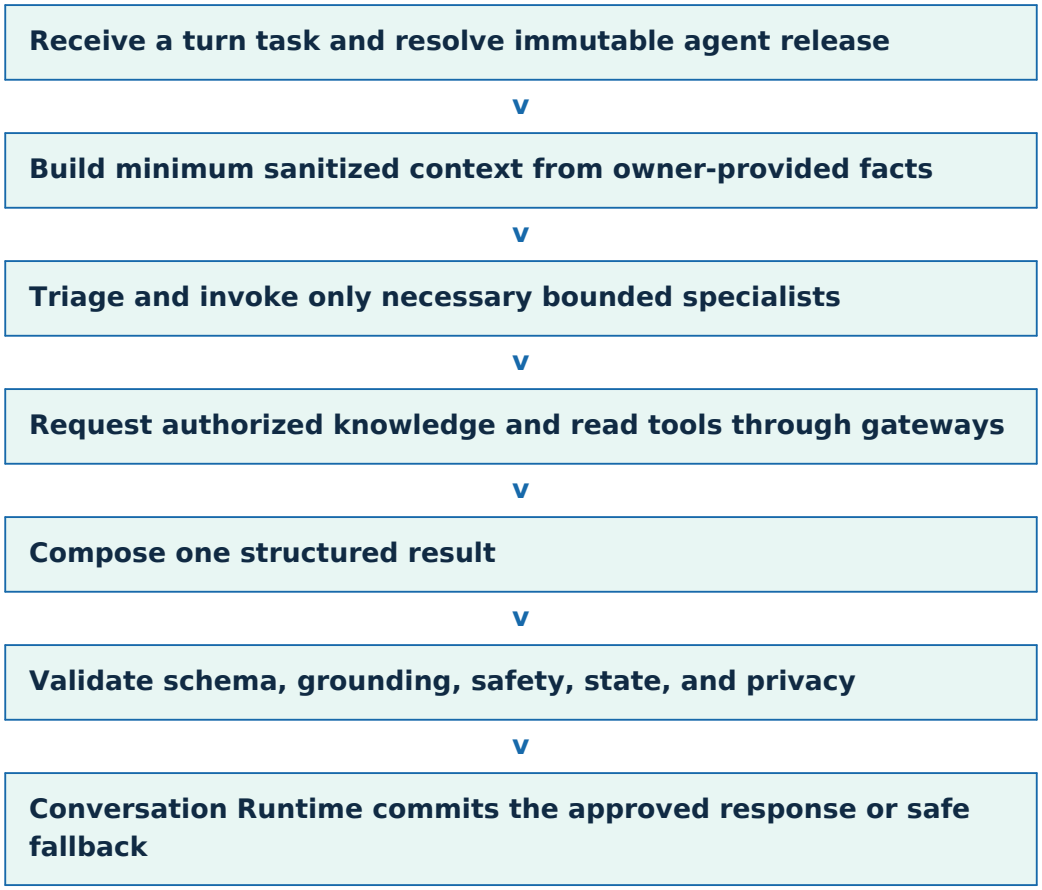
1. Part 8 decision

The customer experiences one coherent support agent. Internally, an orchestrator invokes a small set of typed specialist modules when they add value. They are logical modules within one deployable runtime, not independently autonomous agents.

Specialist	Responsibility	Cannot do
Triage	Intent, entities, urgency, possible journey	Execute tools or decide policy
Knowledge	Request and summarize authorized evidence	Invent policy or bypass ACL
Customer Operations	Choose approved read tools and interpret typed results	Execute refund, return, replacement, or cancellation
Risk and Policy	Identify risks and questions for deterministic policy	Authorize or override policy
Handoff	Draft summary and propose escalation	Assign itself or continue after human takeover

Each module has a typed request/response, fixed role, call and token budget, no recursive delegation, no credentials, no private durable memory, and no write authority.

2. Turn-processing pipeline



The Agent Runtime does not own the conversation database. It receives a turn-specific task and returns a typed result. The Conversation Runtime remains the owner of committed messages and customer-visible delivery.

3. Agent context contract

```
{
  "turn": {"turnId":"turn_77","locale":"en-US","purpose":"CUSTOMER_SUPPORT"},
  "conversation": {
    "recentMessages": [],
    "approvedSummary": {},
    "humanOwnership": "AUTOMATION"
  },
  "subject": {
    "customerReference":"customer_42",
    "verificationClass":"VERIFIED"
  },
  "journey": {"type":"ORDER_TRACKING","state":"CHECKING"},
  "businessFacts": [],
  "knowledgeEvidence": [],
  "allowedReadTools": ["get_order:v1"],
  "responseContract": {"schema":"AgentTurnResultV1"},
  "budgets": {"modelCalls":3,"readToolCalls":2,"deadlineMs":8000}
}
```

Included	Excluded
Recent messages and approved summary	Authentication tokens and provider credentials
Verified subject reference and journey state	Raw roles, OPA internals, or broad permissions
Authoritative business facts and evidence IDs	Action Capabilities
Small allowed read-tool projection	Unrelated customer or cross-tenant data
Budgets and response schema	Arbitrary hidden memory or raw chain-of-thought

Tenant and authorization enforcement live in trusted code. Prompt instructions are behavior guidance, never an authorization boundary.

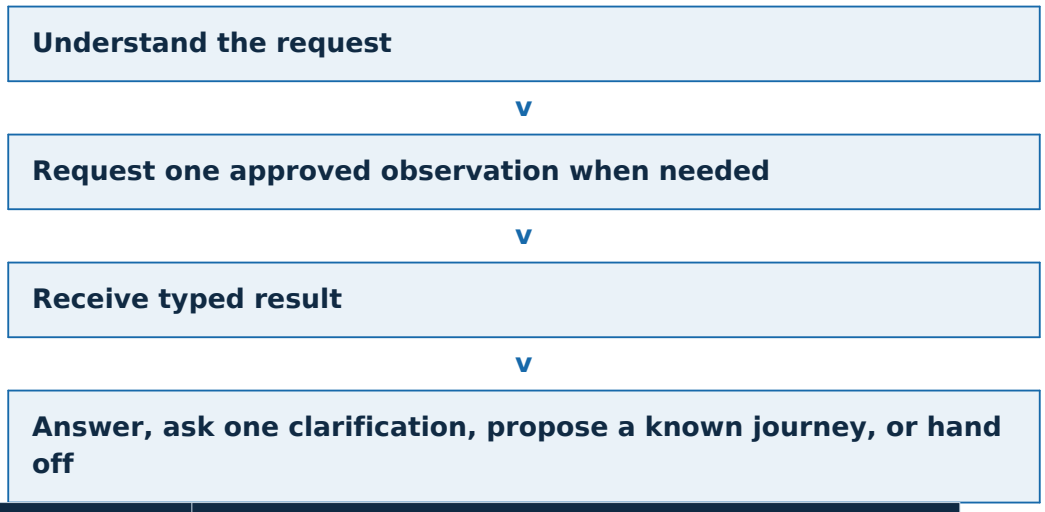
4. Structured output contract

Result type	Meaning
ANSWER	Grounded response supported by evidence and/or business facts
ASK_CLARIFICATION	One targeted question for required missing information
READ_TOOL_REQUEST	Request one approved read observation with typed arguments
JOURNEY_PROPOSAL	Propose a known workflow and collected fields
HANDOFF_PROPOSAL	Propose escalation with a draft structured summary
UNABLE_TO_ANSWER_SAFELY	No grounded, authorized, and safe result is available

```
{
  "resultType": "JOURNEY_PROPOSAL",
  "journeyType": "CREATE_RETURN",
  "collectedFields": {
    "orderId": "ord_123",
    "itemId": "item_2",
    "reason": "damaged"
  },
  "missingFields": ["evidence"],
  "evidenceIds": ["ev_51"]
}
```

Pydantic/JSON Schema validation rejects unexpected fields, malformed types, and unsupported journey/tool names. A valid proposal still does not start a business action automatically; the workflow owner validates current state and deterministic policy.

5. Bounded reasoning and budgets



Budget	Purpose when exceeded
Model calls	Prevent recursive reasoning and provider amplification
Specialist calls	Keep topology finite and observable
Read-tool calls	Protect business providers and cost
Retrieved passages	Limit context dilution and private-data exposure
Tokens and dollars	Control tenant economics
Wall-clock deadline	Protect customer latency and cell capacity

Exact numeric budgets are tuned by evaluations and load tests. Exceeding a budget produces a deterministic fallback, targeted clarification, or handoff - never an unbounded agent loop. Consequential journeys use Temporal plan-and-execute.

6. Prompt and release model

Prompt bundle component	Purpose
Platform safety	Non-overridable interaction and data rules
Tenant behavior	Approved brand, tone, locale, and support boundaries
Specialist instruction	Fixed responsibility and refusal conditions
Tool-use rules	Only approved read tools and declared schemas
Knowledge rules	Treat retrieval as evidence, not authority
Output schema	Machine-checkable result contract
Examples	Expected decisions, failures, and handoff behavior

Every execution records the agent release, prompt version, model route, knowledge release, tool-contract version, policy version when relevant, and evaluation version. This supports reproducibility without storing hidden chain-of-thought.

7. Response validation and customer delivery

Validator	Question
Schema	Does the response match AgentTurnResultV1?
Grounding	Does each material claim point to evidence or authoritative business fact?
State truth	Does wording match the current workflow/provider status?
Privacy	Could the response expose prohibited personal or tenant data?
Tool fidelity	Are tool results represented without alteration or unsupported inference?
Prompt injection	Did untrusted content attempt to change rules, tools, or authority?
Rendering safety	Is customer-visible content safely escaped and size-bounded?

DELIVERY CORRECTION

Raw model tokens are not streamed to the customer. Deterministic progress may stream immediately. The generated answer is grounded and validated first, then the approved content may be delivered in chunks.

8. Failure and fallback behavior

Condition	Behavior
Malformed output	One bounded structured repair; then safe error or handoff
Low confidence or missing fact	Ask one targeted clarification
Primary model unavailable	Use only a pre-approved compatible fallback
No approved fallback	Provide deterministic status and offer human support
Tool fact conflicts with model	Authoritative tool/business fact wins
Knowledge conflicts with policy	Deterministic policy and workflow state win; request review
Human took over	Agent stops customer-facing autonomous progression

8.1 Why no A2A in V1

The five specialists share one product owner, codebase, runtime, trust boundary, and release. A2A would add identity, discovery, delegation, and failure complexity without a real independent-agent boundary. A2A remains a future option for separately operated agents with explicit trust contracts.

PART 8 SUMMARY

One orchestrator uses fixed specialists to interpret and propose. Typed context, budgets, schemas, validators, release pinning, and safe fallbacks keep the AI useful without giving it durable authority.

KLEEM AI / LOW-LEVEL DESIGN

LLD Part 9

Knowledge, Retrieval, Memory, and Model Gateway

Controlled knowledge ingestion, tenant-first hybrid retrieval, evidence and citations, revocation, explicit memory ownership, and approved model routing.

STATUS	**PART 9 CONCEPTUAL BASELINE**
CORE DECISION	Publish immutable knowledge releases; retrieve under trusted tenant/ACL scope using hybrid search and reranking; route all inference through a centralized Model Gateway.
CORE INVARIANT	Knowledge provides authorized evidence, memory provides controlled state, and models provide inference. None can grant permission or control a consequential business action.
VERSION	1.0 - July 24, 2026

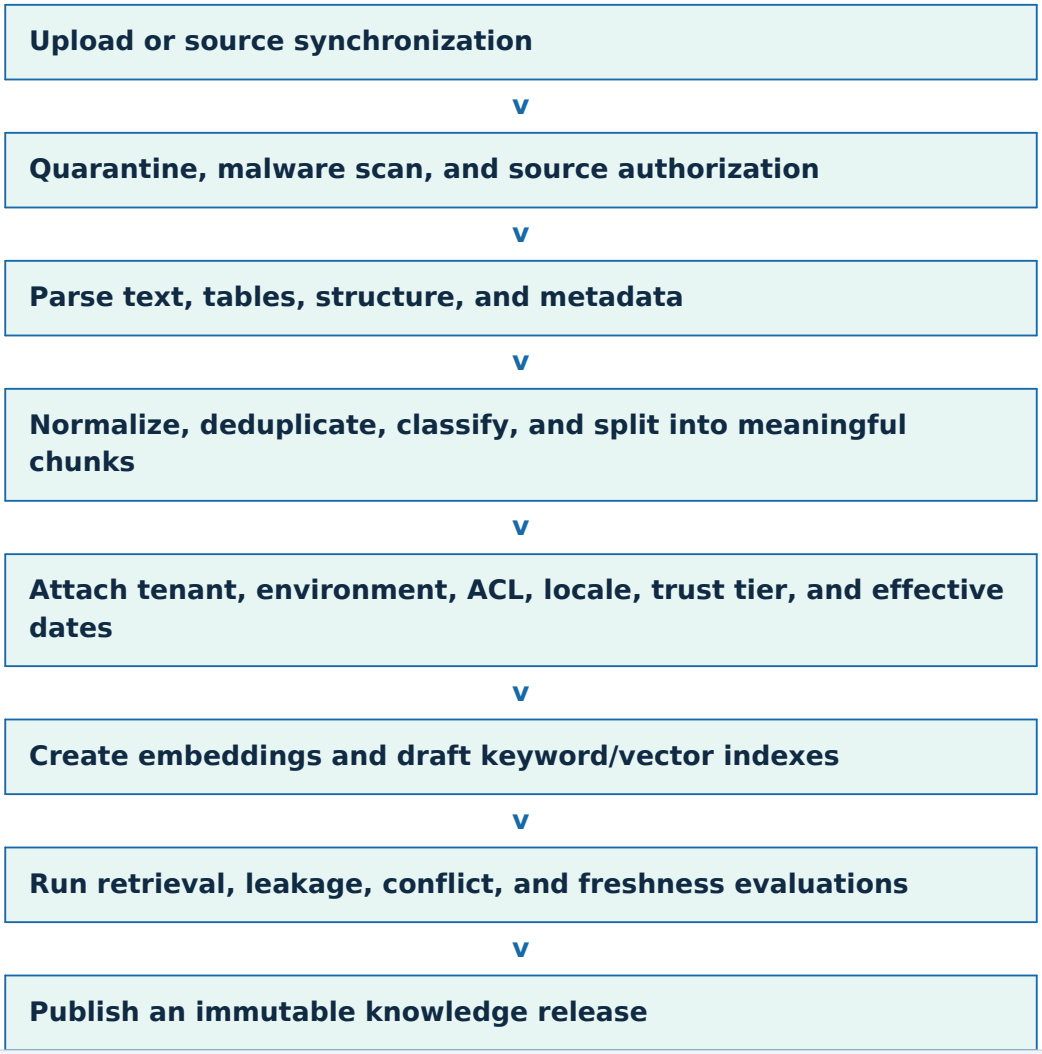
DESIGN RULE

The model may interpret, retrieve, and propose. Trusted services own identity, policy, state transitions, authorization, execution, reconciliation, and final truth.

1. Part 9 decision

A document upload, vector similarity score, remembered summary, or model output is not automatically true or authorized. The platform separates the lifecycle of knowledge, retrieval evidence, memory state, and model execution.

2. Knowledge ingestion and publication




```
{
  "tenantId": "tenant_10",
  "environmentId": "prod",
  "sourceId": "refund-policy",
  "documentVersion": "17",
  "effectiveFrom": "2026-07-01",
  "effectiveTo": null,
  "locale": "en-US",
  "acl": ["support-agent", "customer-safe"],
  "trustTier": "AUTHORITATIVE_POLICY",
  "knowledgeReleaseId": "kr_88"
}
```

Draft ingestion never changes production retrieval. A release promotion atomically selects a tested immutable generation. Revocation remains independently enforceable when an unsafe document must be removed immediately.

3. Trusted retrieval scope

Scope input	Source
Tenant and environment	TrustedTenantContextV1
Knowledge release	Pinned agent/release configuration
Revocation epoch	Knowledge authority
Purpose and principal class	Server-derived request context
Document ACL	Knowledge metadata plus authorization decision
Locale and effective time	Validated request and trusted clock
Query	Sanitized customer question plus structured intent

ISOLATION RULE

Kleem never searches every tenant and asks the model to ignore the wrong documents. Tenant, environment, release, and ACL filters apply before protected content can reach the Agent Runtime.

4. Hybrid retrieval pipeline

Apply tenant, environment, release, ACL, locale, and effective-date filters

v

Run keyword/BM25 search for exact terms and identifiers

v

Run vector search for semantic similarity

v

Combine rankings with a deterministic fusion method

v

Rerank the bounded candidate set

v

Remove duplicates and detect expired or conflicting sources

v

Return a small evidence pack with stable evidence IDs

Search mode	Strength	Example
Keyword	Exact strings, policy numbers, SKU/order IDs	Policy 17.4; SKU-88931
Vector	Paraphrases and semantic meaning	Send back a broken item -> damaged merchandise return

Search mode	Strength	Example
Reranker	Pairwise relevance on a small candidate set	Choose the best policy passage for the exact question

A vector database is a retrieval component, not universal search, permanent memory, authorization, or source truth.

5. Evidence packs and citations

```
{
  "evidenceId": "ev_900",
  "sourceId": "refund-policy",
  "documentVersion": "17",
  "chunkId": "chunk_42",
  "effectiveFrom": "2026-07-01",
  "trustTier": "AUTHORITATIVE_POLICY",
  "text": "Damaged-item evidence is required above $75."
}
```

Evidence condition	Agent behavior
Current, authorized, and unconflicted	Use for grounded claim and retain evidence ID
Missing	State it cannot verify; ask clarification, use business read, or hand off
Expired or revoked	Do not use
Two authoritative sources conflict	Do not choose by similarity; escalate or apply declared precedence
Low-trust source conflicts with authoritative policy	Authoritative policy wins
High similarity but wrong scope	Reject; similarity is not proof of truth or authorization

The response validator maps material claims to evidence IDs or authoritative business facts. Citation records preserve source version and effective time so the system can explain which release supported the answer.

6. Freshness, revocation, and caching

```
Retrieval cache key =
  tenant + environment + knowledge release + ACL fingerprint
  + locale + normalized query + effective-time bucket
  + revocation epoch
```

When a document is revoked, the authority increments the tenant/environment revocation epoch. Old cache entries no longer match. If current revocation state cannot be confirmed for protected guidance, retrieval fails closed or hands off rather than using possibly unsafe policy.

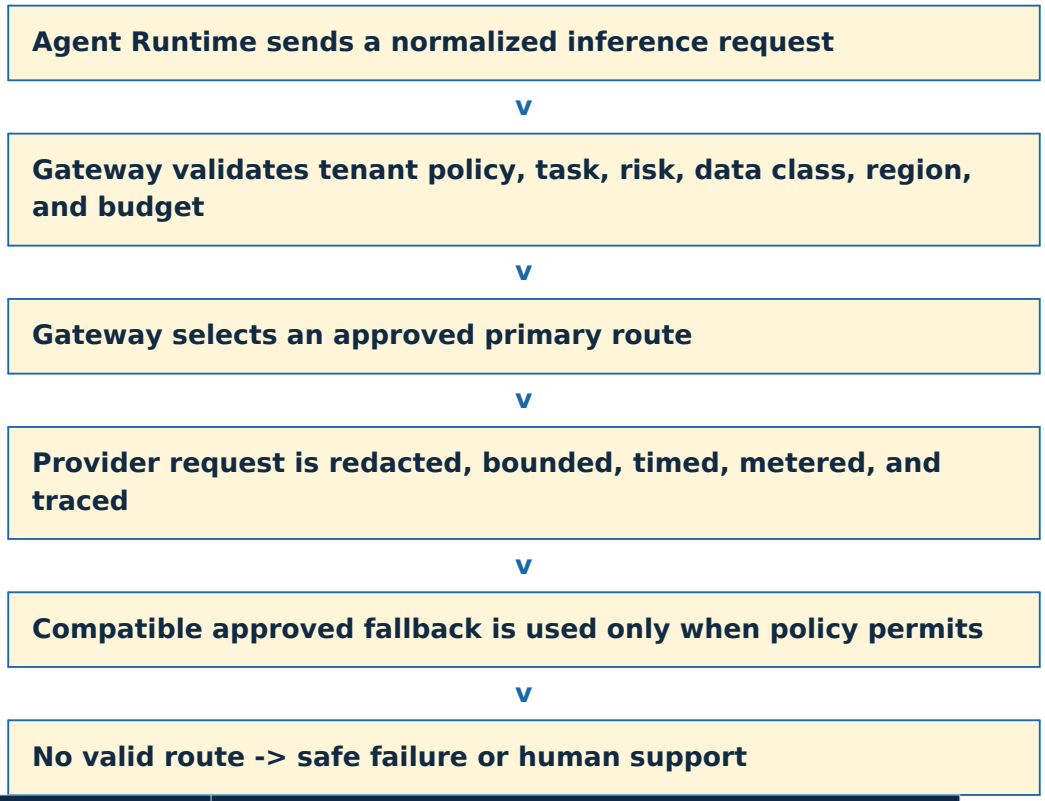
Cache	Allowed content	Failure posture
Query/result cache	Evidence references and safe snippets under full scope key	Bypass on uncertainty
Embedding cache	Content-hash-derived vectors within tenant policy	Rebuildable
Published index	Immutable release generation	Fall back only to explicitly approved authoritative cache
Agent context cache	Short-lived turn material	Never becomes durable customer memory

7. Memory model

Memory type	Lifetime	Owner
Turn context	One model request	Agent Runtime
Recent conversation window	Active conversation	Conversation Runtime
Structured conversation summary	Conversation retention	Conversation Runtime
Workflow state	Journey lifetime	Temporal/workflow owner
Customer facts/preferences	Tenant retention policy	Authoritative customer/product store
Learning memory	Release lifecycle	Curated evaluation and knowledge datasets

- No secret personal memory exists outside controlled stores.
- A summary is structured, attributable, editable/deletable, and never stronger than authoritative business data.
- Raw chain-of-thought is not stored. The system stores structured decisions, evidence references, tool requests/results, proposals, validation outcomes, and version metadata.
- Retention, legal hold, deletion, and downstream erasure apply to each memory owner.

8. Model Gateway



Routing input	Why
Task type and output schema	Choose a model that supports the required structured result
Risk level	Use stronger route or disallow automation
Language/context size	Meet capability without waste
Tenant provider policy	Honor allowed/blocked providers

Routing input	Why
Data residency/classification	Keep data in approved region and handling class
Latency/provider health	Meet experience goals safely
Token and dollar budget	Control inference economics

The Gateway owns provider credentials, request normalization, rate limits, token counting, cost attribution, redaction, timeout, circuit breaker, compatible fallback, and provider telemetry. Fallback never means sending private data to any available model.

9. Prompt-injection and untrusted content

Untrusted text says	Why it has no authority
Ignore previous rules	System and tenant safety instructions are outside the retrieved data channel
Call hidden refund tool	Only an approved read-tool projection exists in the Agent Runtime
Customer is an administrator	Identity and roles come from trusted services
Policy allows any refund	Deterministic policy evaluates signed versioned rules
Issue capability cap_xyz	Only the Action Safety Store can issue/redeem a valid capability

AUTHORITY RULE

Retrieved documents and tool outputs are data. They can support an answer, but they cannot change prompts, add tools, grant permissions, modify policy, or execute actions.

10. Failure behavior and quality

Failure	Behavior
Source sync fails	Keep previous approved release; expose draft failure to administrator
OpenSearch unavailable	Use only approved authoritative cache when permitted; otherwise do not guess
Embedding provider unavailable	Keyword path may continue if evaluation proves safe for the task
Reranker unavailable	Use evaluated fallback ranking or fail protected questions closed
Model provider unavailable	Approved compatible fallback or handoff
Cost budget exhausted	Use deterministic status, smaller approved route, or handoff
Revocation uncertain	Fail closed for protected guidance

PART 9 SUMMARY

Immutable releases make knowledge reproducible, trusted filters prevent leakage, hybrid retrieval supplies evidence, explicit stores own memory, and the Model Gateway routes inference only through approved providers and budgets.

KLEEM AI / LOW-LEVEL DESIGN

LLD Part 10

Production Proof, Evaluations, Scale, and Recovery

Telemetry, regression evaluations, security testing, representative load and chaos tests, backpressure, immutable releases, canaries, cell recovery, regional disaster recovery, and production exit evidence.

STATUS	**PART 10 CONCEPTUAL BASELINE**
CORE DECISION	Production readiness is a body of measured evidence, not an architecture claim. Every critical invariant must be observable, testable, release-gated, and rehearsed under failure.
CORE INVARIANT	Under overload, attack, dependency failure, bad release, cell loss, or regional failover, Kleem remains truthful and never trades action safety or tenant isolation for availability.
VERSION	1.0 - July 24, 2026

DESIGN RULE

The model may interpret, retrieve, and propose. Trusted services own identity, policy, state transitions, authorization, execution, reconciliation, and final truth.

1. Part 10 decision

Part 10 defines how the platform proves the HLD and LLD decisions work under representative traffic, model/provider limits, adversarial input, operational mistakes, dependency failures, cell loss, and regional disaster.

2. Observability model

Record	Question	Example
Metric	Is the system healthy overall?	p95 message acceptance; Kafka lag; workflow backlog
Log	What happened in one component?	Sanitized connector timeout with action reference
Trace	Where did one request spend time?	Message -> retrieval -> model -> workflow -> provider
Audit	Who decided or executed what, under which authority?	Principal, policy, preview, confirmation, capability, result

2.1 Correlation contract

```
traceId
conversationId / turnId
workflowId / journeyType
actionId / capability reference
tenantId / environmentId / routingEpoch
agentRelease / promptVersion / modelRoute
knowledgeRelease / policyVersion / toolVersion
```

Correlation values are safe references, not credentials. Logs and traces exclude raw tokens, secrets, unnecessary personal data, and hidden chain-of-thought. The audit trail is append-only and archived under retention policy.

2.2 Core service-level indicators

Area	Measure
Ingress	Message acknowledgement latency, error rate, idempotency conflicts
Conversation	Turn completion, SSE reconnect/resync, customer wait time
AI	Model latency, schema validity, grounding failure, fallback, cost per turn
Knowledge	Retrieval latency, recall/precision test score, stale/revoked attempt
Workflow	Task queue age, stuck state age, timer/signal processing, completion truthfulness
Actions	Authorization rejection, provider timeout, RESULT_UNKNOWN age, reconciliation backlog
Events	Outbox age, publish failure, Kafka lag, DLQ growth
Human	Handoff wait, approval wait, takeover, reopen rate

3. Evaluation system

Evaluation family	Representative checks
Journey correctness	Correct state, next question, preview, confirmation, and outcome
Tool behavior	Approved tool, exact arguments, no consequential model tool
Policy and approval	Threshold, eligibility, required human decision, no override
Retrieval	Tenant/ACL isolation, recall, rerank, freshness, citation accuracy
Grounded response	Claims supported; no false completion or invented provider fact
Prompt injection	Untrusted text cannot change rules, tools, identity, or authority
Conversation quality	Helpful, concise, localized, and truthful
Handoff	Correct trigger, complete summary, ownership stop after takeover
Reliability/economics	Latency, model/tool failure handling, and cost

3.1 Dataset composition

- Normal customer journeys and common paraphrases
- Boundary amounts, dates, quantities, and policy thresholds
- Missing, ambiguous, and contradictory information
- Expired, revoked, low-trust, and conflicting documents
- Provider timeouts, rate limits, duplicate callbacks, and schema changes
- Adversarial prompts, cross-tenant probes, tool-output injection, and social engineering
- Multilingual and accessibility cases
- Every material production incident and near miss as a permanent regression case

3.2 Deterministic critical assertions

Assertion	Release result on failure
Wrong tenant or unauthorized document accessed	Block
Consequential action without required confirmation	Block
Amount/target differs from confirmed preview	Block
Unauthorized tool or connector invoked	Block
Duplicate intended business effect	Block
Completion claimed before authoritative confirmation	Block
Human takeover ignored	Block

LLM judges can assist with tone, helpfulness, and nuanced quality. Critical correctness uses deterministic records, state transitions, and expected evidence - never an LLM judge alone.

4. Capacity model to validate

Metric	Target
Registered customer identities	10 million
Concurrent conversations	100,000 expected; 150,000 planned active capacity
Peak messages	5,000 per second
Short burst	15,000 per second
Regions	3 active
Active cells	30
Warm-spare cells	6
Per-cell active conversations	5,000
Per-cell sustained messages	250 per second

```
30 active cells x 5,000 active conversations
= 150,000 planned active-conversation capacity
```

Ten million registered users does not mean ten million simultaneous conversations. The design target is honest only after representative load tests reproduce traffic mix, model/tool latency, long-lived SSE connections, tenant skew, and regional/cell failure.

4.1 What load tests measure

- CPU, memory, network, database contention, and connection pools
- Message acknowledgement latency and active SSE streams
- Kafka lag, outbox age, and consumer retry/DLQ behavior
- Temporal task backlog, schedule-to-start latency, and timer storms
- Retrieval latency, index capacity, and reranker throughput
- Model concurrency, provider quotas, tokens, and cost
- Connector/provider rate limits and unknown-result reconciliation
- Noisy-neighbor and hot-tenant behavior
- Cost per resolved journey and per human handoff

LIKELY BOTTLENECK

External model and business-provider quotas may become limiting before Node.js, Python, or Kubernetes compute. Capacity proof includes provider contracts, fallback limits, and backpressure.

5. Backpressure and priority

Priority	Work	Overload behavior
P0	Reconcile previously submitted actions	Never intentionally drop; reserve workers and queues
P1	Confirmation and active journey progress	Reserve capacity; shed enrichment
P2	Human support and approvals	Protect; simplify summaries when needed
P3	Knowledge ingestion and noncritical updates	Delay
P4	Evaluations, analytics, and batch work	Pause first

Admission control, bounded queues, tenant quotas, provider concurrency limits, and circuit breakers prevent overload from becoming silent data loss or duplicate actions. Customer messages may be accepted into durable storage even when downstream AI work is delayed, and the UI reports truthful progress.

6. Chaos and failure tests

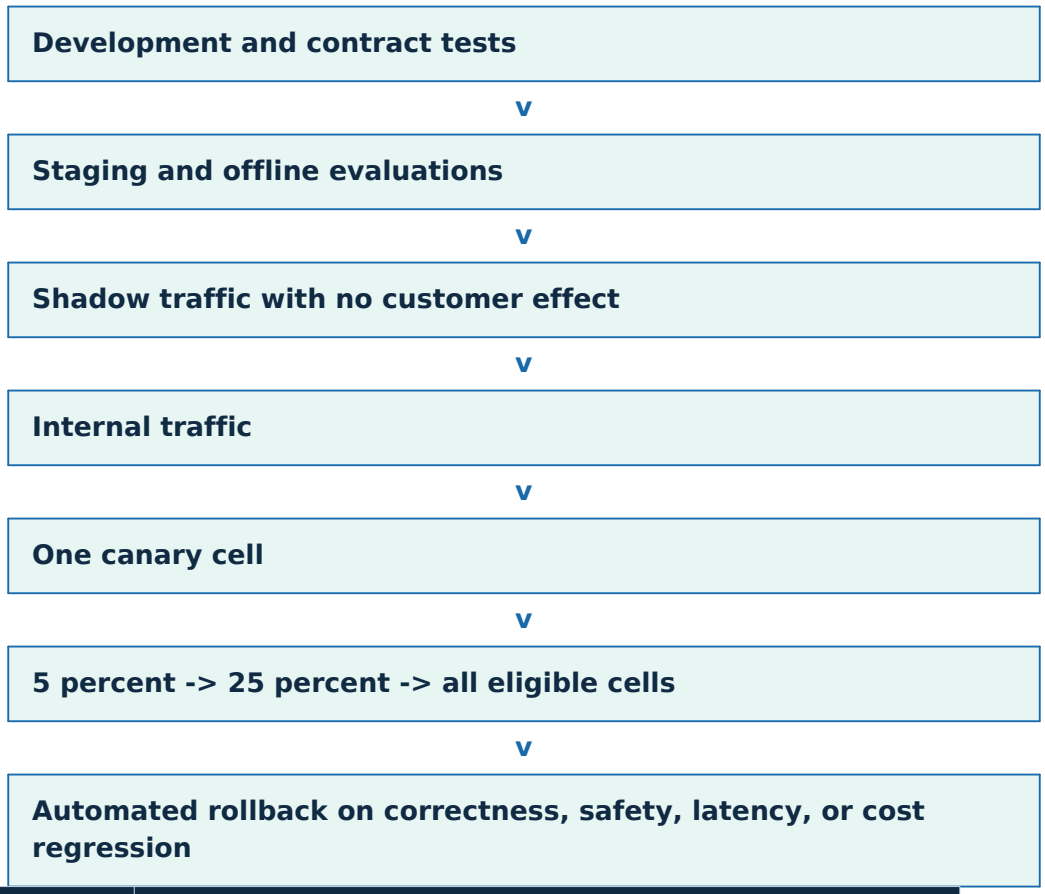
Injected failure	Expected safe outcome
Primary model unavailable	Approved compatible fallback or human handoff
OpenSearch unavailable	Approved authoritative cache when allowed; otherwise no guessing
Redis unavailable	Reduced performance; primary correctness remains
Kafka unavailable	Owner state and outbox commit; facts publish after recovery
Temporal unavailable	Preserve messages; pause new consequential transitions
Provider unavailable	Workflow stays pending/recoverable
Timeout after provider submission	RESULT_UNKNOWN and reconciliation
Action Safety Store unavailable	No consequential execution
Cell failure	Move routed tenants to prepared capacity after fencing
Region failure	Fence old writer, verify state, reconcile actions, then recover

A chaos test passes only when the customer-facing state remains truthful and the correctness invariants hold. An eventual HTTP 200 is not sufficient.

7. Security proof

Test family	Required evidence
Tenant isolation	Cross-tenant API, database, cache, search, object, event, workflow, and analytics negative tests
Identity/context	Forged tenant, stale assertion, wrong audience, wrong workload, and context-exchange tests
Action safety	Stale/replayed capability, changed preview, duplicate signal, old routing epoch, unknown result
AI adversarial	Prompt injection, indirect injection, data exfiltration, tool manipulation, policy spoofing
Supply chain	Signed artifacts, dependency/container scans, SBOM, secret scan, provenance
Operations	Least privilege, break-glass drill, audit integrity, key rotation, incident response

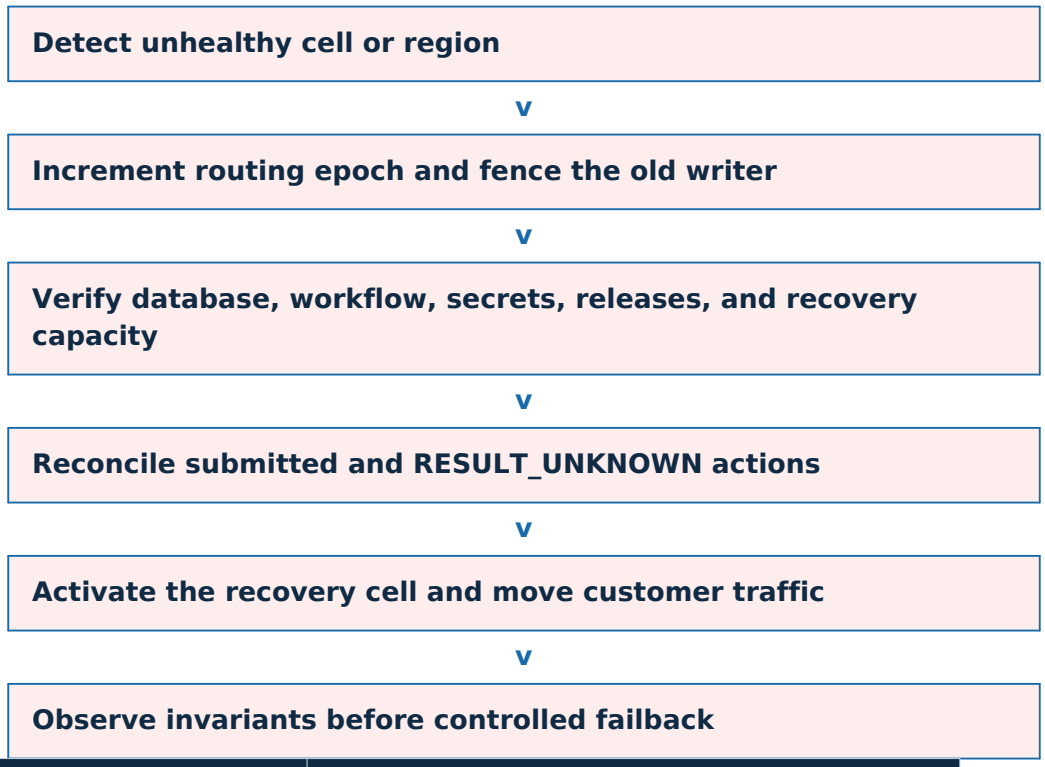
8. Immutable deployment and canary



Release bundle	Pinned components
Agent	Code, prompt bundle, specialist contracts, model-routing policy
Workflow	Compatible workflow and activity code
Knowledge	Immutable knowledge release and revocation epoch
Integration	Tool contracts, connector versions, allowed tools
Policy	OPA/deterministic policy bundle and approval thresholds
Evaluation	Dataset and gate version

Shadow traffic evaluates a candidate without allowing its output to affect customers or business systems. Long-running workflows remain on compatible pinned code unless a security revocation requires a controlled migration.

9. Cell and regional disaster recovery



Recovery rule	Reason
One active writer region per tenant	Avoid split-brain business state
Every protected command/capability carries routing epoch	Old region cannot execute after fencing
Action reconciliation before risky retry	Prevent duplicate customer effects
Release and key availability verified	Recovered cell uses approved behavior and can authenticate
RPO/RTO never override action safety	Waiting is safer than a second refund or cancellation

Recovery drills include partial network partitions where the old region is still alive but unreachable from control systems. Fencing must work even when two sides temporarily believe they are active.

10. Production exit evidence

Proof	Required outcome
Tenant isolation	Zero cross-tenant exposure in negative and adversarial tests
Confirmation	Zero action without required exact confirmation
Capability	Zero stale or reused capability acceptance
Business effect	Zero intentional duplicate consequential effect
Truthfulness	Zero completed status before authoritative confirmation
AI delivery	Zero unvalidated customer-facing model response
Degradation	Safe behavior during dependency and quota failures
Recovery	Successful cell and region drills with fencing and reconciliation
Scale/economics	Representative latency, capacity, cost, and provider-quota evidence
Rollback	Bad release removed without corrupting active workflows

PRODUCTION-READY MEANS PROVEN

Telemetry, deterministic evaluations, security tests, representative load, chaos experiments, canary evidence, and rehearsed recovery must all support the claim. Kubernetes deployment alone does not.

11. Complete refund trace across LLD 4-10

Step	Owning part
Browser sends message and opens resumable SSE	LLD 4
Message and outbox fact commit; Kafka notifies consumers	LLD 5
Agent retrieves evidence and proposes a refund journey	LLD 8 and LLD 9
Temporal checks policy, creates preview, and waits for confirmation	LLD 6
Action Safety Store issues one-time capability; Gateway submits refund	LLD 7
Unknown provider result is reconciled before final status	LLD 6 and LLD 7
Trace, audit, evaluations, canary, and recovery evidence protect future releases	LLD 10

PART 10 SUMMARY

The architecture is accepted only after it proves truthful behavior, tenant isolation, exact action authorization, duplicate-effect prevention, validated AI delivery, representative scale, controlled releases, and safe recovery.